Team Name

Member One, Member Two, Member Three

Your University – ICPC notebook 2026-10-18

1 Contest 2

template 2, .bashrc 2, stress 2, troubleshoot 2

2 Mathematics 2

Equations 2
Recurrences 2
Trigonometry 2
Geometry 2: *Triangles* 2, *Quadrilaterals* 3, *Spherical coordinates* 3, *Polygons and polyhedra* 3, *Volumes and areas* 3, *Hyperspheres* 3, *Rotation about an axis* 3
Derivatives/Integrals 3
Sums 3
Series 3
Probability theory 3: *Discrete distributions* 4, *Continuous distributions* 4
Markov chains 4
Table of integrals 4

3 Data structures 6

Range queries 6: SegmentTree 6, LazySegmentTree 6, PersistentSegmentTree 7, FenwickTree 7, FenwickTree2d 7, RMQ 7, SubMatrix 7, MoQueries 8
Sets, maps, sequences 8: OrderStatisticTree 8, HashMap 8, Treap 8
Union-find 8: UnionFind 8, UnionFindRollback 9
Lines (convex hull trick) 9: LineContainer 9, LiChao 9
Bits and matrices 9: XorBasis 9, PersistentXorTrie 9, Matrix 10

4 Numerical 10

Polynomials and recurrences 10: Polynomial 10, PolyRoots 10, PolyInterpolate 10, BerlekampMassey 10, LinearRecurrence 10
Optimization 11: GoldenSectionSearch 11, Integrate 11, IntegrateAdaptive 11, Simplex 11
Matrices 11: Determinant 11, IntDeterminant 12, SolveLinear 12, SolveLinear2 12, SolveLinearBinary 12, SolveLinearMod 12, MatrixInverse 13, Tridiagonal 13
Fourier transforms 13: FastFourierTransform 13, FastFourierTransformMod 13, NumberTheoreticTransform 13, FastSubsetTransform 14
Formal power series 14: PolyOps 14, MultipointEval 14

5 Number theory 15

Modular arithmetic 15: ModularArithmetic 15, ModInverse 15, ModPow 15, ModLog 15, ModSum 15, ModMulLL 15, ModSqrt 15, PrimitiveRoot 15
Primality 16: FastEratosthenes 16, LinearSieve 16, MillerRabin 16, Factor 16
Divisibility 16: euclid 16, CRT 16, *Bézout's identity* 16, phiFunction 16
Fractions 16: ContinuedFractions 16, FracBinarySearch 17
Pythagorean Triples 17
Theorems 17
Primes 17
Estimates 17
Mobius Function 17

6 Combinatorial 17

Permutations 17: *Factorial* 17, IntPerm 17, *Cycles* 17, *Derangements* 17, *Burnside's lemma* 18
Partitions and subsets 18: *Partition function* 18, *Lucas' Theorem* 18, *Binomials* 18, Binomial 18, multinomial 18
General purpose numbers 18: *Bernoulli numbers* 18, *Stirling numbers of the first kind* 18, *Eulerian numbers* 18, *Stirling numbers of the second kind* 18, *Bell numbers* 18, *Labeled unrooted trees* 18, *Catalan numbers* 18

7 Graph 19

Fundamentals 19: BellmanFord 19, FloydWarshall 19, TopoSort 19, MinMeanCycle 19, SteinerTree 19
Network flow 19: PushRelabel 19, Dinic 20, MinCostMaxFlow 20, MinCut 20, GlobalMinCut 20, GomoryHu 21
Matching 21: HopcroftKarp 21, DFSMatching 21, MinimumVertexCover 21, WeightedMatching 21, GeneralMatching 21
DFS algorithms 22: SCC 22, BiconnectedComponents 22, 2sat 22, EulerWalk 23, DominatorTree 23, DynamicConnectivity 23
Coloring 23: EdgeColoring 23
Trees 24: BinaryLifting 24, LCA 24, CompressTree 24, HLD 24, CentroidDecomposition 24, LinkCutTree 25, DirectedMST 25
Math 25: *Number of Spanning Trees* 26, *Erdős–Gallai theorem* 26, *Matchings, covers, flows* 26

8 Geometry 26

Geometric primitives 26: Point 26, lineDistance 26, SegmentDistance 26, SegmentIntersection 26, lineIntersection 26, sideOf 26, OnSegment 26, linearTransformation 27, LineProjectionReflection 27, Angle 27
Circles 27: CircleIntersection 27, CircleTangents 27, CircleLine 27, CirclePolygonIntersection 27, circumcircle 27, MinimumEnclosingCircle 27, CircleUnion 28
Polygons 28: InsidePolygon 28, PolygonArea 28, PolygonCenter 28, PolygonCut 28, PolygonUnion 28, ConvexHull 28, HullDiameter 29, PointInsideHull 29, LineHullIntersection 29, HalfPlane 29, MinkowskiSum 29
Misc. Point Set Problems 29: ClosestPair 29, ManhattanMST 30, kdTree 30, FastDelaunay 30, PlanarFaces 31
3D 31: Point3D 31, 3dHull 31, PolyhedronVolume 31, sphericalDistance 31

9 Strings 32

KMP 32, Zfunc 32, Manacher 32, MinRotation 32, Lyndon 32, SuffixArray 32, SuffixAutomaton 32, Hashing 33, AhoCorasick 33, PalindromicTree 33, BitsetLCS 34, CyclicLCS 34

10 Various 34

Intervals 34: IntervalContainer 34, IntervalCover 34
Misc. algorithms 34: TernarySearch 34, LIS 34, FastKnapsack 35
Dynamic programming 35: KnuthDP 35, DivideAndConquerDP 35, SMAWK 35, SlopeTrick 35, *Aliens trick (WQS binary search)* 35, *Sum over subsets, other tricks* 35
Debugging tricks 36
Optimization tricks 36: *Bit hacks* 36, *Pragmas* 36, FastMod 36, FastInput 36

11 Tips & tricks 36

Constraints → complexity 36
If you see ... think of ... 36
C++ pitfalls 37
Contest strategy 37

Quick guide

- Range query + point update: SegmentTree **6**, FenwickTree **7**; range update: LazySegmentTree **6**; static min: RMQ **7**
- k-th smallest / count in a range, old versions: PersistentSegmentTree **7**; offline ranges: MoQueries **8**
- Order statistics: OrderStatisticTree **8**; split/merge/reverse sequences: Treap **8**
- Convex hull trick: LineContainer **9**, LiChao **9**; other DP speedups: DivideAndConquerDP **35**, KnuthDP **35**, SMAWK **35**, SlopeTrick **35**, Aliens **35**
- Undo / edge deletions: UnionFindRollback **9**, DynamicConnectivity **23**
- Negative weights: BellmanFord **19**, FloydWarshall **19**; terminals: SteinerTree **19**
- Max flow: Dinic **20**, PushRelabel **19**; with costs: MinCostMaxFlow **20**; cuts: GlobalMinCut **20**, GomoryHu **21**; modelling: König/flows **26**
- Matching: HopcroftKarp **21**, weighted WeightedMatching **21**, general GeneralMatching **21**
- SCC, 2-SAT, bridges: SCC **22**, 2sat **22**, BiconnectedComponents **22**; Euler path EulerWalk **23**; dominators DominatorTree **23**
- Tree paths: LCA **24**, HLD **24**, CentroidDecomposition **24**; changing forest: LinkCutTree **25**; subset of vertices: CompressTree **24**
- Convolution: FastFourierTransform **13**, NumberTheoreticTransform **13**, any mod FastFourierTransformMod **13**; xor/and/or: FastSubsetTransform **14**; series inv/log/exp: PolyOps **14**
- Recurrences: BerlekampMassey **10** + LinearRecurrence **10**, Matrix **10**
- Linear systems: SolveLinear **12**, mod p SolveLinearMod **12**, mod 2 SolveLinearBinary **12**, xor basis XorBasis **9**; LP: Simplex **11**
- Number theory: MillerRabin **16**, Factor **16**, LinearSieve **16**, CRT **16**, ModLog **15**, ModSqrt **15**, PrimitiveRoot **15**; $\binom{n}{k}$: Binomial **18**, identities **18**
- Strings: KMP **32**, Zfunc **32**, AhoCorasick **33**, SuffixArray **32**, SuffixAutomaton **32**, Hashing **33**; palindromes Manacher **32**, PalindromicTree **33**
- Geometry: ConvexHull **28**, InsidePolygon **28**, PointInsideHull **29**, HalfPlane **29**, MinkowskiSum **29**, PolygonUnion **28**, CircleUnion **28**, ClosestPair **29**, FastDelaunay **30**, PlanarFaces **31**; lattice points: Pick **3**

Contest (1)

template.cpp

20 lines

```
#include <bits/stdc++.h>

using namespace std;

using ll = long long;
#define int long long

void solve(int TEST) {}

int32_t main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);

    int t = 1;
    cin >> t;
    while (t--) {
        solve(t);
    }
}
```

.bashrc

3 lines

```
alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++17 \
  -fsanitize=undefined,address'
xmodmap -e 'clear lock' -e 'keycode 66=less greater' #caps = <>
```

stress.sh

6 lines

```
# ./stress.sh sol brute gen [iters]   (gen.cpp reads seed argv[1]:
#   mt19937 rng(atoi(argv[1])); print a small random test)
for i in $(seq 1 ${4:-500}); do
  ./$3 $i > in; ./$1 < in > o1; ./$2 < in > o2
  cmp -s o1 o2 || { echo "diff on seed $i"; cat in; break; }
done
```

troubleshoot.txt

52 lines

```
Pre-submit:
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Make sure to submit the right file.

Wrong answer:
Print your solution! Print debug output, as well.
Are you clearing all data structures between test cases?
Can your algorithm handle the whole range of input?
Read the full problem statement again.
Do you handle all corner cases correctly?
Have you understood the problem correctly?
Any uninitialized variables?
Any overflows?
Confusing N and M, i and j, etc.?
Are you sure your algorithm works?
What special cases have you not thought of?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Is your output format correct? (including whitespace)
Rewrite your solution from the start or let a teammate do it.

Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?
```

Mathematics (2)

2.1 Equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by $x = -b/2a$. Numerically stable: $q = -\frac{1}{2}(b + \text{sgn}(b)\sqrt{b^2 - 4ac})$, $x_1 = q/a$, $x_2 = c/q$.

$$\begin{matrix} ax + by = e \\ cx + dy = f \end{matrix} \Rightarrow \begin{matrix} x = \frac{ed - bf}{ad - bc} \\ y = \frac{af - ec}{ad - bc} \end{matrix}$$

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A_i'}{\det A}$$

where A_i' is A with the i'th column replaced by b.

2.2 Recurrences

If $a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k}$, and $r_1, \ldots, r_k$ are distinct roots of $x^k - c_1 x^{k-1} - \cdots - c_k$, there are $d_1, \ldots, d_k$ s.t.

$$a_n = d_1 r_1^n + \cdots + d_k r_k^n.$$

Non-distinct roots r become polynomial factors, e.g. $a_n = (d_1 n + d_2) r^n$.

Matrix form: $(a_n, \ldots, a_{n-k+1})^T = M(a_{n-1}, \ldots, a_{n-k})^T$ with companion matrix $M =$ first row $c_1 \ldots c_k$, ones on the subdiagonal, so a_n comes from M^n in $O(k^3 \log n)$ (Matrix.h). By Cayley–Hamilton $M^n \equiv x^n \bmod (x^k - c_1 x^{k-1} - \cdots - c_k)$: $O(k^2 \log n)$ (LinearRecurrence.h). Inhomogeneous terms ($+p(n)$, $+q^n$): add them as extra state.

2.3 Trigonometry

$$\begin{aligned}
\sin(v + w) &= \sin v \cos w + \cos v \sin w \\
\cos(v + w) &= \cos v \cos w - \sin v \sin w \\
\tan(v + w) &= \frac{\tan v + \tan w}{1 - \tan v \tan w} \\
\sin v + \sin w &= 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2} \\
\cos v + \cos w &= 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2} \\
\sin v \sin w &= \tfrac{1}{2}(\cos(v - w) - \cos(v + w)) \\
\cos v \cos w &= \tfrac{1}{2}(\cos(v - w) + \cos(v + w)) \\
\sin v \cos w &= \tfrac{1}{2}(\sin(v + w) + \sin(v - w))
\end{aligned}$$

$\sin 2x = 2 \sin x \cos x$, $\cos 2x = \cos^2 x - \sin^2 x = 1 - 2\sin^2 x$, $\sin^2 x = \frac{1 - \cos 2x}{2}$. With $t = \tan \frac{x}{2}$: $\sin x = \frac{2t}{1+t^2}$, $\cos x = \frac{1-t^2}{1+t^2}$, $dx = \frac{2\,dt}{1+t^2}$.

$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w.

$$\begin{aligned}
a \cos x + b \sin x &= r \cos(x - \phi) \\
a \sin x + b \cos x &= r \sin(x + \phi)
\end{aligned}$$

where $r = \sqrt{a^2 + b^2}, \phi = \text{atan2}(b, a)$.

2.4 Geometry

2.4.1 Triangles

Side lengths: a, b, c

Semiperimeter: $p = \frac{a + b + c}{2}$

Area: $A = \sqrt{p(p - a)(p - b)(p - c)} = \frac{1}{2} ab \sin \gamma$

Circumradius: $R = \frac{abc}{4A}$

Inradius: $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2+2c^2-a^2}$
Length of bisector (divides angles in two):
$s_a = \sqrt{bc\left[1-\left(\frac{a}{b+c}\right)^2\right]}$
Law of sines: $\frac{\sin\alpha}{a} = \frac{\sin\beta}{b} = \frac{\sin\gamma}{c} = \frac{1}{2R}$
Law of cosines: $a^2 = b^2+c^2-2bc\cos\alpha$
Law of tangents: $\frac{a+b}{a-b} = \frac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$
Incenter $= (aA+bB+cC)/(a+b+c)$ (vertices weighted by opposite sides); centroid $= (A+B+C)/3$; Euler line: $OH = 3\,OG$.

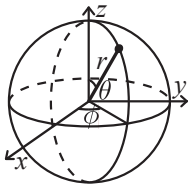
2.4.2 Quadrilaterals

With side lengths a, b, c, d, diagonals e, f, diagonals angle θ, area A and magic flux $F = b^2+d^2-a^2-c^2$:

$$4A = 2ef\cdot\sin\theta = F\tan\theta = \sqrt{4e^2f^2-F^2}$$

For cyclic quadrilaterals the sum of opposite angles is $180°$, $ef = ac+bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$.

2.4.3 Spherical coordinates



$$\begin{array}{cc} x = r\sin\theta\cos\phi & r = \sqrt{x^2+y^2+z^2} \\ y = r\sin\theta\sin\phi & \theta = \operatorname{acos}(z/\sqrt{x^2+y^2+z^2}) \\ z = r\cos\theta & \phi = \operatorname{atan2}(y,x) \end{array}$$

2.4.4 Polygons and polyhedra

Shoelace: $2A = \sum_i (x_i y_{i+1} - x_{i+1}y_i)$ (PolygonArea.h).
Pick: lattice polygon: $A = I + \frac{B}{2} - 1$, I = interior and B = boundary lattice points; segment (x_1,y_1)–(x_2,y_2) has $\gcd(|x_1-x_2|, |y_1-y_2|)+1$ lattice points.
Euler: convex polyhedron $V-E+F = 2$; planar graph with C components: $V-E+F = 1+C$ (outer face counted); simple planar graph has $E \le 3V-6$.

2.4.5 Volumes and areas

Pyramid/cone $V = \frac{1}{3}Sh$. Frustum $V = \frac{1}{3}h(S_1+\sqrt{S_1S_2}+S_2)$.
Sphere $V = \frac{4}{3}\pi R^3$, $S = 4\pi R^2$. Cap of height h: $V = \frac{\pi h^2}{3}(3R-h)$, $S = 2\pi Rh$. Ellipsoid $V = \frac{4}{3}\pi abc$. Cone lateral $S = \pi r\sqrt{r^2+h^2}$.
Tetrahedron $V = \frac{1}{6}|(b-a)\cdot((c-a)\times(d-a))|$. From edges: O-ABC with a=AB, b=BC, c=CA, d=OC, e=OA, f=OB:

$$\begin{aligned}(12V)^2 = {} & a^2d^2(b^2+c^2+e^2+f^2-a^2-d^2) \\ & + b^2e^2(c^2+a^2+f^2+d^2-b^2-e^2) \\ & + c^2f^2(a^2+b^2+d^2+e^2-c^2-f^2) \\ & - a^2b^2c^2 - a^2e^2f^2 - d^2b^2f^2 - d^2e^2c^2\end{aligned}$$

2.4.6 Hyperspheres

n-ball volume $V_n = \frac{\pi^{n/2}}{\Gamma(n/2+1)}R^n$, surface $S_{n-1} = \frac{n}{R}V_n$;
$V_n = \frac{2\pi R^2}{n}V_{n-2}$, $V_1 = 2R$, $V_2 = \pi R^2$, $V_4 = \frac{\pi^2}{2}R^4$, $V_5 = \frac{8\pi^2}{15}R^5$, $V_6 = \frac{\pi^3}{6}R^6$.

2.4.7 Rotation about an axis

Rotation by θ around unit axis A, $c = \cos\theta$, $s = \sin\theta$:

$$\begin{pmatrix} c+(1-c)A_x^2 & (1-c)A_xA_y - sA_z & (1-c)A_xA_z + sA_y \\ (1-c)A_xA_y + sA_z & c+(1-c)A_y^2 & (1-c)A_yA_z - sA_x \\ (1-c)A_xA_z - sA_y & (1-c)A_yA_z + sA_x & c+(1-c)A_z^2 \end{pmatrix}$$

Rodrigues: $v' = vc + (A\times v)s + A(A\cdot v)(1-c)$
(P3::rotate).

2.5 Derivatives/Integrals

$$\begin{array}{cc} \frac{d}{dx}\arcsin x = \frac{1}{\sqrt{1-x^2}} & \frac{d}{dx}\arccos x = -\frac{1}{\sqrt{1-x^2}} \\ \frac{d}{dx}\tan x = 1+\tan^2 x & \frac{d}{dx}\arctan x = \frac{1}{1+x^2} \\ \int \tan ax = -\frac{\ln|\cos ax|}{a} & \int x\sin ax = \frac{\sin ax - ax\cos ax}{a^2} \\ \int e^{-x^2} = \frac{\sqrt{\pi}}{2}\operatorname{erf}(x) & \int xe^{ax}dx = \frac{e^{ax}}{a^2}(ax-1) \end{array}$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

Definite: $\int_0^\infty x^n e^{-x}dx = n! = \Gamma(n+1)$, $\int_0^\infty e^{-ax^2}dx = \frac{1}{2}\sqrt{\pi/a}$, $\int_0^1 x^a(1-x)^b dx = \frac{a!\,b!}{(a+b+1)!}$, $\int_0^{\pi/2}\sin^n x\,dx = \frac{n-1}{n}\int_0^{\pi/2}\sin^{n-2}x\,dx$.
Simpson: $\int_a^b f \approx \frac{b-a}{6}(f(a)+4f(\frac{a+b}{2})+f(b))$ (Integrate.h). More: see the table of integrals at the end of this chapter.

2.6 Sums

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1}-c^a}{c-1}, c \ne 1$$

$$\begin{aligned} 1+2+3+\cdots+n &= \frac{n(n+1)}{2} \\ 1^2+2^2+3^2+\cdots+n^2 &= \frac{n(2n+1)(n+1)}{6} \\ 1^3+2^3+3^3+\cdots+n^3 &= \frac{n^2(n+1)^2}{4} \\ 1^4+2^4+3^4+\cdots+n^4 &= \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30} \\ 1^5+2^5+\cdots+n^5 &= \frac{n^6}{6}+\frac{n^5}{2}+\frac{5n^4}{12}-\frac{n^2}{12} \\ 1^6+2^6+\cdots+n^6 &= \frac{n^7}{7}+\frac{n^6}{2}+\frac{n^5}{2}-\frac{n^3}{6}+\frac{n}{42} \end{aligned}$$

$\sum_{i=1}^n i^k$ is a polynomial of degree $k+1$: interpolate from $k+2$ values (or Bernoulli numbers, Combinatorial).
$\sum_{k=1}^n k(k+1)\cdots(k+r-1) = \frac{n(n+1)\cdots(n+r)}{r+1}$.
$\sum_{k=0}^n kx^k = \frac{x-(n+1)x^{n+1}+nx^{n+2}}{(1-x)^2}$.
$H_n = \sum_{k\le n}\frac{1}{k} \approx \ln n + 0.5772 + \frac{1}{2n}$.

2.7 Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots, (-\infty < x < \infty)$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, (-1 < x \le 1)$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \ldots, (-1 \le x \le 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots, (-\infty < x < \infty)$$

$$\frac{1}{(1-x)^k} = \sum_{n\ge 0}\binom{n+k-1}{k-1}x^n, \quad \arctan x = x - \frac{x^3}{3} + \frac{x^5}{5} - \ldots$$

2.8 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x. It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x xp_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x-\mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear (even for dependent variables):

$$\mathbb{E}(aX+bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y,

$$V(aX+bY) = a^2V(X) + b^2V(Y).$$

For $X \in \mathbb{N}$: $\mathbb{E}(X) = \sum_{k\geq 1} P(X \geq k)$. Expected maximum of n uniform $[0,1]$: $\frac{n}{n+1}$. Coupon collector: nH_n draws.

2.8.1 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\mathrm{Bin}(n,p)$, $n = 1,2,\ldots$, $0 \leq p \leq 1$.

$$p(k) = \binom{n}{k} p^k (1-p)^{n-k}$$

$$\mu = np,\ \sigma^2 = np(1-p)$$

$\mathrm{Bin}(n,p)$ is approximately $\mathrm{Po}(np)$ for small p.

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $\mathrm{Fs}(p)$, $0 \leq p \leq 1$.

$$p(k) = p(1-p)^{k-1},\ k = 1,2,\ldots$$

$$\mu = \frac{1}{p},\ \sigma^2 = \frac{1-p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $\mathrm{Po}(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda}\frac{\lambda^k}{k!}, k = 0,1,2,\ldots$$

$$\mu = \lambda,\ \sigma^2 = \lambda$$

2.8.2 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $\mathrm{U}(a,b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a+b}{2},\ \sigma^2 = \frac{(b-a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $\mathrm{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda},\ \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu,\sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1,\sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2,\sigma_2^2)$ then

$$aX_1 + bX_2 + c \sim \mathcal{N}(a\mu_1 + b\mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

2.9 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. Let $X_1, X_2, \ldots$ be a sequence of random variables generated by the Markov process. Then there is a transition matrix $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, and $\mathbf{p}^{(n)} = \mathbf{P}^n\mathbf{p}^{(0)}$ is the probability distribution for X_n (i.e., $p_i^{(n)} = \Pr(X_n = i)$), where $\mathbf{p}^{(0)}$ is the initial distribution.

π is a stationary distribution if $\pi = \pi\mathbf{P}$. If the Markov chain is *irreducible* (it is possible to get to any state from any state), then $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state i. π_j/π_i is the expected number of visits in state j between two visits in state i.

For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, π_i is proportional to node i's degree.

A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k\to\infty} \mathbf{P}^k = \mathbf{1}\pi$.

A Markov chain is an A-chain if the states can be partitioned into two sets $\mathbf{A}$ and $\mathbf{G}$, such that all states in $\mathbf{A}$ are absorbing ($p_{ii} = 1$), and all states in $\mathbf{G}$ leads to an absorbing state in $\mathbf{A}$. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is j, is $a_{ij} = p_{ij} + \sum_{k\in\mathbf{G}} a_{ik}p_{kj}$. The expected time until absorption, when the initial state is i, is $t_i = 1 + \sum_{k\in\mathbf{G}} p_{ki}t_k$. Expected-value DPs on graphs with cycles: write one linear equation per state and use SolveLinear (Gaussian elimination).

2.10 Table of integrals

$$\int \frac{\mathrm{d}x}{ax+b} = \frac{1}{a}\ln|ax+b|$$
$$\int (ax+b)^\mu\,\mathrm{d}x = \frac{(ax+b)^{\mu+1}}{a(\mu+1)}\ (\mu \neq -1)$$
$$\int \frac{x\,\mathrm{d}x}{ax+b} = \frac{1}{a^2}(ax+b-b\ln|ax+b|)$$
$$\int \frac{x^2\,\mathrm{d}x}{ax+b} = \frac{1}{a^3}\left(\tfrac{1}{2}(ax+b)^2 - 2b(ax+b) + b^2\ln|ax+b|\right)$$
$$\int \frac{\mathrm{d}x}{x(ax+b)} = -\frac{1}{b}\ln\left|\frac{ax+b}{x}\right|$$
$$\int \frac{\mathrm{d}x}{x^2(ax+b)} = -\frac{1}{bx} + \frac{a}{b^2}\ln\left|\frac{ax+b}{x}\right|$$
$$\int \frac{x\,\mathrm{d}x}{(ax+b)^2} = \frac{1}{a^2}\left(\ln|ax+b| + \frac{b}{ax+b}\right)$$
$$\int \frac{x^2\,\mathrm{d}x}{(ax+b)^2} = \frac{1}{a^3}\left(ax+b-2b\ln|ax+b| - \frac{b^2}{ax+b}\right)$$
$$\int \frac{\mathrm{d}x}{x(ax+b)^2} = \frac{1}{b(ax+b)} - \frac{1}{b^2}\ln\left|\frac{ax+b}{x}\right|$$

$\sqrt{ax+b}$

$$\int \sqrt{ax+b}\,\mathrm{d}x = \frac{2}{3a}\sqrt{(ax+b)^3}$$
$$\int x\sqrt{ax+b}\,\mathrm{d}x = \frac{2}{15a^2}(3ax-2b)\sqrt{(ax+b)^3}$$
$$\int x^2\sqrt{ax+b}\,\mathrm{d}x = \frac{2}{105a^3}(15a^2x^2 - 12abx + 8b^2)\sqrt{(ax+b)^3}$$
$$\int \frac{x\,\mathrm{d}x}{\sqrt{ax+b}} = \frac{2}{3a^2}(ax-2b)\sqrt{ax+b}$$
$$\int \frac{x^2\,\mathrm{d}x}{\sqrt{ax+b}} = \frac{2}{15a^3}(3a^2x^2 - 4abx + 8b^2)\sqrt{ax+b}$$
$$\int \frac{\mathrm{d}x}{x\sqrt{ax+b}} = \begin{cases} \frac{1}{\sqrt{b}}\ln\left|\frac{\sqrt{ax+b}-\sqrt{b}}{\sqrt{ax+b}+\sqrt{b}}\right| & (b>0) \\ \frac{2}{\sqrt{-b}}\arctan\sqrt{\frac{ax+b}{-b}} & (b<0) \end{cases}$$
$$\int \frac{\mathrm{d}x}{x^2\sqrt{ax+b}} = -\frac{\sqrt{ax+b}}{bx} - \frac{a}{2b}\int \frac{\mathrm{d}x}{x\sqrt{ax+b}}$$
$$\int \frac{\sqrt{ax+b}}{x}\,\mathrm{d}x = 2\sqrt{ax+b} + b\int \frac{\mathrm{d}x}{x\sqrt{ax+b}}$$
$$\int \frac{\sqrt{ax+b}}{x^2}\,\mathrm{d}x = -\frac{\sqrt{ax+b}}{x} + \frac{a}{2}\int \frac{\mathrm{d}x}{x\sqrt{ax+b}}$$

$x^2 \pm a^2$, ax^2+b $(a>0)$

$$\int \frac{\mathrm{d}x}{x^2+a^2} = \frac{1}{a}\arctan\frac{x}{a}$$
$$\int \frac{\mathrm{d}x}{x^2-a^2} = \frac{1}{2a}\ln\left|\frac{x-a}{x+a}\right|$$
$$\int \frac{\mathrm{d}x}{(x^2+a^2)^n} = \frac{x}{2(n-1)a^2(x^2+a^2)^{n-1}} + \frac{2n-3}{2(n-1)a^2}\int \frac{\mathrm{d}x}{(x^2+a^2)^{n-1}}$$

$$\int \frac{\mathrm{d}x}{ax^2+b} = \begin{cases} \frac{1}{\sqrt{ab}} \arctan\left(\sqrt{\frac{a}{b}}\,x\right) & (b>0) \\ \frac{1}{2\sqrt{-ab}} \ln\left|\frac{\sqrt{a}x-\sqrt{-b}}{\sqrt{a}x+\sqrt{-b}}\right| & (b<0) \end{cases}$$

$$\int \frac{x\,\mathrm{d}x}{ax^2+b} = \frac{1}{2a}\ln\left|ax^2+b\right|$$

$$\int \frac{x^2\,\mathrm{d}x}{ax^2+b} = \frac{x}{a} - \frac{b}{a}\int \frac{\mathrm{d}x}{ax^2+b}$$

$$\int \frac{\mathrm{d}x}{x(ax^2+b)} = \frac{1}{2b}\ln\frac{x^2}{|ax^2+b|}$$

$$\int \frac{\mathrm{d}x}{x^2(ax^2+b)} = -\frac{1}{bx} - \frac{a}{b}\int \frac{\mathrm{d}x}{ax^2+b}$$

$$\int \frac{\mathrm{d}x}{x^3(ax^2+b)} = \frac{a}{2b^2}\ln\frac{|ax^2+b|}{x^2} - \frac{1}{2bx^2}$$

$$\int \frac{\mathrm{d}x}{(ax^2+b)^2} = \frac{x}{2b(ax^2+b)} + \frac{1}{2b}\int \frac{\mathrm{d}x}{ax^2+b}$$

ax^2+bx+c $(a>0)$

$$\int \frac{\mathrm{d}x}{ax^2+bx+c} = \begin{cases} \frac{2}{\sqrt{4ac-b^2}} \arctan\frac{2ax+b}{\sqrt{4ac-b^2}} & (b^2<4ac) \\ \frac{1}{\sqrt{b^2-4ac}} \ln\left|\frac{2ax+b-\sqrt{b^2-4ac}}{2ax+b+\sqrt{b^2-4ac}}\right| & (b^2>4ac) \end{cases}$$

$$\int \frac{x\,\mathrm{d}x}{ax^2+bx+c} = \frac{1}{2a}\ln|ax^2+bx+c| - \frac{b}{2a}\int \frac{\mathrm{d}x}{ax^2+bx+c}$$

$\sqrt{x^2+a^2}$ $(a>0)$

$$\int \frac{\mathrm{d}x}{\sqrt{x^2+a^2}} = \ln(x+\sqrt{x^2+a^2}) = \operatorname{arsinh}\frac{x}{a} + C$$

$$\int \frac{\mathrm{d}x}{\sqrt{(x^2+a^2)^3}} = \frac{x}{a^2\sqrt{x^2+a^2}}$$

$$\int \frac{x\,\mathrm{d}x}{\sqrt{x^2+a^2}} = \sqrt{x^2+a^2}$$

$$\int \frac{x\,\mathrm{d}x}{\sqrt{(x^2+a^2)^3}} = -\frac{1}{\sqrt{x^2+a^2}}$$

$$\int \frac{x^2\,\mathrm{d}x}{\sqrt{x^2+a^2}} = \frac{x}{2}\sqrt{x^2+a^2} - \frac{a^2}{2}\ln(x+\sqrt{x^2+a^2})$$

$$\int \frac{x^2\,\mathrm{d}x}{\sqrt{(x^2+a^2)^3}} = -\frac{x}{\sqrt{x^2+a^2}} + \ln(x+\sqrt{x^2+a^2})$$

$$\int \frac{\mathrm{d}x}{x\sqrt{x^2+a^2}} = \frac{1}{a}\ln\frac{\sqrt{x^2+a^2}-a}{|x|}$$

$$\int \frac{\mathrm{d}x}{x^2\sqrt{x^2+a^2}} = -\frac{\sqrt{x^2+a^2}}{a^2x}$$

$$\int \sqrt{x^2+a^2}\,\mathrm{d}x = \frac{x}{2}\sqrt{x^2+a^2} + \frac{a^2}{2}\ln(x+\sqrt{x^2+a^2})$$

$$\int \sqrt{(x^2+a^2)^3}\,\mathrm{d}x = \frac{x}{8}(2x^2+5a^2)\sqrt{x^2+a^2} + \frac{3}{8}a^4\ln(x+\sqrt{x^2+a^2})$$

$$\int x\sqrt{x^2+a^2}\,\mathrm{d}x = \frac{1}{3}\sqrt{(x^2+a^2)^3}$$

$$\int x^2\sqrt{x^2+a^2}\,\mathrm{d}x = \frac{x}{8}(2x^2+a^2)\sqrt{x^2+a^2} - \frac{a^4}{8}\ln(x+\sqrt{x^2+a^2})$$

$$\int \frac{\sqrt{x^2+a^2}}{x}\,\mathrm{d}x = \sqrt{x^2+a^2} + a\ln\frac{\sqrt{x^2+a^2}-a}{|x|}$$

$$\int \frac{\sqrt{x^2+a^2}}{x^2}\,\mathrm{d}x = -\frac{\sqrt{x^2+a^2}}{x} + \ln(x+\sqrt{x^2+a^2})$$

$\sqrt{x^2-a^2}$ $(a>0)$

$$\int \frac{\mathrm{d}x}{\sqrt{x^2-a^2}} = \ln\left|x+\sqrt{x^2-a^2}\right|$$

$$\int \frac{\mathrm{d}x}{\sqrt{(x^2-a^2)^3}} = -\frac{x}{a^2\sqrt{x^2-a^2}}$$

$$\int \frac{x\,\mathrm{d}x}{\sqrt{x^2-a^2}} = \sqrt{x^2-a^2}$$

$$\int \frac{x\,\mathrm{d}x}{\sqrt{(x^2-a^2)^3}} = -\frac{1}{\sqrt{x^2-a^2}}$$

$$\int \frac{x^2\,\mathrm{d}x}{\sqrt{x^2-a^2}} = \frac{x}{2}\sqrt{x^2-a^2} + \frac{a^2}{2}\ln|x+\sqrt{x^2-a^2}|$$

$$\int \frac{x^2\,\mathrm{d}x}{\sqrt{(x^2-a^2)^3}} = -\frac{x}{\sqrt{x^2-a^2}} + \ln|x+\sqrt{x^2-a^2}|$$

$$\int \frac{\mathrm{d}x}{x\sqrt{x^2-a^2}} = \frac{1}{a}\arccos\frac{a}{|x|}$$

$$\int \frac{\mathrm{d}x}{x^2\sqrt{x^2-a^2}} = \frac{\sqrt{x^2-a^2}}{a^2x}$$

$$\int \sqrt{x^2-a^2}\,\mathrm{d}x = \frac{x}{2}\sqrt{x^2-a^2} - \frac{a^2}{2}\ln|x+\sqrt{x^2-a^2}|$$

$$\int \sqrt{(x^2-a^2)^3}\,\mathrm{d}x = \frac{x}{8}(2x^2-5a^2)\sqrt{x^2-a^2} + \frac{3}{8}a^4\ln|x+\sqrt{x^2-a^2}|$$

$$\int x\sqrt{x^2-a^2}\,\mathrm{d}x = \frac{1}{3}\sqrt{(x^2-a^2)^3}$$

$$\int x^2\sqrt{x^2-a^2}\,\mathrm{d}x = \frac{x}{8}(2x^2-a^2)\sqrt{x^2-a^2} - \frac{a^4}{8}\ln|x+\sqrt{x^2-a^2}|$$

$$\int \frac{\sqrt{x^2-a^2}}{x}\,\mathrm{d}x = \sqrt{x^2-a^2} - a\arccos\frac{a}{|x|}$$

$$\int \frac{\sqrt{x^2-a^2}}{x^2}\,\mathrm{d}x = -\frac{\sqrt{x^2-a^2}}{x} + \ln|x+\sqrt{x^2-a^2}|$$

$\sqrt{a^2-x^2}$ $(a>0)$

$$\int \frac{\mathrm{d}x}{\sqrt{a^2-x^2}} = \arcsin\frac{x}{a}$$

$$\int \frac{\mathrm{d}x}{\sqrt{(a^2-x^2)^3}} = \frac{x}{a^2\sqrt{a^2-x^2}}$$

$$\int \frac{x\,\mathrm{d}x}{\sqrt{a^2-x^2}} = -\sqrt{a^2-x^2}$$

$$\int \frac{x\,\mathrm{d}x}{\sqrt{(a^2-x^2)^3}} = \frac{1}{\sqrt{a^2-x^2}}$$

$$\int \frac{x^2\,\mathrm{d}x}{\sqrt{a^2-x^2}} = -\frac{x}{2}\sqrt{a^2-x^2} + \frac{a^2}{2}\arcsin\frac{x}{a}$$

$$\int \frac{x^2\,\mathrm{d}x}{\sqrt{(a^2-x^2)^3}} = \frac{x}{\sqrt{a^2-x^2}} - \arcsin\frac{x}{a}$$

$$\int \frac{\mathrm{d}x}{x\sqrt{a^2-x^2}} = \frac{1}{a}\ln\frac{a-\sqrt{a^2-x^2}}{|x|}$$

$$\int \frac{\mathrm{d}x}{x^2\sqrt{a^2-x^2}} = -\frac{\sqrt{a^2-x^2}}{a^2x}$$

$$\int \sqrt{a^2-x^2}\,\mathrm{d}x = \frac{x}{2}\sqrt{a^2-x^2} + \frac{a^2}{2}\arcsin\frac{x}{a}$$

$$\int \sqrt{(a^2-x^2)^3}\,\mathrm{d}x = \frac{x}{8}(5a^2-2x^2)\sqrt{a^2-x^2} + \frac{3}{8}a^4\arcsin\frac{x}{a}$$

$$\int x\sqrt{a^2-x^2}\,\mathrm{d}x = -\frac{1}{3}\sqrt{(a^2-x^2)^3}$$

$$\int x^2\sqrt{a^2-x^2}\,\mathrm{d}x = \frac{x}{8}(2x^2-a^2)\sqrt{a^2-x^2} + \frac{a^4}{8}\arcsin\frac{x}{a}$$

$$\int \frac{\sqrt{a^2-x^2}}{x}\,\mathrm{d}x = \sqrt{a^2-x^2} + a\ln\frac{a-\sqrt{a^2-x^2}}{|x|}$$

$$\int \frac{\sqrt{a^2-x^2}}{x^2}\,\mathrm{d}x = -\frac{\sqrt{a^2-x^2}}{x} - \arcsin\frac{x}{a}$$

$\sqrt{\pm ax^2+bx+c}$ $(a>0)$

$$\int \frac{\mathrm{d}x}{\sqrt{ax^2+bx+c}} = \frac{1}{\sqrt{a}}\ln|2ax+b+2\sqrt{a}\sqrt{ax^2+bx+c}|$$

$$\int \sqrt{ax^2+bx+c}\,\mathrm{d}x = \frac{2ax+b}{4a}\sqrt{ax^2+bx+c} + \frac{4ac-b^2}{8\sqrt{a^3}}\ln|2ax+b+2\sqrt{a}\sqrt{ax^2+bx+c}|$$

$$\int \frac{x\,\mathrm{d}x}{\sqrt{ax^2+bx+c}} = \frac{1}{a}\sqrt{ax^2+bx+c} - \frac{b}{2\sqrt{a^3}}\ln|2ax+b+2\sqrt{a}\sqrt{ax^2+bx+c}|$$

$$\int \frac{\mathrm{d}x}{\sqrt{c+bx-ax^2}} = \frac{1}{\sqrt{a}}\arcsin\frac{2ax-b}{\sqrt{b^2+4ac}}$$

$$\int \sqrt{c+bx-ax^2}\,\mathrm{d}x = \frac{2ax-b}{4a}\sqrt{c+bx-ax^2} + \frac{b^2+4ac}{8\sqrt{a^3}}\arcsin\frac{2ax-b}{\sqrt{b^2+4ac}}$$

$$\int \frac{x\,\mathrm{d}x}{\sqrt{c+bx-ax^2}} = -\frac{1}{a}\sqrt{c+bx-ax^2} + \frac{b}{2\sqrt{a^3}}\arcsin\frac{2ax-b}{\sqrt{b^2+4ac}}$$

$\sqrt{\pm\frac{x-a}{x-b}}$, $\sqrt{(x-a)(b-x)}$

$$\int \sqrt{\tfrac{x-a}{x-b}}\,\mathrm{d}x = (x-b)\sqrt{\tfrac{x-a}{x-b}} + (b-a)\ln(\sqrt{|x-a|}+\sqrt{|x-b|})$$

For $a<b$, with $u = \arcsin\sqrt{\frac{x-a}{b-a}}$:

$$\int \sqrt{\tfrac{x-a}{b-x}}\,\mathrm{d}x = (x-b)\sqrt{\tfrac{x-a}{b-x}} + (b-a)u$$

$$\int \frac{\mathrm{d}x}{\sqrt{(x-a)(b-x)}} = 2u$$

$$\int \sqrt{(x-a)(b-x)}\,\mathrm{d}x = \frac{2x-a-b}{4}\sqrt{(x-a)(b-x)} + \frac{(b-a)^2}{4}u$$

Exponentials and logarithms

$$\int a^x \,dx = \frac{a^x}{\ln a}$$
$$\int xa^x \,dx = \frac{xa^x}{\ln a} - \frac{a^x}{(\ln a)^2}$$
$$\int x^n e^{ax} \,dx = \frac{1}{a}x^n e^{ax} - \frac{n}{a}\int x^{n-1}e^{ax} \,dx$$
$$\int x^n a^x \,dx = \frac{x^n a^x}{\ln a} - \frac{n}{\ln a}\int x^{n-1}a^x \,dx$$
$$\int e^{ax}\sin bx \,dx = \frac{e^{ax}(a\sin bx - b\cos bx)}{a^2+b^2}$$
$$\int e^{ax}\cos bx \,dx = \frac{e^{ax}(b\sin bx + a\cos bx)}{a^2+b^2}$$
$$\int e^{ax}\sin^n bx \,dx = \frac{e^{ax}\sin^{n-1} bx\,(a\sin bx - nb\cos bx)}{a^2+b^2n^2} + \frac{n(n-1)b^2}{a^2+b^2n^2}\int e^{ax}\sin^{n-2} bx \,dx$$
$$\int e^{ax}\cos^n bx \,dx = \frac{e^{ax}\cos^{n-1} bx\,(a\cos bx + nb\sin bx)}{a^2+b^2n^2} + \frac{n(n-1)b^2}{a^2+b^2n^2}\int e^{ax}\cos^{n-2} bx \,dx$$
$$\int \ln x \,dx = x\ln x - x$$
$$\int \frac{dx}{x\ln x} = \ln|\ln x|$$
$$\int x^n \ln x \,dx = \frac{x^{n+1}}{n+1}\left(\ln x - \frac{1}{n+1}\right)$$
$$\int (\ln x)^n \,dx = x(\ln x)^n - n\int (\ln x)^{n-1} \,dx$$
$$\int x^m (\ln x)^n \,dx = \frac{x^{m+1}(\ln x)^n}{m+1} - \frac{n}{m+1}\int x^m (\ln x)^{n-1} \,dx$$

Trigonometric functions

$$\int \tan x \,dx = -\ln|\cos x|$$
$$\int \cot x \,dx = \ln|\sin x|$$
$$\int \sec x \,dx = \ln|\sec x + \tan x|$$
$$\int \csc x \,dx = \ln|\csc x - \cot x| = \ln\left|\tan\frac{x}{2}\right|$$
$$\int \sec^2 x \,dx = \tan x$$
$$\int \csc^2 x \,dx = -\cot x$$
$$\int \sec x\tan x \,dx = \sec x$$
$$\int \sin^2 x \,dx = \frac{x}{2} - \frac{\sin 2x}{4}$$
$$\int \cos^2 x \,dx = \frac{x}{2} + \frac{\sin 2x}{4}$$
$$\int \sin^n x \,dx = -\frac{\sin^{n-1} x\cos x}{n} + \frac{n-1}{n}\int \sin^{n-2} x \,dx$$
$$\int \cos^n x \,dx = \frac{\cos^{n-1} x\sin x}{n} + \frac{n-1}{n}\int \cos^{n-2} x \,dx$$
$$\int \frac{dx}{\sin^n x} = -\frac{1}{n-1}\frac{\cos x}{\sin^{n-1} x} + \frac{n-2}{n-1}\int \frac{dx}{\sin^{n-2} x}$$
$$\int \frac{dx}{\cos^n x} = \frac{1}{n-1}\frac{\sin x}{\cos^{n-1} x} + \frac{n-2}{n-1}\int \frac{dx}{\cos^{n-2} x}$$
$$\int \cos^m x\sin^n x \,dx = \frac{\cos^{m-1} x\sin^{n+1} x}{m+n} + \frac{m-1}{m+n}\int \cos^{m-2} x\sin^n x \,dx = -\frac{\cos^{m+1} x\sin^{n-1} x}{m+n} + \frac{n-1}{m+n}\int \cos^m x\sin^{n-2} x \,dx$$
$$\int \sin ax\cos bx \,dx = -\frac{\cos(a+b)x}{2(a+b)} - \frac{\cos(a-b)x}{2(a-b)}$$
$$\int \sin ax\sin bx \,dx = -\frac{\sin(a+b)x}{2(a+b)} + \frac{\sin(a-b)x}{2(a-b)}$$
$$\int \cos ax\cos bx \,dx = \frac{\sin(a+b)x}{2(a+b)} + \frac{\sin(a-b)x}{2(a-b)}$$
$$\int \frac{dx}{a+b\sin x} = \begin{cases} \frac{2}{\sqrt{a^2-b^2}}\arctan\frac{a\tan\frac{x}{2}+b}{\sqrt{a^2-b^2}} & (a^2>b^2) \\ \frac{1}{\sqrt{b^2-a^2}}\ln\left|\frac{a\tan\frac{x}{2}+b-\sqrt{b^2-a^2}}{a\tan\frac{x}{2}+b+\sqrt{b^2-a^2}}\right| & (a^2<b^2) \end{cases}$$
$$\int \frac{dx}{a+b\cos x} = \begin{cases} \frac{2}{\sqrt{a^2-b^2}}\arctan\frac{(a-b)\tan\frac{x}{2}}{\sqrt{a^2-b^2}} & (a^2>b^2) \\ \frac{1}{\sqrt{b^2-a^2}}\ln\left|\frac{(a-b)\tan\frac{x}{2}-\sqrt{b^2-a^2}}{(a-b)\tan\frac{x}{2}+\sqrt{b^2-a^2}}\right| & (a^2<b^2) \end{cases}$$
$$\int \frac{dx}{a^2\cos^2 x + b^2\sin^2 x} = \frac{1}{ab}\arctan\left(\frac{b}{a}\tan x\right)$$
$$\int \frac{dx}{a^2\cos^2 x - b^2\sin^2 x} = \frac{1}{2ab}\ln\left|\frac{b\tan x + a}{b\tan x - a}\right|$$
$$\int x\sin ax \,dx = \frac{\sin ax}{a^2} - \frac{x\cos ax}{a}$$
$$\int x\cos ax \,dx = \frac{\cos ax}{a^2} + \frac{x\sin ax}{a}$$
$$\int x^2\sin ax \,dx = -\frac{x^2\cos ax}{a} + \frac{2x\sin ax}{a^2} + \frac{2\cos ax}{a^3}$$
$$\int x^2\cos ax \,dx = \frac{x^2\sin ax}{a} + \frac{2x\cos ax}{a^2} - \frac{2\sin ax}{a^3}$$

Inverse trigonometric functions ($a > 0$)

$$\int \arcsin\frac{x}{a} \,dx = x\arcsin\frac{x}{a} + \sqrt{a^2-x^2}$$
$$\int x\arcsin\frac{x}{a} \,dx = \left(\frac{x^2}{2} - \frac{a^2}{4}\right)\arcsin\frac{x}{a} + \frac{x}{4}\sqrt{a^2-x^2}$$
$$\int x^2\arcsin\frac{x}{a} \,dx = \frac{x^3}{3}\arcsin\frac{x}{a} + \frac{1}{9}(x^2+2a^2)\sqrt{a^2-x^2}$$
$$\int \arccos\frac{x}{a} \,dx = x\arccos\frac{x}{a} - \sqrt{a^2-x^2}$$
$$\int x\arccos\frac{x}{a} \,dx = \left(\frac{x^2}{2} - \frac{a^2}{4}\right)\arccos\frac{x}{a} - \frac{x}{4}\sqrt{a^2-x^2}$$
$$\int x^2\arccos\frac{x}{a} \,dx = \frac{x^3}{3}\arccos\frac{x}{a} - \frac{1}{9}(x^2+2a^2)\sqrt{a^2-x^2}$$
$$\int \arctan\frac{x}{a} \,dx = x\arctan\frac{x}{a} - \frac{a}{2}\ln(a^2+x^2)$$
$$\int x\arctan\frac{x}{a} \,dx = \frac{1}{2}(a^2+x^2)\arctan\frac{x}{a} - \frac{a}{2}x$$
$$\int x^2\arctan\frac{x}{a} \,dx = \frac{x^3}{3}\arctan\frac{x}{a} - \frac{a}{6}x^2 + \frac{a^3}{6}\ln(a^2+x^2)$$

Data structures (3)

3.1 Range queries

SegmentTree.h

Description: Zero-indexed max-tree. Bounds are inclusive to the left and exclusive to the right. Can be changed by modifying T, f and unit.
Usage: `Tree t(n); t.update(i, x); int m = t.query(l, r);`
Time: $\mathcal{O}(\log N)$

19 lines

```
struct Tree {
  typedef int T;
  static constexpr T unit = INT_MIN;
  T f(T a, T b) { return max(a, b); } // (any associative fn)
  vector<T> s; int n;
  Tree(int n = 0, T def = unit) : s(2*n, def), n(n) {}
  void update(int pos, T val) {
    for (s[pos += n] = val; pos /= 2;)
      s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
  }
  T query(int b, int e) { // query [b, e)
    T ra = unit, rb = unit;
    for (b += n, e += n; b < e; b /= 2, e /= 2) {
      if (b % 2) ra = f(ra, s[b++]);
      if (e % 2) rb = f(s[--e], rb);
    }
    return f(ra, rb);
  }
};
```

LazySegmentTree.h

Description: Segment tree with ability to add or set values of large intervals, and compute max of intervals. Can be changed to other things.
Usage: `Node* tr = new Node(v, 0, (int)v.size()); // or new Node(lo, hi) for a sparse tree over [lo, hi) with all -inf (children made on demand)`
`tr->add(l, r, x); tr->set(l, r, x); tr->query(l, r); // all [l, r)`
Time: $\mathcal{O}(\log N)$.

50 lines

```
const int inf = 1e9;
struct Node {
  Node *l = 0, *r = 0;
  int lo, hi, mset = inf, madd = 0, val = -inf;
  Node(int lo,int hi):lo(lo),hi(hi){} // Large interval of -inf
  Node(vector<int>& v, int lo, int hi) : lo(lo), hi(hi) {
    if (lo + 1 < hi) {
      int mid = lo + (hi - lo)/2;
      l = new Node(v, lo, mid); r = new Node(v, mid, hi);
      val = max(l->val, r->val);
    }
    else val = v[lo];
  }
  int query(int L, int R) {
    if (R <= lo || hi <= L) return -inf;
    if (L <= lo && hi <= R) return val;
    push();
    return max(l->query(L, R), r->query(L, R));
  }
  void set(int L, int R, int x) {
```

```
    if (R <= lo || hi <= L) return;
    if (L <= lo && hi <= R) mset = val = x, madd = 0;
    else {
      push(), l->set(L, R, x), r->set(L, R, x);
      val = max(l->val, r->val);
    }
  }
  void add(int L, int R, int x) {
    if (R <= lo || hi <= L) return;
    if (L <= lo && hi <= R) {
      if (mset != inf) mset += x;
      else madd += x;
      val += x;
    }
    else {
      push(), l->add(L, R, x), r->add(L, R, x);
      val = max(l->val, r->val);
    }
  }
  void push() {
    if (!l) {
      int mid = lo + (hi - lo)/2;
      l = new Node(lo, mid); r = new Node(mid, hi);
    }
    if (mset != inf)
      l->set(lo,hi,mset), r->set(lo,hi,mset), mset = inf;
    else if (madd)
      l->add(lo,hi,madd), r->add(lo,hi,madd), madd = 0;
  }
};
```

PersistentSegmentTree.h

Description: Persistent segment tree on positions $[0, n)$ with point add and range sum. Every add returns a new root; old roots stay valid. Node 0 is the empty tree. Classic use, k-th smallest of $a[l..r)$: insert compressed values in array order, then walk the difference of versions r and l (kth). Same idea: count of values in $[lo, hi)$ inside $a[l..r)$ = query(root[r]) - query(root[l]).
Usage: `PST t(n); vector<int> root = {0}; for (int x : a) root.push_back(t.add(root.back(), x, 1)); int v = t.kth(root[l], root[r], k); // k is 0-based`
Memory: $\mathcal{O}(\log n)$ new nodes per add
Time: $\mathcal{O}(\log n)$ per operation

32 lines

```
struct PST {
  int n; vector<int> L{0}, R{0}; vector<ll> S{0};
  PST(int n) : n(n) {}
  int upd(int v, int p, ll x, int l, int r) {
    int u = S.size();
    L.push_back(L[v]), R.push_back(R[v]);
    S.push_back(S[v] + x);
    if (r - l > 1) {
      int m = (l + r) / 2, c;
      if (p < m) c = upd(L[v], p, x, l, m), L[u] = c;
      else c = upd(R[v], p, x, m, r), R[u] = c;
    }
    return u;
  }
  int add(int rt, int p, ll x) { return upd(rt, p, x, 0, n); }
  ll sum(int v, int a, int b, int l, int r) {
    if (!v || b <= l || r <= a) return 0;
    if (a <= l && r <= b) return S[v];
    int m = (l + r) / 2;
    return sum(L[v], a, b, l, m) + sum(R[v], a, b, m, r);
  }
  ll query(int rt, int a, int b) { return sum(rt,a,b,0,n); }
  int kth(int v, int u, ll k) { // in version u minus version v
    int l = 0, r = n;
    while (r - l > 1) {
      int m = (l + r) / 2; ll c = S[L[u]] - S[L[v]];
      if (k < c) v = L[v], u = L[u], r = m;
      else k -= c, v = R[v], u = R[u], l = m;
    }
    return l;
  }
};
```

FenwickTree.h

Description: Computes partial sums a[0] + a[1] + ... + a[pos - 1], and updates single elements a[i], taking the difference between the old and new value.
Usage: `FT f(n); f.update(i, d); ll s = f.query(r); // [0, r)`
`int p = f.lower_bound(s); // first prefix sum >= s`
Time: Both operations are $\mathcal{O}(\log N)$.

22 lines

```
struct FT {
  vector<ll> s;
  FT(int n) : s(n) {}
  void update(int pos, ll dif) { // a[pos] += dif
    for (; pos < (int)s.size(); pos |= pos + 1) s[pos] += dif;
  }
  ll query(int pos) { // sum of values in [0, pos)
    ll res = 0;
    for (; pos > 0; pos &= pos - 1) res += s[pos-1];
    return res;
  }
  int lower_bound(ll sum) {// min pos st sum of [0, pos] >= sum
    // Returns n if no sum is >= sum, or -1 if empty sum is.
    if (sum <= 0) return -1;
    int pos = 0;
    for (int pw = 1 << 25; pw; pw >>= 1) {
      if (pos + pw <= (int)s.size() && s[pos + pw-1] < sum)
        pos += pw, sum -= s[pos-1];
    }
    return pos;
  }
};
```

FenwickTree2d.h

Description: Computes sums a[i,j] for all i<I, j<J, and increases single elements a[i,j]. Requires that the elements to be updated are known in advance (call fakeUpdate() before init()).
Usage: `FT2 f(X); for each future update: f.fakeUpdate(x, y);`
`f.init(); f.update(x, y, d); f.query(x, y); // i<x, j<y`
Time: $\mathcal{O}\left(\log^2 N\right)$. (Use persistent segment trees for $\mathcal{O}(\log N)$.)

"FenwickTree.h" 25 lines

```
struct FT2 {
  vector<vector<int>> ys; vector<FT> ft;
  FT2(int limx) : ys(limx) {}
  void fakeUpdate(int x, int y) {
    for (; x < (int)ys.size(); x |= x + 1) ys[x].push_back(y);
  }
  void init() {
    for (vector<int>& v : ys)
      sort(v.begin(), v.end()), ft.emplace_back(v.size());
  }
  int ind(int x, int y) {
    return (int)(lower_bound(ys[x].begin(), ys[x].end(), y) -
      ys[x].begin());
  }
  void update(int x, int y, ll dif) {
    for (; x < (int)ys.size(); x |= x + 1)
      ft[x].update(ind(x, y), dif);
  }
  ll query(int x, int y) {
    ll sum = 0;
    for (; x; x &= x - 1)
      sum += ft[x-1].query(ind(x-1, y));
    return sum;
  }
};
```

RMQ.h

Description: Range Minimum Queries on an array. Returns min(V[a], V[a + 1], ... V[b - 1]) in constant time.
Usage: `RMQ rmq(values); // values is a vector<int>`
`rmq.query(inclusive, exclusive);`
Time: $\mathcal{O}(|V| \log |V| + Q)$

15 lines

```
struct RMQ { // for ll / max: change vector<int>, int and min
  vector<vector<int>> jmp;
  RMQ(const vector<int>& V) : jmp(1, V) {
    for (int pw = 1, k = 1; pw * 2 <= V.size(); pw *= 2, ++k) {
      jmp.emplace_back((int)V.size() - pw * 2 + 1);
      for (int j = 0; j < jmp[k].size(); j++)
        jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
    }
  }
  int query(int a, int b) {
    assert(a < b); // or return inf if a == b
    int dep = 31 - __builtin_clz(b - a);
    return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
  }
};
```

SubMatrix.h

Description: Calculate submatrix sums quickly, given upper-left and lower-right corners (half-open).
Usage: `SubMatrix m(matrix); // vector<vector<int>>`
`m.sum(0, 0, 2, 2); // top left 4 elements`
Time: $\mathcal{O}\left(N^2 + Q\right)$

12 lines

```
struct SubMatrix {
  vector<vector<ll>> p;
  SubMatrix(vector<vector<int>>& v) {
    int R = v.size(), C = v[0].size();
    p.assign(R+1, vector<ll>(C+1));
    for (int r = 0; r < R; r++) for (int c = 0; c < C; c++)
      p[r+1][c+1] = v[r][c] + p[r][c+1] + p[r+1][c] - p[r][c];
  }
  ll sum(int u, int l, int d, int r) {
    return p[d][r] - p[d][l] - p[u][r] + p[u][l];
  }
};
```

MoQueries.h

Description: Answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. If values are on tree edges, change step to add/remove the edge (a, c) and remove the initial add call (but keep in).

Usage: `vector<int> ans = mo(Q); // Q[i] = {l, r}, half-open [l, r);`
`write add/del/calc; moTree: Q[i] = {u, v} vertices`

Time: $\mathcal{O}\left(N\sqrt{Q}\right)$

55 lines

```
void add(int ind, int end) { ... } // add a[ind] (end = 0 or 1)
void del(int ind, int end) { ... } // remove a[ind]
int calc() { ... } // compute current answer

vector<int> mo(vector<pair<int, int>> Q) {
  int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
  vector<int> s(Q.size()), res = s;
#define K(x) make_pair(x.first / blk, \
  x.second ^ -(x.first / blk & 1))
  iota(s.begin(), s.end(), 0);
  sort(s.begin(), s.end(),
    [&](int s, int t) { return K(Q[s]) < K(Q[t]); });
  for (int qi : s) {
    pair<int, int> q = Q[qi];
    while (L > q.first) add(--L, 0);
    while (R < q.second) add(R++, 1);
    while (L < q.first) del(L++, 0);
    while (R > q.second) del(--R, 1);
    res[qi] = calc();
  }
  return res;
}

vector<int> moTree(vector<array<int, 2>> Q,
  vector<vector<int>>& ed, int root = 0) {
  int N = ed.size(), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
  vector<int> s(Q.size()), res = s, I(N), L(N), R(N), in(N),
    par(N);
  add(0, 0), in[0] = 1;
  auto dfs = [&](int x, int p, int dep, auto& f) -> void {
    par[x] = p;
    L[x] = N;
    if (dep) I[x] = N++;
    for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
    if (!dep) I[x] = N++;
    R[x] = N;
  };
  dfs(root, -1, 0, dfs);
#define K(x) make_pair(I[x[0]] / blk, \
  I[x[1]] ^ -(I[x[0]] / blk & 1))
  iota(s.begin(), s.end(), 0);
  sort(s.begin(), s.end(),
    [&](int s, int t) { return K(Q[s]) < K(Q[t]); });
  for (int qi : s) for (int end = 0; end < 2; end++) {
    int &a = pos[end], b = Q[qi][end], i = 0;
#define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
                  else { add(c, end); in[c] = 1; } a = c; }
    while (!(L[b] <= L[a] && R[a] <= R[b]))
      I[i++] = b, b = par[b];
    while (a != b) step(par[a]);
    while (i--) step(I[i]);
    if (end) res[qi] = calc();
  }
  return res;
}
```

3.2 Sets, maps, sequences

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. To get a map, change null_type to the value type. For a multiset, store pairs (value, unique id).

Usage: `Tree t; t.insert(x); t.order_of_key(x); // count < x`
`*t.find_by_order(k); // k-th smallest, 0-based`

Time: $\mathcal{O}\left(\log N\right)$

17 lines

```
#include <bits/extc++.h> // above #define int

using namespace __gnu_pbds;

// key type int; for pairs/ll replace both ints
typedef tree<int, null_type, less<int>, rb_tree_tag,
    tree_order_statistics_node_update> Tree;

void example() {
  Tree t, t2; t.insert(8);
  auto it = t.insert(10).first;
  assert(it == t.lower_bound(9));
  assert(t.order_of_key(10) == 1);
  assert(t.order_of_key(11) == 2);
  assert(*t.find_by_order(0) == 8);
  t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}
```

HashMap.h

Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

Usage: `h[key] = 7; if (h.find(key) != h.end()) ...`

8 lines

```
#include <bits/extc++.h> // above #define int

// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
  const uint64_t C = ll(4e18 * acos(0)) | 71;
  ll operator()(ll x) const { return __builtin_bswap64(x*C); }
};
__gnu_pbds::gp_hash_table<ll,int,chash> h({},{},{},{},{1<<16});
```

Treap.h

Description: A short self-balancing tree. It acts as a sequential container with log-time splits/joins, and is easy to augment with additional data.

Usage: `Node* t = 0; t = ins(t, new Node(5), 0); // at index 0`
`auto [l, r] = split(t, k); // first k elements | rest`

Time: $\mathcal{O}\left(\log N\right)$

54 lines

```
struct Node {
  Node *l = 0, *r = 0;
  int val, y, c = 1;
  Node(int val) : val(val), y(rand()) {}
  void recalc();
};

int cnt(Node* n) { return n ? n->c : 0; }
void Node::recalc() { c = cnt(l) + cnt(r) + 1; }

void each(Node* n, vector<int>& out) { // in-order values
  if (n) each(n->l, out), out.push_back(n->val),
    each(n->r, out);
}

pair<Node*, Node*> split(Node* n, int k) {
  if (!n) return {};
  if (cnt(n->l) >= k) { // "n->val >= k" for lower_bound(k)
    auto [L,R] = split(n->l, k);
    n->l = R;
    n->recalc();
    return {L, n};
  } else {
    auto [L,R] = split(n->r,k - cnt(n->l) - 1); // and just "k"
    n->r = L;
    n->recalc();
    return {n, R};
  }
}

Node* merge(Node* l, Node* r) {
  if (!l) return r;
  if (!r) return l;
  if (l->y > r->y) {
    l->r = merge(l->r, r);
    return l->recalc(), l;
  } else {
    r->l = merge(l, r->l);
    return r->recalc(), r;
  }
}

Node* ins(Node* t, Node* n, int pos) {
  auto [l,r] = split(t, pos);
  return merge(merge(l, n), r);
}

// Example application: move the range [l, r) to index k
void move(Node*& t, int l, int r, int k) {
  Node *a, *b, *c;
  tie(a,b) = split(t, l); tie(b,c) = split(b, r - l);
  if (k <= l) t = merge(ins(a, b, k), c);
  else t = merge(a, ins(c, b, k - r));
}
```

3.3 Union-find

UnionFind.h

Description: Disjoint-set data structure.

Usage: `UF uf(n); uf.join(a, b); uf.sameSet(a, b); uf.size(a);`

Time: $\mathcal{O}\left(\alpha(N)\right)$

14 lines

```
struct UF {
  vector<int> e;
  UF(int n) : e(n, -1) {}
  bool sameSet(int a, int b) { return find(a) == find(b); }
  int size(int x) { return -e[find(x)]; }
```

```
  int find(int x) { return e[x] < 0 ? x : e[x] = find(e[x]); }
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    e[a] += e[b]; e[b] = a;
    return true;
  }
};
```

UnionFindRollback.h

Description: Disjoint-set data structure with undo. If undo is not needed, skip st, time() and rollback().
Usage: `int t = uf.time(); ...; uf.rollback(t);`
Time: $\mathcal{O}(\log(N))$

21 lines

```
struct RollbackUF {
  vector<int> e; vector<pair<int, int>> st;
  RollbackUF(int n) : e(n, -1) {}
  int size(int x) { return -e[find(x)]; }
  int find(int x) { return e[x] < 0 ? x : find(e[x]); }
  int time() { return (int)st.size(); }
  void rollback(int t) {
    for (int i = time(); i --> t;)
      e[st[i].first] = st[i].second;
    st.resize(t);
  }
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    st.push_back({a, e[a]});
    st.push_back({b, e[b]});
    e[a] += e[b]; e[b] = a;
    return true;
  }
};
```

3.4 Lines (convex hull trick)

LineContainer.h

Description: Container where you can add lines of the form kx+m, and query maximum values at points x. Useful for dynamic programming ("convex hull trick").
Usage: `LineContainer lc; lc.add(k, m); ll y = lc.query(x);`
`// max of k*x + m; for min negate k, m and y`
Time: $\mathcal{O}(\log N)$

30 lines

```
struct Line {
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  static const ll inf = LLONG_MAX;
  ll div(ll a, ll b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b); }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};
```

LiChao.h

Description: Li Chao tree: add lines $y = kx + m$ and query the minimum at integer $x \in [lo, hi)$. addSeg adds a line that only exists for $x \in [a, b)$. For maximum, negate k, m and the answer. Simpler than LineContainer when queries are on a known integer range and needed for segments. Watch out for overflow in $kx + m$.
Usage: `LiChao t(-1e9, 1e9+1); t.add({k, m}); ll y = t.query(x);`
Time: $\mathcal{O}(\log C)$ per add/query, $\mathcal{O}\left(\log^2 C\right)$ per addSeg

47 lines

```
struct Line {
  ll k, m;
  ll operator()(ll x) const { return k * x + m; }
};
struct LiChao {
  struct N { Line f; int c[2]{}; };
  vector<N> t; ll lo, hi;
  LiChao(ll lo, ll hi) : t(2, {{0, LLONG_MAX}}), lo(lo),
    hi(hi) {}
  void ins(Line g, int v, ll l, ll r) {
    for (;;) {
      ll m = l + (r - l) / 2;
      bool L = g(l) < t[v].f(l), M = g(m) < t[v].f(m);
      if (M) swap(t[v].f, g);
      if (r - l == 1) return;
      int d = L == M; // 0: left half, 1: right half
      (d ? l : r) = m;
      if (!t[v].c[d]) {
        t[v].c[d] = t.size(); t.push_back({g}); return;
      }
      v = t[v].c[d];
    }
  }
  void add(Line g) { ins(g, 1, lo, hi); }
  void seg(Line g, ll a, ll b, int v, ll l, ll r) {
    if (b <= l || r <= a) return;
    if (a <= l && r <= b) return ins(g, v, l, r);
    ll m = l + (r - l) / 2;
    for (int d = 0; d < 2; d++) {
      if (!t[v].c[d])
        t[v].c[d] = t.size(), t.push_back({{0, LLONG_MAX}});
      seg(g, a, b, t[v].c[d], d ? m : l, d ? r : m);
    }
  }
  void addSeg(Line g, ll a, ll b) { seg(g, a, b, 1, lo, hi); }
  ll query(ll x) {
    ll res = LLONG_MAX, l = lo, h = hi;
    for (int v = 1; v;) {
      res = min(res, t[v].f(x));
      ll m = l + (h - l) / 2;
      int d = x >= m;
      (d ? l : h) = m;
      v = t[v].c[d];
    }
    return res;
  }
};
```

3.5 Bits and matrices

XorBasis.h

Description: Linear basis of non-negative 63-bit integers under XOR (vector space over $\mathbb{F}_2$). rank = basis size; the set of all subset xors has 2^{rank} distinct values. kth(k) gives the k-th smallest of them (0-based, $k < 2^{rank}$, 0 included).
Usage: `XorBasis b; for (ll x : a) b.insert(x); ll best = b.maxXor();`
Time: $\mathcal{O}(\log C)$ per insert/query, $\mathcal{O}\left(\log^2 C\right)$ for kth

30 lines

```
struct XorBasis {
  ll b[63] = {}; int rank = 0;
  bool insert(ll x) { // false if x was already representable
    for (int i = 62; i >= 0; i--) if (x >> i & 1) {
      if (!b[i]) return b[i] = x, ++rank;
      x ^= b[i];
    }
    return false;
  }
  bool contains(ll x) {
    for (int i = 62; i >= 0; i--) if (x >> i & 1) x ^= b[i];
    return !x;
  }
  ll maxXor(ll x = 0) { // max of x ^ (some subset xor)
    for (int i = 62; i >= 0; i--) x = max(x, x ^ b[i]);
    return x;
  }
  ll kth(ll k) {
    vector<ll> v;
    for (int i = 0; i < 63; i++) if (b[i]) {
      ll x = b[i];
      for (int j = i; j--;) if (x >> j & 1) x ^= b[j];
      v.push_back(x);
    }
    ll r = 0;
    for (int i = 0; i < v.size(); i++)
      if (k >> i & 1) r ^= v[i];
    return r;
  }
};
```

PersistentXorTrie.h

Description: Persistent binary trie over B-bit numbers. Version i holds $a[0..i)$. maxXor(l, r, x) $= \max_{l \le j < r} x \oplus a[j]$ (requires $l < r$). Without persistence, a plain trie (one version) answers "max xor with any inserted value".
Usage: `XorTrie t; for (int v : a) t.add(v); int m = t.maxXor(l, r, x);`
Memory: $\mathcal{O}(NB)$
Time: $\mathcal{O}(B)$ per operation

27 lines

```
struct XorTrie {
```

```
  static const int B = 30;
  vector<array<int, 2>> nx{{0, 0}};
  vector<int> cnt{0}, root{0};
  int clone(int v) {
    nx.push_back(nx[v]); cnt.push_back(cnt[v] + 1);
    return (int)cnt.size() - 1;
  }
  void add(int x) {
    int v = root.back(), u = clone(v);
    root.push_back(u);
    for (int b = B; b--;) {
      int c = x >> b & 1, w = clone(nx[v][c]);
      nx[u][c] = w; u = w; v = nx[v][c];
    }
  }
  int maxXor(int l, int r, int x) {
    int u = root[r], v = root[l], res = 0;
    for (int b = B; b--;) {
      int c = !(x >> b & 1);
      if (cnt[nx[u][c]] > cnt[nx[v][c]]) res |= 1 << b;
      else c ^= 1;
      u = nx[u][c], v = nx[v][c];
    }
    return res;
  }
};
```

Matrix.h

Description: Matrix product and power modulo mod, e.g. for linear recurrences: if $v_{i+1} = Av_i$ then $v_n = A^n v_0$ (a vector is an $n \times 1$ matrix). For doubles change ll to double and drop % mod.
Usage: `Mat A = {{1,1},{1,0}}; Mat B = pw(A, n); // B[0][1] = Fib(n)`
Time: $\mathcal{O}\left(N^3 \log p\right)$

19 lines

```
const ll mod = 1e9 + 7;
typedef vector<vector<ll>> Mat;
Mat operator*(const Mat& a, const Mat& b) {
  int n = a.size(), k = b.size(), m = b[0].size();
  Mat c(n, vector<ll>(m));
  for (int i = 0; i < n; i++)
    for (int j = 0; j < k; j++)
      if (a[i][j])
        for (int l = 0; l < m; l++)
          c[i][l] = (c[i][l] + a[i][j] * b[j][l]) % mod;
  return c;
}
Mat pw(Mat a, ll p) {
  Mat r((int)a.size(), vector<ll>(a.size()));
  for (int i = 0; i < a.size(); i++) r[i][i] = 1;
  for (; p; p /= 2, a = a * a)
    if (p & 1) r = r * a;
  return r;
}
```

Numerical (4)

4.1 Polynomials and recurrences

Polynomial.h

Usage: `Poly p{{c0, c1, c2}}; double y = p(x); // c0 + c1 x + ...`
`p.diff(); // derivative; p.divroot(r) divides by (x - r)`

18 lines

```
struct Poly {
  vector<double> a;
  double operator()(double x) const {
    double val = 0;
    for (int i = a.size(); i--;) (val *= x) += a[i];
    return val;
  }
  void diff() {
    for (int i = 1; i < a.size(); i++) a[i-1] = i*a[i];
    a.pop_back();
  }
  void divroot(double x0) {
    double b = a.back(), c; a.back() = 0;
    for (int i = (int)a.size() - 1; i--;)
      c = a[i], a[i] = a[i + 1] * x0 + b, b = c;
    a.pop_back();
  }
};
```

PolyRoots.h

Description: Finds the real roots to a polynomial.
Usage: `polyRoots({{2,-3,1}},-1e9,1e9) // solve x^2-3x+2 = 0`
Time: $\mathcal{O}\left(n^2 \log(1/\epsilon)\right)$

"Polynomial.h" 23 lines

```
vector<double> polyRoots(Poly p, double xmin, double xmax) {
  if ((int)p.a.size() == 2) { return {-p.a[0]/p.a[1]}; }
  vector<double> ret;
  Poly der = p;
  der.diff();
  auto dr = polyRoots(der, xmin, xmax);
  dr.push_back(xmin-1);
  dr.push_back(xmax+1);
  sort(dr.begin(), dr.end());
  for (int i = 0; i < (int)dr.size()-1; i++) {
    double l = dr[i], h = dr[i+1];
    bool sign = p(l) > 0;
    if (sign ^ (p(h) > 0)) {
      for (int it = 0; it < 60; it++) { // while (h - l > 1e-8)
        double m = (l + h) / 2, f = p(m);
        if ((f <= 0) ^ sign) l = m;
        else h = m;
      }
      ret.push_back((l + h) / 2);
    }
  }
  return ret;
}
```

PolyInterpolate.h

Description: Given n points (x[i], y[i]), computes an n-1-degree polynomial p that passes through them: $p(x) = a[0]*x^0 + ... + a[n-1]*x^{n-1}$. For numerical precision, pick $x[k] = c*\cos(k/(n-1)*\pi), k = 0 \dots n-1$.
Usage: `vd a = interpolate(x, y, n); // p(x) = sum a[i] x^i`
Time: $\mathcal{O}\left(n^2\right)$

13 lines

```
typedef vector<double> vd;
vd interpolate(vd x, vd y, int n) {
  vd res(n), temp(n);
  for (int k = 0; k < n-1; k++) for (int i = k+1; i < n; i++)
    y[i] = (y[i] - y[k]) / (x[i] - x[k]);
  double last = 0; temp[0] = 1;
  for (int k = 0; k < n; k++) for (int i = 0; i < n; i++) {
    res[i] += y[k] * temp[i];
    swap(last, temp[i]);
    temp[i] -= last * x[k];
  }
  return res;
}
```

BerlekampMassey.h

Description: Recovers any n-order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$.
Usage: `berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}`
Time: $\mathcal{O}\left(N^2\right)$

"../number-theory/ModPow.h" 22 lines

```
vector<ll> berlekampMassey(vector<ll> s) {
  int n = s.size(), L = 0, m = 0;
  vector<ll> C(n), B(n), T;
  C[0] = B[0] = 1;

  ll b = 1;
  for (int i = 0; i < n; i++) { ++m;
    ll d = s[i] % mod;
    for (int j = 1; j < L + 1; j++)
      d = (d + C[j] * s[i - j]) % mod;
    if (!d) continue;
    T = C; ll coef = d * modpow(b, mod-2) % mod;
    for (int j = m; j < n; j++)
      C[j] = (C[j] - coef * B[j - m]) % mod;
    if (2 * L > i) continue;
    L = i + 1 - L; B = T; b = d; m = 0;
  }

  C.resize(L + 1); C.erase(C.begin());
  for (ll& x : C) x = (mod - x) % mod;
  return C;
}
```

LinearRecurrence.h

Description: Generates the k'th term of an n-order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \dots \geq n-1]$ and $tr[0 \dots n-1]$. Faster than matrix multiplication. Useful together with Berlekamp–Massey.
Usage: `linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci number`
`// needs a global const ll mod`
Time: $\mathcal{O}\left(n^2 \log k\right)$

27 lines

```
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
  int n = tr.size();

  auto combine = [&](Poly a, Poly b) {
    Poly res(n * 2 + 1);
    for (int i = 0; i < n+1; i++) for (int j = 0; j < n+1; j++)
      res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
    for (int i = 2 * n; i > n; --i) for (int j = 0; j < n; j++)
```

```
      res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
    res.resize(n + 1);
    return res;
  };

  Poly pol(n + 1), e(pol);
  pol[0] = e[1] = 1;

  for (++k; k; k /= 2) {
    if (k % 2) pol = combine(pol, e);
    e = combine(e, e);
  }

  ll res = 0;
  for (int i = 0; i < n; i++)
    res = (res + pol[i + 1] * S[i]) % mod;
  return res;
}
```

4.2 Optimization

GoldenSectionSearch.h

Description: Finds the argument minimizing the function f in the interval $[a,b]$ assuming f is unimodal on the interval, i.e. has only one local minimum and no local maximum. The maximum error in the result is *eps*. Works equally well for maximization with a small change in the code. See TernarySearch.h in the Various chapter for a discrete version.

Usage: `double xmin = gss(-1000,1000,[&](double x){ return 4+x+.3*x*x; });`

Time: $\mathcal{O}\left(\log((b-a)/\epsilon)\right)$

14 lines

```
double gss(double a, double b, function<double(double)> f) {
  double r = (sqrt(5)-1)/2, eps = 1e-7;
  double x1 = b - r*(b-a), x2 = a + r*(b-a);
  double f1 = f(x1), f2 = f(x2);
  while (b-a > eps)
    if (f1 < f2) { //change to > to find maximum
      b = x2; x2 = x1; f2 = f1;
      x1 = b - r*(b-a); f1 = f(x1);
    } else {
      a = x1; x1 = x2; f1 = f2;
      x2 = a + r*(b-a); f2 = f(x2);
    }
  return a;
}
```

Integrate.h

Description: Simple integration of a function over an interval using Simpson's rule. The error should be proportional to h^4, although in practice you will want to verify that the result is stable to desired precision when epsilon changes.

Usage: `double I = quad(0, M_PI, [](double x) { return sin(x); }); // 2`

7 lines

```
double quad(double a, double b, function<double(double)> f,
    const int n = 1000) {
  double h = (b - a) / 2 / n, v = f(a) + f(b);
  for (int i = 1; i < n*2; i++)
    v += f(a + i*h) * (i&1 ? 4 : 2);
  return v * h / 3;
}
```

IntegrateAdaptive.h

Description: Fast integration using an adaptive Simpson's rule.

Usage: `double sphereVolume = quad(-1, 1, [](double x) {
return quad(-1, 1, [&](double y) {
return quad(-1, 1, [&](double z) {
return x*x + y*y + z*z < 1; });});});`

14 lines

```
typedef double d;
typedef function<d(d)> F;
#define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6

d rec(F& f, d a, d b, d eps, d S) {
  d c = (a + b) / 2;
  d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
  if (abs(T - S) <= 15 * eps || b - a < 1e-10)
    return T + (T - S) / 15;
  return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2, S2);
}
d quad(d a, d b, F f, d eps = 1e-8) {
  return rec(f, a, b, eps, S(a, b));
}
```

Simplex.h

Description: Solves a general linear maximization problem: maximize $c^T x$ subject to $Ax \leq b$, $x \geq 0$. Returns -inf if there is no solution, inf if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. The input vector is set to an optimal x (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable.

Usage: `vvd A = {{1,-1}, {-1,1}, {-1,-2}};
vd b = {1,1,-4}, c = {-1,-1}, x;
T val = LPSolver(A, b, c).solve(x);`

Time: $\mathcal{O}\left(NM * \#pivots\right)$, where a pivot may be e.g. an edge relaxation. $\mathcal{O}\left(2^n\right)$ in the general case.

80 lines

```
typedef double T; // long double, Rational, double + mod<P>...
typedef vector<T> vd;
typedef vector<vd> vvd;

const T eps = 1e-8, inf = 1/.0;
#define MP make_pair
#define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s])) s=j

struct LPSolver {
  int m, n;
  vector<int> N, B;
  vvd D;

  LPSolver(const vvd& A, const vd& b, const vd& c) :
    m(b.size()), n(c.size()), N(n+1), B(m), D(m+2, vd(n+2)) {
    for (int i = 0; i < m; i++)
      for (int j = 0; j < n; j++) D[i][j] = A[i][j];
    for (int i = 0; i < m; i++) {
      B[i] = n + i;
      D[i][n] = -1;
      D[i][n + 1] = b[i];
    }
    for (int j = 0; j < n; j++) {
      N[j] = j;
      D[m][j] = -c[j];
    }
    N[n] = -1; D[m+1][n] = 1;
  }

  void pivot(int r, int s) {
    T *a = D[r].data(), inv = 1 / a[s];
    for (int i = 0; i < m + 2; i++)
      if (i != r && abs(D[i][s]) > eps) {
        T *b = D[i].data(), inv2 = b[s] * inv;
        for (int j = 0; j < n + 2; j++) b[j] -= a[j] * inv2;
        b[s] = a[s] * inv2;
      }
    for (int j = 0; j < n+2; j++) if (j != s) D[r][j] *= inv;
    for (int i = 0; i < m+2; i++) if (i != r) D[i][s] *= -inv;
    D[r][s] = inv;
    swap(B[r], N[s]);
  }

  bool simplex(int phase) {
    int x = m + phase - 1;
    for (;;) {
      int s = -1;
      for (int j = 0; j < n + 1; j++)
        if (N[j] != -phase) ltj(D[x]);
      if (D[x][s] >= -eps) return true;
      int r = -1;
      for (int i = 0; i < m; i++) {
        if (D[i][s] <= eps) continue;
        if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
                     < MP(D[r][n+1] / D[r][s], B[r])) r = i;
      }
      if (r == -1) return false;
      pivot(r, s);
    }
  }

  T solve(vd &x) {
    int r = 0;
    for (int i = 1; i < m; i++)
      if (D[i][n + 1] < D[r][n + 1]) r = i;
    if (D[r][n+1] < -eps) {
      pivot(r, n);
      if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
      for (int i = 0; i < m; i++) if (B[i] == -1) {
        int s = 0;
        for (int j = 1; j < n+1; j++) ltj(D[i]);
        pivot(i, s);
      }
    }
    bool ok = simplex(1); x = vd(n);
    for (int i = 0; i < m; i++)
      if (B[i] < n) x[B[i]] = D[i][n + 1];
    return ok ? D[m][n+1] : inf;
  }
};
```

4.3 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.

Usage: `double d = det(a); // destroys a`

Time: $\mathcal{O}\left(N^3\right)$

17 lines

```cpp
double det(vector<vector<double>>& a) {
  int n = a.size(); double res = 1;
  for (int i = 0; i < n; i++) {
    int b = i;
    for (int j = i + 1; j < n; j++)
      if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
    if (i != b) swap(a[i], a[b]), res *= -1;
    res *= a[i][i];
    if (res == 0) return 0;
    for (int j = i+1; j < n; j++) {
      double v = a[j][i] / a[i][i];
      if (v != 0)
        for (int k = i + 1; k < n; k++) a[j][k] -= v * a[i][k];
    }
  }
  return res;
}
```

IntDeterminant.h

Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version.
Usage: `ll d = det(a); // modulo the const mod; destroys a`
Time: $\mathcal{O}\left(N^3\right)$

18 lines

```cpp
const ll mod = 12345;
ll det(vector<vector<ll>>& a) {
  int n = a.size(); ll ans = 1;
  for (int i = 0; i < n; i++) {
    for (int j = i+1; j < n; j++) {
      while (a[j][i] != 0) { // gcd step
        ll t = a[i][i] / a[j][i];
        if (t) for (int k = i; k < n; k++)
          a[i][k] = (a[i][k] - a[j][k] * t) % mod;
        swap(a[i], a[j]);
        ans *= -1;
      }
    }
    ans = ans * a[i][i] % mod;
    if (!ans) return 0;
  }
  return (ans + mod) % mod;
}
```

SolveLinear.h

Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost.
Usage: `vd x(m); int r = solveLinear(A, b, x); // -1: none`
Time: $\mathcal{O}\left(n^2m\right)$

39 lines

```cpp
typedef vector<double> vd;
const double eps = 1e-12;

int solveLinear(vector<vd>& A, vd& b, vd& x) {
  int n = A.size(), m = x.size(), rank = 0, br, bc;
  if (n) assert((int)A[0].size() == m);
  vector<int> col(m); iota(col.begin(), col.end(), 0);

  for (int i = 0; i < n; i++) {
    double v, bv = 0;
    for (int r = i; r < n; r++) for (int c = i; c < m; c++)
      if ((v = fabs(A[r][c])) > bv)
        br = r, bc = c, bv = v;
    if (bv <= eps) {
      for (int j = i; j < n; j++)
        if (fabs(b[j]) > eps) return -1;
      break;
    }
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    for (int j = 0; j < n; j++) swap(A[j][i], A[j][bc]);
    bv = 1/A[i][i];
    for (int j = i+1; j < n; j++) {
      double fac = A[j][i] * bv;
      b[j] -= fac * b[i];
      for (int k = i+1; k < m; k++) A[j][k] -= fac*A[i][k];
    }
    rank++;
  }

  x.assign(m, 0);
  for (int i = rank; i--;) {
    b[i] /= A[i][i];
    x[col[i]] = b[i];
    for (int j = 0; j < i; j++) b[j] -= A[j][i] * b[i];
  }
  return rank; // (multiple solutions if rank < m)
}
```

SolveLinear2.h

Description: To get all uniquely determined values of x back from SolveLinear, make the following changes:

"SolveLinear.h" 9 lines

```cpp
for (int j = 0; j < n; j++)
  if (j != i) // instead of for (int j = i+1; j < n; j++)
    // ... then at the end:
    x.assign(m, undefined);
for (int i = 0; i < rank; i++) {
  for (int j = rank; j < m; j++)
    if (fabs(A[i][j]) > eps) goto fail;
  x[col[i]] = b[i] / A[i][i];
fail:; }
```

SolveLinearBinary.h

Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b.
Usage: `bs x; int r = solveLinear(A, b, x, m); // A: bitset rows`
Time: $\mathcal{O}\left(n^2m\right)$

34 lines

```cpp
typedef bitset<1000> bs;

int solveLinear(vector<bs>& A, vector<int>& b, bs& x, int m) {
  int n = A.size(), rank = 0, br;
  assert(m <= x.size());
  vector<int> col(m); iota(col.begin(), col.end(), 0);
  for (int i = 0; i < n; i++) {
    for (br=i; br<n; ++br) if (A[br].any()) break;
    if (br == n) {
      for (int j = i; j < n; j++) if(b[j]) return -1;
      break;
    }
    int bc = (int)A[br]._Find_next(i-1);
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    for (int j = 0; j < n; j++) if (A[j][i] != A[j][bc]) {
      A[j].flip(i); A[j].flip(bc);
    }
    for (int j = i+1; j < n; j++) if (A[j][i]) {
      b[j] ^= b[i];
      A[j] ^= A[i];
    }
    rank++;
  }

  x = bs();
  for (int i = rank; i--;) {
    if (!b[i]) continue;
    x[col[i]] = 1;
    for (int j = 0; j < i; j++) b[j] ^= A[j][i];
  }
  return rank; // (multiple solutions if rank < m)
}
```

SolveLinearMod.h

Description: Solves $Ax = b$ modulo a prime (entries in $[0, \mathrm{mod})$). Returns the rank, or -1 if there is no solution; x gets one solution with free variables 0. The number of solutions is $\mathrm{mod}^{m-\mathrm{rank}}$. Destroys A and b. For mod 2 use SolveLinearBinary.
Usage: `vector<vector<ll>> A(n, vector<ll>(m)); vector<ll> b(n), x(m);`
`int r = solveLinearMod(A, b, x);`
Time: $\mathcal{O}\left(n^2m\right)$

"../number-theory/ModPow.h" 27 lines

```cpp
int solveLinearMod(vector<vector<ll>>& A, vector<ll>& b,
    vector<ll>& x) {
  int n = A.size(), m = x.size(), r = 0;
  vector<int> where(m, -1);
  for (int c = 0; c < m; c++) {
    if (r == n) break;
    int p = r;
    while (p < n && !A[p][c]) p++;
    if (p == n) continue;
    swap(A[p], A[r]), swap(b[p], b[r]), where[c] = r;
    ll iv = modpow(A[r][c], mod - 2);
    for (int j = c; j < m; j++) A[r][j] = A[r][j] * iv % mod;
    b[r] = b[r] * iv % mod;
    for (int i = 0; i < n; i++) if (i != r && A[i][c]) {
      ll f = A[i][c];
      for (int j = c; j < m; j++)
        A[i][j] = (A[i][j] - f * A[r][j] % mod + mod) % mod;
      b[i] = (b[i] - f * b[r] % mod + mod) % mod;
    }
    r++;
  }
  for (int i = r; i < n; i++) if (b[i]) return -1;
  x.assign(m, 0);
  for (int c = 0; c < m; c++)
    if (where[c] >= 0) x[c] = b[where[c]];
  return r;
}
```

MatrixInverse.h

Description: Invert matrix A. Returns rank; result is stored in A unless singular (rank < n). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1})$ (mod p^k) where A^{-1} starts as the inverse of A mod p, and k is doubled in each step.
Usage: `if (matInv(A) == n) ... // A now holds the inverse`
Time: $\mathcal{O}(n^3)$

36 lines

```
int matInv(vector<vector<double>>& A) {
  int n = A.size(); vector<int> col(n);
  vector<vector<double>> tmp(n, vector<double>(n));
  for (int i = 0; i < n; i++) tmp[i][i] = 1, col[i] = i;

  for (int i = 0; i < n; i++) {
    int r = i, c = i;
    for (int j = i; j < n; j++) for (int k = i; k < n; k++)
      if (fabs(A[j][k]) > fabs(A[r][c]))
        r = j, c = k;
    if (fabs(A[r][c]) < 1e-12) return i;
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    for (int j = 0; j < n; j++)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
    swap(col[i], col[c]);
    double v = A[i][i];
    for (int j = i+1; j < n; j++) {
      double f = A[j][i] / v;
      A[j][i] = 0;
      for (int k = i+1; k < n; k++) A[j][k] -= f*A[i][k];
      for (int k = 0; k < n; k++) tmp[j][k] -= f*tmp[i][k];
    }
    for (int j = i+1; j < n; j++) A[i][j] /= v;
    for (int j = 0; j < n; j++) tmp[i][j] /= v;
    A[i][i] = 1;
  }

  for (int i = n-1; i > 0; --i) for (int j = 0; j < i; j++) {
    double v = A[j][i];
    for (int k = 0; k < n; k++) tmp[j][k] -= v*tmp[i][k];
  }

  for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++) A[col[i]][col[j]] = tmp[i][j];
  return n;
}
```

Tridiagonal.h

Description: $x = \text{tridiagonal}(d, p, q, b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i,\ 1 \le i \le n,$$

where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, ..., -1, 1\}, \{0, c_1, c_2, \ldots, c_n\}, \{b_1, b_2, \ldots, b_n, 0\}, \{a_0, d_1, d_2, \ldots, d_n, a_{n+1}\}).$$

Fails if the solution is not unique.
If $|d_i| > |p_i| + |q_{i-1}|$ for all i, or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither `tr` nor the check for `diag[i] == 0` is needed.
Usage: `vector<T> x = tridiagonal(diag, super, sub, b);`
Time: $\mathcal{O}(N)$

26 lines

```
typedef double T;
vector<T> tridiagonal(vector<T> diag, const vector<T>& super,
    const vector<T>& sub, vector<T> b) {
  int n = b.size(); vector<int> tr(n);
  for (int i = 0; i < n-1; i++) {
    if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] == 0
      b[i+1] -= b[i] * diag[i+1] / super[i];
      if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
      diag[i+1] = sub[i]; tr[++i] = 1;
    } else {
      diag[i+1] -= super[i]*sub[i]/diag[i];
      b[i+1] -= b[i]*sub[i]/diag[i];
    }
  }
  for (int i = n; i--;) {
    if (tr[i]) {
      swap(b[i], b[i-1]);
      diag[i-1] = diag[i];
      b[i] /= super[i-1];
    } else {
      b[i] /= diag[i];
      if (i) b[i-1] -= b[i]*super[i-1];
    }
  }
  return b;
}
```

4.4 Fourier transforms

FastFourierTransform.h

Description: fft(a) computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k. N must be a power of 2. Useful for convolution: `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n, reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16}; higher for random inputs). Otherwise, use NTT/FFTMod.
Usage: `vd c = conv(a, b); // round(c[i]) for integers`
Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)

41 lines

```
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
  int n = a.size(), L = 31 - __builtin_clz(n);
  static vector<complex<long double>> R(2, 1);
  static vector<C> rt(2, 1);  // (^ 10% faster if double)
  for (static int k = 2; k < n; k *= 2) {
    R.resize(n); rt.resize(n);
    auto x = polar(1.0L, acos(-1.0L) / k);
    for (int i = k; i < 2 * k; i++)
      rt[i] = R[i] = i & 1 ? R[i / 2] * x : R[i / 2];
  }
  vector<int> rev(n);
  for (int i = 0; i < n; i++)
    rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  for (int i = 0; i < n; i++)
    if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k)
      for (int j = 0; j < k; j++) {
        C z = rt[j+k] * a[i+j+k]; // 25% faster hand-rolled
        a[i + j + k] = a[i + j] - z;
        a[i + j] += z;
      }
}
vd conv(const vd& a, const vd& b) {
  if (a.empty() || b.empty()) return {};
  vd res((int)a.size() + (int)b.size() - 1);
  int L = 32 - __builtin_clz(res.size()), n = 1 << L;
  vector<C> in(n), out(n);
  copy(a.begin(), a.end(), begin(in));
  for (int i = 0; i < b.size(); i++) in[i].imag(b[i]);
  fft(in);
  for (C& x : in) x *= x;
  for (int i = 0; i < n; i++)
    out[i] = in[-i & (n - 1)] - conj(in[i]);
  fft(out);
  for (int i = 0; i < res.size(); i++)
    res[i] = imag(out[i]) / (4 * n);
  return res;
}
```

FastFourierTransformMod.h

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$.
Usage: `vl c = convMod(a, b, 1000000007);`
Time: $\mathcal{O}(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)

"FastFourierTransform.h" 24 lines

```
typedef vector<ll> vl;
vl convMod(const vl &a, const vl &b, ll M) {
  if (a.empty() || b.empty()) return {};
  vl res((int)a.size() + (int)b.size() - 1);
  int B=32-__builtin_clz(res.size()), n=1<<B, cut=(int)sqrt(M);
  vector<C> L(n), R(n), outs(n), outl(n);
  for (int i = 0; i < a.size(); i++)
    L[i] = C((int)a[i] / cut, (int)a[i] % cut);
  for (int i = 0; i < b.size(); i++)
    R[i] = C((int)b[i] / cut, (int)b[i] % cut);
  fft(L), fft(R);
  for (int i = 0; i < n; i++) {
    int j = -i & (n - 1);
    outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
    outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
  }
  fft(outl), fft(outs);
  for (int i = 0; i < res.size(); i++) {
    ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
    ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
    res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
  }
  return res;
}
```

NumberTheoreticTransform.h

Description: ntt(a) computes $\hat{f}(k) = \sum_x a[x]g^{xk}$ for all k, where $g = \text{root}^{(mod-1)/N}$. N must be a power of 2. Useful for convolution modulo specific nice primes of the form $2^a b + 1$, where the convolution result has size at most 2^a. For arbitrary modulo, see FFTMod. conv(a, b) = c, where $c[x] = \sum a[i]b[x-i]$. For manual convolution: NTT the inputs, multiply pointwise, divide by n, reverse(start+1, end), NTT back. Inputs must be in [0, mod).
Usage: vl c = conv(a, b); // mod 998244353
Time: $\mathcal{O}(N \log N)$

45 lines

```
const ll mod = (119 << 23) + 1, root = 62; // = 998244353
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
// and 483 << 21 (same root). The last two are > 10^9.
typedef vector<ll> vl;
ll modpow(ll b, ll e) {
  ll ans = 1;
  for (; e; b = b * b % mod, e /= 2)
    if (e & 1) ans = ans * b % mod;
  return ans;
}
void ntt(vl &a) {
  int n = a.size(), L = 31 - __builtin_clz(n);
  static vl rt(2, 1);
  for (static int k = 2, s = 2; k < n; k *= 2, s++) {
    rt.resize(n);
    ll z[] = {1, modpow(root, mod >> s)};
    for (int i = k; i < 2 * k; i++)
      rt[i] = rt[i / 2] * z[i & 1] % mod;
  }
  vector<int> rev(n);
  for (int i = 0; i < n; i++)
    rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  for (int i = 0; i < n; i++)
    if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k)
      for (int j = 0; j < k; j++) {
        ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
        a[i + j + k] = ai - z + (z > ai ? mod : 0);
        ai += (ai + z >= mod ? z - mod : z);
      }
}
vl conv(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};
  int s = (int)a.size() + (int)b.size() - 1,
      B = 32 - __builtin_clz(s), n = 1 << B;
  ll inv = modpow(n, mod - 2);
  vl L(a), R(b), out(n);
  L.resize(n), R.resize(n);
  ntt(L), ntt(R);
  for (int i = 0; i < n; i++)
    out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
  ntt(out);
  return {out.begin(), out.begin() + s};
}
```

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x \oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two. Keep only the line(s) marked with your operation.
Usage: vector<int> c = conv(a, b); // sz power of 2; AND (or OR/XOR)
Time: $\mathcal{O}(N \log N)$

15 lines

```
void FST(vector<int>& a, bool inv) {
  for (int n = a.size(), s = 1; s < n; s *= 2)
    for (int i = 0; i < n; i++) if (!(i & s)) {
      int &u = a[i], &v = a[i + s], x = u, y = v;
      if (inv) u = y - x, v = x; else u = y, v = x + y; // AND
      if (inv) u = y, v = x - y; else u = x + y, v = x; // OR
      u = x + y, v = x - y; // XOR
    }
  if (inv) for (int& x : a) x /= (int)a.size(); // XOR only
}
vector<int> conv(vector<int> a, vector<int> b) {
  FST(a, 0); FST(b, 0);
  for (int i = 0; i < a.size(); i++) a[i] *= b[i];
  FST(a, 1); return a;
}
```

4.5 Formal power series

PolyOps.h

Description: Formal power series mod x^n (mod = 998244353, from NTT): inverse ($a_0 \neq 0$), log ($a_0 = 1$), exp ($a_0 = 0$), sqrt ($a_0 = 1$; otherwise start b with ModSqrt of a_0), and division with remainder ($a = bq + r$, $\deg r < \deg b$, b.back() $\neq 0$). For powers use $a^k = \exp(k \log a)$ when $a_0 = 1$. Typical use: counting with generating functions, e.g. sets of connected structures = exp(·), sequences = $1/(1-\cdot)$.
Usage: vl b = polyInv(a, n); // a*b = 1 in the first n terms
auto [q, r] = polyDiv(a, b);
Time: $\mathcal{O}(n \log n)$ each (exp has a large constant)

"NumberTheoreticTransform.h" 60 lines

```
vl polyInv(const vl& a, int n) {
  vl b = {modpow(a[0], mod - 2)};
  for (int k = 1; k < n; k *= 2) { // b = b(2 - ab)
    vl c(a.begin(), a.begin() + min((int)a.size(), 2 * k));
    c = conv(c, conv(b, b)); c.resize(2 * k); b.resize(2 * k);
    for (int i = 0; i < 2 * k; i++)
      b[i] = (2 * b[i] - c[i] + mod) % mod;
  }
  b.resize(n); return b;
}
vl deriv(vl a) {
  for (int i = 1; i < a.size(); i++) a[i-1] = a[i] * i % mod;
  if (a.size()) a.pop_back();
  return a;
}
vl integ(vl a) {
  a.push_back(0);
  for (int i = (int)a.size() - 1; i; i--)
    a[i] = a[i-1] * modpow(i, mod - 2) % mod;
  a[0] = 0; return a;
}
vl polyLog(const vl& a, int n) {
  vl r = integ(conv(deriv(a), polyInv(a, n)));
  r.resize(n); return r;
}
vl polyExp(const vl& a, int n) { // b = b(1 - log b + a)
  vl b = {1};
  for (int k = 1; k < n; k *= 2) {
    vl c = polyLog(b, 2 * k);
    for (int i = 0; i < 2 * k; i++)
      c[i] = ((i < (int)a.size() ? a[i] : 0) - c[i] + mod) %
        mod;
    c[0] = (c[0] + 1) % mod;
    b = conv(b, c); b.resize(2 * k);
  }
  b.resize(n); return b;
}
vl polySqrt(const vl& a, int n) { // b = (b + a / b) / 2
  vl b = {1};
  for (int k = 1; k < n; k *= 2) {
    vl c(a.begin(), a.begin() + min((int)a.size(), 2 * k));
    c = conv(c, polyInv(b, 2 * k));
    c.resize(2 * k); b.resize(2 * k);
    for (int i = 0; i < 2 * k; i++)
      b[i] = (b[i] + c[i]) * ((mod + 1) / 2) % mod;
  }
  b.resize(n); return b;
}
pair<vl, vl> polyDiv(const vl& a, const vl& b) {
  int n = a.size(), m = b.size();
  if (n < m) return {{}, a};
  vl ra(a.rbegin(), a.rend()), rb(b.rbegin(), b.rend());
  ra.resize(n - m + 1);
  vl q = conv(ra, polyInv(rb, n - m + 1));
  q.resize(n - m + 1); reverse(q.begin(), q.end());
  vl r = conv(b, q); r.resize(m - 1);
  for (int i = 0; i < m - 1; i++)
    r[i] = (a[i] - r[i] + mod) % mod;
  return {q, r};
}
```

MultipointEval.h

Description: Evaluates a polynomial at many points, and interpolates the polynomial of degree $< n$ through n points with distinct x, mod 998244353. Uses the product tree of $(X - x_i)$ (node v has children $2v$, $2v+1$; leaves $n..2n-1$).
Usage: vl y = multiEval(a, xs); vl p = interpolate(xs, ys);
Time: $\mathcal{O}\left(n \log^2 n\right)$

"PolyOps.h" 39 lines

```
struct SubProd {
  int n; vector<vl> t;
  SubProd(const vl& x) : n(x.size()), t(2 * n) {
    for (int i = 0; i < n; i++)
      t[n + i] = {(mod - x[i]) % mod, 1};
    for (int i = n-1; i > 0; i--) t[i] = conv(t[2*i],t[2*i+1]);
  }
  void eval(int v, vl a, vl& res) {
    a = polyDiv(a, t[v]).second;
    if (v >= n) {
      res[v - n] = (int)a.size() ? a[0] : 0;
      return;
    }
    eval(2 * v, a, res), eval(2 * v + 1, a, res);
  }
};
vl multiEval(const vl& a, const vl& x) {
  vl res(x.size());
  if (x.size()) SubProd(x).eval(1, a, res);
  return res;
}
vl interpolate(const vl& x, const vl& y) {
```

```
  int n = x.size();
  if (!n) return {};
  SubProd s(x); vl w(n);
  s.eval(1, deriv(s.t[1]), w);
  vector<vl> u(2 * n);
  for (int i = 0; i < n; i++)
    u[n + i] = {y[i] * modpow(w[i], mod - 2) % mod};
  for (int i = n - 1; i > 0; i--) {
    vl a = conv(u[2*i], s.t[2*i+1]);
    vl b = conv(u[2*i+1], s.t[2*i]);
    a.resize(max((int)a.size(), (int)b.size()));
    for (int j = 0; j < b.size(); j++)
      a[j] = (a[j] + b[j]) % mod;
    u[i] = a;
  }
  u[1].resize(n); return u[1];
}
```

Number theory (5)

5.1 Modular arithmetic

ModularArithmetic.h

Description: Operators for modular arithmetic. You need to set mod to some number first and then you can use the structure.
Usage: `Mod a(5), b(3); Mod c = a / b, d = a ^ 10; // c.x`

"euclid.h" 19 lines

```
const ll mod = 17; // change to something else
struct Mod {
  ll x;
  Mod(ll y) : Mod(y%mod+mod,0){}
  Mod(ll y,int) : x(y<mod?y:y-mod){}
  Mod operator+(Mod b) { return {x + b.x,0}; }
  Mod operator-(Mod b) { return {x - b.x + mod,0}; }
  Mod operator*(Mod b) { return {x * b.x % mod,0}; }
  Mod operator/(Mod b) { return *this * invert(b); }
  Mod invert(Mod a) {
    ll x, y, g = euclid(a.x, mod, x, y);
    assert(g == 1); return x;
  }
  Mod operator^(ll e) {
    if (!e) return 1;
    Mod r = *this ^ (e / 2); r = r * r;
    return e&1 ? *this * r : r;
  }
};
```

ModInverse.h

Description: Pre-computation of modular inverses. Assumes LIM $\leq$ mod and that mod is a prime.
Usage: `// then inv[i] * i = 1 (mod mod) for 1 <= i < LIM`

4 lines

```
const ll mod = 1000000007, LIM = 200000;
ll* inv = new ll[LIM] - 1; inv[1] = 1;
for (int i = 2; i < LIM; i++)
  inv[i] = mod - (mod / i) * inv[mod % i] % mod;
```

ModPow.h

Description: $b^e \bmod \text{mod}$. Files that include this file may set their own mod (e.g. NTT); keep only one definition.
Usage: `ll x = modpow(b, e); // b^e mod mod`

8 lines

```
const ll mod = 1000000007; // faster if const
ll modpow(ll b, ll e) {
  ll ans = 1;
  for (; e; b = b * b % mod, e /= 2)
    if (e & 1) ans = ans * b % mod;
  return ans;
}
```

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. modLog(a,1,m) can be used to calculate the order of a.
Usage: `ll x = modLog(a, b, m); // smallest x > 0, a^x = b, or -1`
Time: $\mathcal{O}\left(\sqrt{m}\right)$

11 lines

```
ll modLog(ll a, ll b, ll m) {
  ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
  unordered_map<ll, ll> A;
  while (j <= n && (e = f = e * a % m) != b % m)
    A[e * b % m] = j++;
  if (e == b % m) return j;
  if (__gcd(m, e) == __gcd(m, b))
    for (int i = 2; i < n+2; i++) if (A.count(e = e * f % m))
      return n * i - A[e];
  return -1;
}
```

ModSum.h

Description: Sums of mod'ed arithmetic progressions.
`modsum(to, c, k, m)` $= \sum_{i=0}^{\text{to}-1} (ki + c)\%m$. `divsum` is similar but for floored division.
Usage: `ll s = modsum(n, c, k, m); // sum_{i<n} (k*i + c) mod m`
Time: $\log(m)$, with a large constant.

16 lines

```
typedef unsigned long long ull;
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }

ull divsum(ull to, ull c, ull k, ull m) {
  ull res = k / m * sumsq(to) + c / m * to;
  k %= m; c %= m;
  if (!k) return res;
  ull to2 = (to * k + c) / m;
  return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}

ll modsum(ull to, ll c, ll k, ll m) {
  c = ((c % m) + m) % m;
  k = ((k % m) + m) % m;
  return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

ModMulLL.h

Description: Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.
Usage: `ull r = modmul(a, b, m), p = modpow(b, e, m);`
Time: $\mathcal{O}(1)$ for modmul, $\mathcal{O}(\log b)$ for modpow

11 lines

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
  ll ret = a * b - M * ull(1.L / M * a * b);
  return ret + M * (ret < 0) - M * (ret >= (ll)M);
}
ull modpow(ull b, ull e, ull mod) {
  ull ans = 1;
  for (; e; b = modmul(b, b, mod), e /= 2)
    if (e & 1) ans = modmul(ans, b, mod);
  return ans;
}
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution). Requires $p < 3 \cdot 10^9$ (products fit in ll).
Usage: `ll x = sqrt(a, p); // x*x = a (mod p); asserts if no root`
Time: $\mathcal{O}\left(\log^2 p\right)$ worst case, $\mathcal{O}(\log p)$ for most p

30 lines

```
ll modpow(ll b, ll e, ll m) { // (or use ModMulLL.h)
  ll r = 1;
  for (b %= m; e; b = b * b % m, e /= 2)
    if (e & 1) r = r * b % m;
  return r;
}
ll sqrt(ll a, ll p) {
  a %= p; if (a < 0) a += p;
  if (a == 0) return 0;
  assert(modpow(a, (p-1)/2, p) == 1); // else no solution
  if (p % 4 == 3) return modpow(a, (p+1)/4, p);
  // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
  ll s = p - 1, n = 2;
  int r = 0, m;
  while (s % 2 == 0)
    ++r, s /= 2;
  while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
  ll x = modpow(a, (s + 1) / 2, p);
  ll b = modpow(a, s, p), g = modpow(n, s, p);
  for (;; r = m) {
    ll t = b;
    for (m = 0; m < r && t != 1; ++m)
      t = t * t % p;
    if (m == 0) return x;
    ll gs = modpow(g, 1LL << (r - m - 1), p);
    g = gs * gs % p;
    x = x * gs % p;
    b = b * g % p;
  }
}
```

PrimitiveRoot.h

Description: Smallest primitive root g modulo a prime p: the powers of g hit every nonzero residue. g is primitive iff $g^{(p-1)/q} \neq 1$ for every prime $q \mid p-1$. Needed e.g. for NTT with an unusual prime, or discrete k-th roots ($x^k = a$: write $x = g^y$, $a = g^{\log_g a}$, solve $ky \equiv \log a \pmod{p-1}$). Primitive roots mod m exist iff $m = 1, 2, 4, p^k, 2p^k$.
Usage: `ll g = primitiveRoot(998244353); // 3`
Time: $\mathcal{O}\left(\sqrt{p}\right)$ for factoring $p-1$ (use Factor.h for big p)

"ModMulLL.h" 13 lines

```
ll primitiveRoot(ll p) {
  vector<ll> f; ll x = p - 1;
  for (ll d = 2; d * d <= x; d++) if (x % d == 0) {
    f.push_back(d);
    while (x % d == 0) x /= d;
  }
  if (x > 1) f.push_back(x);
  for (ll g = 1;; g++) {
    bool ok = 1;
```

```
    for (ll q : f) ok &= modpow(g, (p - 1) / q, p) != 1;
    if (ok) return g;
  }
}
```

5.2 Primality

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than LIM.
Usage: `vector<int> pr = eratosthenes(); // primes < LIM; isPrime[x]`
Time: LIM=1e9 $\approx$ 1.5s

21 lines

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vector<int> eratosthenes() {
  const int S = (int)round(sqrt(LIM)), R = LIM / 2;
  vector<int> pr = {2}, sieve(S + 1);
  pr.reserve((int)(LIM / log(LIM) * 1.1));
  vector<pair<int, int>> cp;
  for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
    cp.push_back({i, i * i / 2});
    for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
  }
  for (int L = 1; L <= R; L += S) {
    array<bool, S> block{};
    for (auto &[p, idx] : cp)
      for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
    for (int i = 0; i < (min(S, R - L)); i++)
      if (!block[i]) pr.push_back((L + i) * 2 + 1);
  }
  for (int i : pr) isPrime[i] = 1;
  return pr;
}
```

LinearSieve.h

Description: Linear sieve: lp[i] = smallest prime factor, pr = all primes, plus Euler's phi and Möbius mu, for all $i <$ LIM. Factorize $x <$ LIM in $O(\log x)$: while $(x > 1)$ $p = \text{lp}[x]$, $x /= p$. Any multiplicative f works the same way: split on whether $p = \text{lp}[i]$.
Usage: `sieve(); // then lp, pr, phi, mu`
Time: $\mathcal{O}(LIM)$

16 lines

```
const int LIM = 1e6; // ~12 bytes per number
int lp[LIM], phi[LIM], mu[LIM];
vector<int> pr;
void sieve() {
  phi[1] = mu[1] = 1;
  for (int i = 2; i < LIM; i++) {
    if (!lp[i])
      lp[i] = i, pr.push_back(i), phi[i] = i - 1, mu[i] = -1;
    for (int p : pr) {
      if (p > lp[i] || (ll)i * p >= LIM) break;
      lp[i * p] = p;
      phi[i * p] = phi[i] * (p == lp[i] ? p : p - 1);
      mu[i * p] = p == lp[i] ? 0 : -mu[i];
    }
  }
}
```

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
Usage: `bool p = isPrime(n); // deterministic, n < 7e18`
Time: 7 times the complexity of $a^b \mod c$.

"ModMulLL.h" 12 lines

```
bool isPrime(ull n) {
  if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
  ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
      s = __builtin_ctzll(n-1), d = n >> s;
  for (ull a : A) {   // ^ count trailing zeroes
    ull p = modpow(a%n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
      p = modmul(p, p, n);
    if (p != n-1 && i != s) return 0;
  }
  return 1;
}
```

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Usage: `vector<ull> f = factor(n); // unsorted, with repeats`
Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

"ModMulLL.h", "MillerRabin.h" 18 lines

```
ull pollard(ull n) {
  ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  auto f = [&](ull x) { return modmul(x, x, n) + i; };
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
    x = f(x), y = f(f(y));
  }
  return __gcd(prd, n);
}
vector<ull> factor(ull n) {
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ull x = pollard(n);
  auto l = factor(x), r = factor(n / x);
  l.insert(l.end(), r.begin(), r.end());
  return l;
}
```

5.3 Divisibility

euclid.h

Description: Finds two integers x and y, such that $ax+by = \gcd(a,b)$. If you just need gcd, use the built in __gcd instead. If a and b are coprime, then x is the inverse of $a \pmod b$.
Usage: `ll x, y, g = euclid(a, b, x, y); // a*x + b*y = g`

5 lines

```
ll euclid(ll a, ll b, ll &x, ll &y) {
  if (!b) return x = 1, y = 0, a;
  ll d = euclid(b, a % b, y, x);
  return y -= a/b * x, d;
}
```

CRT.h

Description: Chinese Remainder Theorem.
`crt(a, m, b, n)` computes x such that $x \equiv a \pmod m$, $x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \le x < \text{lcm}(m,n)$. Assumes $mn < 2^{62}$.
Usage: `ll x = crt(a, m, b, n); // x = a mod m, x = b mod n`
Time: $\log(n)$

"euclid.h" 7 lines

```
ll crt(ll a, ll m, ll b, ll n) {
  if (n > m) swap(a, b), swap(m, n);
  ll x, y, g = euclid(m, n, x, y);
  assert((a - b) % g == 0); // else no solution
  x = (b - a) % n * x % n / g * m + a;
  return x < 0 ? x + m*n/g : x;
}
```

5.3.1 Bézout's identity

For $a \neq 0$, $b \neq 0$, then $d = gcd(a,b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

Description: *Euler's* ϕ function is defined as $\phi(n) := \#$ of positive integers $\le n$ that are coprime with n. $\phi(1) = 1$, p prime $\Rightarrow \phi(p^k) = (p-1)p^{k-1}$, m, n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1} p_2^{k_2} ... p_r^{k_r}$ then $\phi(n) = (p_1-1)p_1^{k_1-1} ... (p_r-1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n}(1 - 1/p)$. $\sum_{d|n} \phi(d) = n$, $\sum_{1 \le k \le n, \gcd(k,n)=1} k = n\phi(n)/2, n > 1$
Euler's thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.
Fermat's little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod p$ $\forall a$.
Usage: `calculatePhi(); // then phi[i] for i < LIM`

8 lines

```
const int LIM = 5000000;
int phi[LIM];

void calculatePhi() {
  for (int i = 0; i < LIM; i++) phi[i] = i&1 ? i : i/2;
  for (int i = 3; i < LIM; i += 2) if(phi[i] == i)
    for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}
```

5.4 Fractions

ContinuedFractions.h

Description: Given N and a real number $x \ge 0$, finds the closest rational approximation p/q with $p, q \le N$. It will obey $|p/q - x| \le 1/qN$. For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. ($p_k/q_k$ alternates between $> x$ and $< x$.) If x is rational, y eventually becomes ∞; if x is the root of a degree 2 polynomial the a's eventually become cyclic.
Usage: `auto [p, q] = approximate(x, N); // p, q <= N`
Time: $\mathcal{O}(\log N)$

21 lines

```
typedef double d; // for N ~ 1e7; long double for N ~ 1e9
pair<ll, ll> approximate(d x, ll N) {
  ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX; d y = x;
```

```
  for (;;) {
    ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ) / Q : inf),
       a = (ll)floor(y), b = min(a, lim),
       NP = b*P + LP, NQ = b*Q + LQ;
    if (a > b) {
      // If b > a/2, we have a semi-convergent that gives us a
      // better approximation; if b = a/2, we *may* have one.
      // Return {P, Q} here for a more canonical approximation.
      return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q)) ?
        make_pair(NP, NQ) : make_pair(P, Q);
    }
    if (abs(y = 1/(y - (d)a)) > 3*N) {
      return {NP, NQ};
    }
    LP = P; P = NP;
    LQ = Q; Q = NQ;
  }
}
```

FracBinarySearch.h

Description: Given f and N, finds the smallest fraction $p/q \in [0,1]$ such that $f(p/q)$ is true, and $p, q \le N$. You may want to throw an exception from f if it finds an exact solution, in which case N can be removed.

Usage: `fracBS([](Frac f) { return f.p>=3*f.q; }, 10); // {1,3}`

Time: $\mathcal{O}(\log(N))$

24 lines

```
struct Frac { ll p, q; };

Frac fracBS(function<bool(Frac)> f, ll N) {
  bool dir = 1, A = 1, B = 1;
  Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
  if (f(lo)) return lo;
  assert(f(hi));
  while (A || B) {
    ll adv = 0, step = 1; // move hi if dir, else lo
    for (int si = 0; step; (step *= 2) >>= si) {
      adv += step;
      Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
      if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
        adv -= step; si = 2;
      }
    }
    hi.p += lo.p * adv;
    hi.q += lo.q * adv;
    dir = !dir;
    swap(lo, hi);
    A = B; B = !!adv;
  }
  return dir ? hi : lo;
}
```

5.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2),\ \ b = k \cdot (2mn),\ \ c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

5.6 Theorems

Fermat/Euler: $a^{\phi(m)} \equiv 1 \pmod m$ for $\gcd(a,m) = 1$, so $a^{-1} \equiv a^{p-2} \pmod p$.

Power tower / huge exponents: for any a and $b \ge \log_2 m$: $a^b \equiv a^{(b \bmod \phi(m))+\phi(m)} \pmod m$ (no coprimality needed); apply recursively for $a^{b^{c^{\cdots}}}$ (ϕ reaches 1 in $O(\log m)$ steps).

Wilson: p prime $\iff (p-1)! \equiv -1 \pmod p$.

Hensel lifting: if $f(x) \equiv 0 \pmod{p^k}$ and $f'(x) \not\equiv 0 \pmod p$ then $x' = x - f(x)f'(x)^{-1}$ solves it mod p^{k+1} (Newton). Solve mod each p^k, merge with CRT.

Quadratic residues: a is a square mod odd prime p iff $a^{(p-1)/2} \equiv 1$ (Euler's criterion).

Frobenius / Chicken McNugget: the largest number not of the form $ax + by$ ($x, y \ge 0$, $\gcd(a,b) = 1$) is $ab - a - b$; there are $\frac{(a-1)(b-1)}{2}$ such numbers.

$\gcd(a^m - 1, a^n - 1) = a^{\gcd(m,n)} - 1$; $\gcd(F_m, F_n) = F_{\gcd(m,n)}$ (Fibonacci).

5.7 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit), $2^{61} - 1$ (Mersenne). There are 78498 primes less than 1 000 000, 664579 below 10^7, 5761455 below 10^8, 50847534 below 10^9; $\pi(n) \approx n/\ln n$. Max prime gap below 10^9 is 282, below 10^{18} it is < 1500.

NTT primes $c \cdot 2^k + 1$ (primitive root g): 998244353 ($119 \cdot 2^{23}$+1, g=3), 167772161 ($5 \cdot 2^{25}$+1, g=3), 469762049 ($7 \cdot 2^{26}$+1, g=3), 1004535809 ($479 \cdot 2^{21}$+1, g=3), 754974721 ($45 \cdot 2^{24}$+1, g=11).

Other: 10^9+7 (g=5), 10^9+9 (g=13), 10^{18}+3.

Handy primes: 100003, 200003, 300007, 400009, 500009, 600011, 700001, 800011, 900001, 1000003, 2000003, 3000017, 4100011, 5000011, 8000009, 9000011, 10000019, 20000003, 50000017, 50100007, 100000007, 100200011, 200100007, 250000019.

Primitive roots exist modulo any prime power p^a, except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.8 Estimates

$\sum_{d|n} d = O(n \log\log n)$.

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$ (exact maxima: 1344 for $n \le 10^9$, 103680 for $n \le 10^{18}$). A number $\le 10^{18}$ has at most 15 distinct prime factors.

$\sum_{i=1}^{n} \lfloor n/i \rfloor = O(n \log n)$ and $\lfloor n/i \rfloor$ takes $O(\sqrt{n})$ distinct values: `for (l = 1; l <= n; l = r + 1) { r = n / (n / l); ... }`.

5.9 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g(n/d)$$

Other useful formulas/forms:

$\sum_{d|n} \mu(d) = [n = 1]$ (very useful)

$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n) g(d)$

$g(n) = \sum_{1 \le m \le n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \le m \le n} \mu(m) g(\lfloor \frac{n}{m} \rfloor)$

Combinatorial (6)

6.1 Permutations

6.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

IntPerm.h

Description: Permutation -> integer conversion. (Not order preserving.) Integer -> permutation can use a lookup table.

Usage: `int code = permToInt(p); // p: perm of 0..n-1, code < n!`

Time: $\mathcal{O}(n)$

6 lines

```
int permToInt(vector<int>& v) {
  int use = 0, i = 0, r = 0;
  for(int x:v) r = r * ++i + __builtin_popcount(use & -(1<<x)),
    use |= 1 << x;                    // (note: minus, not ~!)
  return r;
}
```

6.1.2 Cycles

Let $g_S(n)$ be the number of n-permutations whose cycle lengths all belong to the set S. Then

$$\sum_{n=0}^{\infty} g_S(n) \frac{x^n}{n!} = \exp\left(\sum_{n \in S} \frac{x^n}{n}\right)$$

6.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rceil$$

$D(n) =$
$1, 0, 1, 2, 9, 44, 265, 1854, 14833, 133496, 1334961, \ldots$

6.1.4 Burnside's lemma

Given a group G of symmetries and a set X, the number of elements of X *up to symmetry* equals

$$\frac{1}{|G|}\sum_{g\in G}|X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).
If $f(n)$ counts "configurations" (of some sort) of length n, we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n}\sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n}\sum_{k|n} f(k)\phi(n/k).$$

Pólya: colouring positions with k colours, $|X^g| = k^{c(g)}$ where $c(g)$ = number of cycles of g on the positions. Necklaces (n beads, rotations): $N = \frac{1}{n}\sum_{d|n}\phi(d)k^{n/d}$. Bracelets (+ reflections): $\frac{N}{2} + \frac{k^{(n+1)/2}}{2}$ for odd n, $\frac{N}{2} + \frac{(k+1)k^{n/2}}{4}$ for even n.

6.2 Partitions and subsets

6.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1,\ p(n) = \sum_{k\in\mathbb{Z}\setminus\{0\}} (-1)^{k+1} p(n - k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	∼2e5	∼2e8

6.2.2 Lucas' Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + ... + n_1 p + n_0$ and $m = m_k p^k + ... + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^{k}\binom{n_i}{m_i} \pmod p$.

6.2.3 Binomials

$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$, $k\binom{n}{k} = n\binom{n-1}{k-1}$, $\sum_k \binom{n}{k}^2 = \binom{2n}{n}$, $\sum_{i=r}^{n}\binom{i}{r} = \binom{n+1}{r+1}$ (hockey stick), $\sum_k \binom{r}{k}\binom{s}{n-k} = \binom{r+s}{n}$ (Vandermonde), $\binom{n}{m}\binom{m}{k} = \binom{n}{k}\binom{n-k}{m-k}$.

Stars and bars: $x_1 + \cdots + x_k = n$ has $\binom{n+k-1}{k-1}$ solutions with $x_i \geq 0$, $\binom{n-1}{k-1}$ with $x_i \geq 1$.

Inclusion–exclusion: $|\bigcup A_i| = \sum_{\emptyset\neq S}(-1)^{|S|+1}|\bigcap_{i\in S} A_i|$; e.g. surjections $[n] \to [k]$: $\sum_j (-1)^j\binom{k}{j}(k-j)^n = k!\,S(n,k)$.

Ballot: sequences of m (+1) and n (−1), $m \geq n$, with all prefix sums ≥ 0: $\binom{m+n}{n} - \binom{m+n}{n-1}$ (reflection trick; $m = n$ gives Catalan).

Binomial.h

Description: $\binom{n}{k} \bmod p$ for $n <$ LIM with precomputed factorials, p prime $\geq$ LIM. For huge n and small prime p (set LIM $= p$) use lucas: $\binom{n}{k} \equiv \prod \binom{n_i}{k_i}$ over base-p digits.
Usage: `initFact(); ll c = C(n, k);`
Time: $\mathcal{O}(LIM)$ precomputation, $\mathcal{O}(1)$ per query

"../number-theory/ModPow.h" 18 lines

```
const int LIM = 1e6 + 5; // > max n (for lucas: LIM = mod)
ll fact[LIM], ifact[LIM];
void initFact() {
  fact[0] = 1;
  for (int i = 1; i < LIM; i++) fact[i] = fact[i-1] * i % mod;
  ifact[LIM-1] = modpow(fact[LIM-1], mod - 2);
  for (int i = LIM-1; i; i--) ifact[i-1] = ifact[i] * i % mod;
}
ll C(ll n, ll k) {
  if (k < 0 || k > n) return 0;
  return fact[n] * ifact[k] % mod * ifact[n-k] % mod;
}
ll lucas(ll n, ll k) { // needs mod < LIM
  ll r = 1;
  for (; n || k; n /= mod, k /= mod)
    r = r * C(n % mod, k % mod) % mod;
  return r;
}
```

multinomial.h

Description: Computes $\binom{k_1 + \cdots + k_n}{k_1, k_2, \ldots, k_n} = \frac{(\sum k_i)!}{k_1!k_2!...k_n!}$.
Usage: `ll c = multinomial(v); // (sum v)! / prod(v[i]!)`

6 lines

```
ll multinomial(vector<int>& v) {
  ll c = 1, m = v.empty() ? 1 : v[0];
  for (int i = 1; i < v.size(); i++)
    for (int j = 0; j < v[i]; j++) c = c * ++m / (j + 1);
  return c;
}
```

6.3 General purpose numbers

6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t - 1}$ (FFT-able).
$B[0, \ldots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \ldots]$
Sums of powers:

$$\sum_{i=1}^{n} n^m = \frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^\infty f(x)dx - \sum_{k=1}^{\infty}\frac{B_k}{k!}f^{(k-1)}(m)$$

$$\approx \int_m^\infty f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

6.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ c(0,0) = 1$$
$$\sum_{k=0}^{n} c(n,k)x^k = x(x+1)\ldots(x+n-1)$$

$c(8,k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$
$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \ldots$

6.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j:s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j:s s.t. $\pi(j) \geq j$, k j:s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$

$$E(n,0) = E(n,n-1) = 1$$

$$E(n,k) = \sum_{j=0}^{k}(-1)^j\binom{n+1}{j}(k+1-j)^n$$

6.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$

$$S(n,1) = S(n,n) = 1$$

$$S(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$$

6.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \ldots$. For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod p$$

6.3.6 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i: $n_1 n_2 \cdots n_k n^{k-2}$
with degrees d_i: $(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

6.3.7 Catalan numbers

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2}C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \ldots$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

Graph (7)

7.1 Fundamentals

BellmanFord.h

Description: Calculates shortest paths from s in a graph that might have negative edge weights. Unreachable nodes get dist = inf; nodes reachable through negative-weight cycles get dist = -inf. Assumes $V^2 \max|w_i| < \sim 2^{63}$.
Usage: `vector<Node> nodes(n); vector<Ed> eds = {{a, b, w}, ...}; bellmanFord(nodes, eds, s); // nodes[v].dist, .prev`
Time: $\mathcal{O}(VE)$

24 lines

```
const ll inf = LLONG_MAX;
struct Ed { int a, b, w, s() { return a < b ? a : -a; }};
struct Node { ll dist = inf; int prev = -1; };

void bellmanFord(vector<Node>& nodes, vector<Ed>& eds, int s) {
  nodes[s].dist = 0;
  sort(eds.begin(), eds.end(),
    [](Ed a, Ed b) { return a.s() < b.s(); });

  int lim = (int)nodes.size() / 2 + 2; // /3+100 if shuffled
  for (int i = 0; i < lim; i++) for (Ed ed : eds) {
    Node cur = nodes[ed.a], &dest = nodes[ed.b];
    if (abs(cur.dist) == inf) continue;
    ll d = cur.dist + ed.w;
    if (d < dest.dist) {
      dest.prev = ed.a;
      dest.dist = (i < lim-1 ? d : -inf);
    }
  }
  for (int i = 0; i < lim; i++) for (Ed e : eds) {
    if (nodes[e.a].dist == -inf)
      nodes[e.b].dist = -inf;
  }
}
```

FloydWarshall.h

Description: Calculates all-pairs shortest path in a directed graph that might have negative edge weights. Input is an distance matrix m, where $m[i][j] =$ `inf` if i and j are not adjacent. As output, $m[i][j]$ is set to the shortest distance between i and j, `inf` if no path, or `-inf` if the path goes through a negative-weight cycle.
Usage: `vector<vector<ll>> m(n, vector<ll>(n, inf)); m[a][b] = min(m[a][b], w); floydWarshall(m);`
Time: $\mathcal{O}\left(N^3\right)$

17 lines

```
const ll inf = 1LL << 62;
void floydWarshall(vector<vector<ll>>& m) {
  int n = m.size();
  for (int i = 0; i < n; i++) m[i][i] = min(m[i][i], 0LL);
  for (int k = 0; k < n; k++)
    for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
        if (m[i][k] != inf && m[k][j] != inf) {
          auto newDist = max(m[i][k] + m[k][j], -inf);
          m[i][j] = min(m[i][j], newDist);
        }
  for (int k = 0; k < n; k++)
    if (m[k][k] < 0)
      for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
          if (m[i][k] != inf && m[k][j] != inf) m[i][j] = -inf;
}
```

TopoSort.h

Description: Topological sorting. Given is an oriented graph. Output is an ordering of vertices, such that there are edges only from left to right. If there are cycles, the returned list will have size smaller than n – nodes reachable from cycles will not be returned.
Usage: `vector<int> ord = topoSort(adj); // (int)ord.size() < n iff there is a cycle`
Time: $\mathcal{O}(|V| + |E|)$

9 lines

```
vector<int> topoSort(const vector<vector<int>>& gr) {
  vector<int> indeg(gr.size()), q;
  for (auto& li : gr) for (int x : li) indeg[x]++;
  for (int i = 0; i < gr.size(); i++)
    if (indeg[i] == 0) q.push_back(i);
  for (int j = 0; j < q.size(); j++) for (int x : gr[q[j]])
    if (--indeg[x] == 0) q.push_back(x);
  return q;
}
```

MinMeanCycle.h

Description: Minimum mean weight of a directed cycle, $\min_C \frac{w(C)}{|C|}$ (Karp). Negative weights are fine. Returns 1e18 if the graph is acyclic. For the maximum, negate all weights. Alternatively, binary search $\lambda +$ negative cycle check with weights $w - \lambda$ (works for ratio $\sum a / \sum b$ too).
Usage: `double mu = minMeanCycle(n, eds); // eds: {a, b, w}, a -> b`
Time: $\mathcal{O}(VE)$

17 lines

```
double minMeanCycle(int n, vector<array<ll, 3>>& eds) {
  const ll inf = LLONG_MAX / 4;
  vector<vector<ll>> d(n + 1, vector<ll>(n, inf));
  d[0].assign(n, 0); // walks of exactly k edges, any start
  for (int k = 0; k < n; k++)
    for (auto [a, b, w] : eds)
      if (d[k][a] < inf)
        d[k + 1][b] = min(d[k + 1][b], d[k][a] + w);
  double res = 1e18;
  for (int v = 0; v < n; v++) if (d[n][v] < inf) {
    double mx = -1e18;
    for (int k = 0; k < n; k++) if (d[k][v] < inf)
      mx = max(mx, double(d[n][v] - d[k][v]) / (n - k));
    res = min(res, mx);
  }
  return res;
}
```

SteinerTree.h

Description: Minimum Steiner tree: cheapest connected subgraph containing all k terminals of an undirected graph with non-negative weights. dp[mask][v] = cheapest tree spanning the terminals in mask plus vertex v: merge two submasks at v, then relax along edges with Dijkstra. Returns -1 if impossible.
Usage: `ll c = steiner(adj, terms); // adj[v] = {(u, w)}`
Time: $\mathcal{O}\left(3^k n + 2^k m \log m\right)$

26 lines

```
ll steiner(
  vector<vector<pair<int, int>>>& g, vector<int>& terms) {
  int n = g.size(), k = terms.size();
  const ll inf = LLONG_MAX / 4;
  if (!k) return 0;
  vector<vector<ll>> dp(1 << k, vector<ll>(n, inf));
  for (int i = 0; i < k; i++) dp[1 << i][terms[i]] = 0;
  for (int m = 1; m < 1 << k; m++) {
    vector<ll>& d = dp[m];
    for (int s = (m - 1) & m; s; s = (s - 1) & m)
      for (int v = 0; v < n; v++)
        d[v] = min(d[v], dp[s][v] + dp[m ^ s][v]);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>,
      greater<>> pq;
    for (int v = 0; v < n; v++)
      if (d[v] < inf) pq.push({d[v], v});
    while (!pq.empty()) {
      auto [c, v] = pq.top(); pq.pop();
      if (c > d[v]) continue;
      for (auto [u, w] : g[v]) if (c + w < d[u])
        pq.push({d[u] = c + w, u});
    }
  }
  ll r = *min_element(dp.back().begin(), dp.back().end());
  return r < inf ? r : -1;
}
```

7.2 Network flow

PushRelabel.h

Description: Push-relabel using the highest label selection rule and the gap heuristic. Quite fast in practice. To obtain the actual flow, look at positive values only.
Usage: `PushRelabel f(n); f.addEdge(a, b, c); ll F = f.calc(s, t);`
Time: $\mathcal{O}\left(V^2\sqrt{E}\right)$

49 lines

```
struct PushRelabel {
  struct Edge {
    int dest, back;
    ll f, c;
  };
  vector<vector<Edge>> g;
  vector<ll> ec;
  vector<Edge*> cur;
  vector<vector<int>> hs; vector<int> H;
  PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {}

  void addEdge(int s, int t, ll cap, ll rcap=0) {
    if (s == t) return;
    g[s].push_back({t, (int)g[t].size(), 0, cap});
    g[t].push_back({s, (int)g[s].size()-1, 0, rcap});
  }

  void addFlow(Edge& e, ll f) {
    Edge &back = g[e.dest][e.back];
    if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
    e.f += f; e.c -= f; ec[e.dest] += f;
    back.f -= f; back.c += f; ec[back.dest] -= f;
  }
  ll calc(int s, int t) {
    int v = g.size(); H[s] = v; ec[t] = 1;
    vector<int> co(2*v); co[0] = v-1;
    for (int i = 0; i < v; i++) cur[i] = g[i].data();
    for (Edge& e : g[s]) addFlow(e, e.c);

    for (int hi = 0;;) {
      while (hs[hi].empty()) if (!hi--) return -ec[s];
```

```
      int u = hs[hi].back(); hs[hi].pop_back();
      while (ec[u] > 0)  // discharge u
        if (cur[u] == g[u].data() + (int)g[u].size()) {
          H[u] = 1e9;
          for (Edge& e : g[u]) if (e.c && H[u] > H[e.dest]+1)
            H[u] = H[e.dest]+1, cur[u] = &e;
          if (++co[H[u]], !--co[hi] && hi < v)
            for (int i = 0; i < v; i++)
              if (hi < H[i] && H[i] < v)
                --co[H[i]], H[i] = v + 1;
          hi = H[u];
        } else if (cur[u]->c && H[u] == H[cur[u]->dest]+1)
          addFlow(*cur[u], min(ec[u], cur[u]->c));
        else ++cur[u];
    }
  }
  bool leftOfMinCut(int a) { return H[a] >= g.size(); }
};
```

Dinic.h

Description: Flow algorithm with complexity $O(VE\log U)$ where $U = \max|\text{cap}|$. $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{V}E)$ for bipartite matching.
Usage: `Dinic f(n); f.addEdge(a, b, cap); ll F = f.calc(s, t); f.leftOfMinCut(v); // after calc: v on the s side`

43 lines

```
struct Dinic {
  struct Edge {
    int to, rev;
    ll c, oc;
    ll flow() { return max(oc - c, 0LL); } // if you need flows
  };
  vector<int> lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
  void addEdge(int a, int b, ll c, ll rcap = 0) {
    adj[a].push_back({b, (int)adj[b].size(), c, c});
    adj[b].push_back({a, (int)adj[a].size() - 1, rcap, rcap});
  }
  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < (int)adj[v].size(); i++) {
      Edge& e = adj[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (ll p = dfs(e.to, t, min(f, e.c))) {
          e.c -= p, adj[e.to][e.rev].c += p;
          return p;
        }
    }
    return 0;
  }
  ll calc(int s, int t) {
    ll flow = 0; q[0] = s;
    for (int L = 0; L < 31; L++)
      do { // 'int L=30' maybe faster for random data
        lvl = ptr = vector<int>(q.size());
        int qi = 0, qe = lvl[s] = 1;
        while (qi < qe && !lvl[t]) {
          int v = q[qi++];
          for (Edge e : adj[v])
            if (!lvl[e.to] && e.c >> (30 - L))
              q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
        }
        while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
      } while (lvl[t]);
    return flow;
  }
  bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

MinCostMaxFlow.h

Description: Min-cost max-flow. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only.
Usage: `MCMF f(n); f.addEdge(a, b, cap, cost); auto [flow, cost] = f.maxflow(s, t);`
Time: $\mathcal{O}(FE\log(V))$ where F is max flow. $\mathcal{O}(VE)$ for setpi.

83 lines

```
#include <bits/extc++.h> // above #define int

const ll INF = numeric_limits<ll>::max() / 4;

struct MCMF {
  struct edge {
    int from, to, rev;
    ll cap, cost, flow;
  };
  int N;
  vector<vector<edge>> ed;
  vector<int> seen;
  vector<ll> dist, pi;
  vector<edge*> par;

  MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}

  void addEdge(int from, int to, ll cap, ll cost) {
    if (from == to) return;
    ed[from].push_back(
      edge{from, to, (int)ed[to].size(), cap, cost, 0});
    ed[to].push_back(
      edge{to, from, (int)ed[from].size() - 1, 0, -cost, 0});
  }

  void path(int s) {
    fill(seen.begin(), seen.end(), 0);
    fill(dist.begin(), dist.end(), INF);
    dist[s] = 0; ll di;

    __gnu_pbds::priority_queue<pair<ll, int>> q;
    vector<decltype(q)::point_iterator> its(N);
    q.push({ 0, s });

    while (!q.empty()) {
      s = q.top().second; q.pop();
      seen[s] = 1; di = dist[s] + pi[s];
      for (edge& e : ed[s]) if (!seen[e.to]) {
        ll val = di - pi[e.to] + e.cost;
        if (e.cap - e.flow > 0 && val < dist[e.to]) {
          dist[e.to] = val;
          par[e.to] = &e;
          if (its[e.to] == q.end())
            its[e.to] = q.push({ -dist[e.to], e.to });
          else
            q.modify(its[e.to], { -dist[e.to], e.to });
        }
      }
    }
    for (int i = 0; i < N; i++)
      pi[i] = min(pi[i] + dist[i], INF);
  }

  pair<ll, ll> maxflow(int s, int t) {
    ll totflow = 0, totcost = 0;
    while (path(s), seen[t]) {
      ll fl = INF;
      for (edge* x = par[t]; x; x = par[x->from])
        fl = min(fl, x->cap - x->flow);

      totflow += fl;
      for (edge* x = par[t]; x; x = par[x->from]) {
        x->flow += fl;
        ed[x->to][x->rev].flow -= fl;
      }
    }
    for (int i = 0; i < N; i++)
      for (edge& e : ed[i]) totcost += e.cost * e.flow;
    return {totflow, totcost/2};
  }

  // If some costs can be negative, call this before maxflow:
  void setpi(int s) { // (otherwise, leave this out)
    fill(pi.begin(), pi.end(), INF); pi[s] = 0;
    int it = N, ch = 1; ll v;
    while (ch-- && it--)
      for (int i = 0; i < N; i++) if (pi[i] != INF)
        for (edge& e : ed[i]) if (e.cap)
          if ((v = pi[i] + e.cost) < pi[e.to])
            pi[e.to] = v, ch = 1;
    assert(it >= 0); // negative cost cycle
  }
};
```

MinCut.h

Description: After running max-flow, the left side of a min-cut from s to t is given by all vertices reachable from s, only traversing edges with positive residual capacity.

GlobalMinCut.h

Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.
Usage: `auto [w, side] = globalMinCut(mat); // mat: weights`
Time: $\mathcal{O}(V^3)$

22 lines

```
pair<int, vector<int>> globalMinCut(vector<vector<int>> mat) {
  pair<int, vector<int>> best = {INT_MAX, {}};
  int n = mat.size();
  vector<vector<int>> co(n);
  for (int i = 0; i < n; i++) co[i] = {i};
  for (int ph = 1; ph < n; ph++) {
    vector<int> w = mat[0];
    size_t s = 0, t = 0;
    // O(V^2) -> O(E log V) with prio. queue
    for (int it = 0; it < n - ph; it++) {
      w[t] = INT_MIN;
      s = t, t = max_element(w.begin(), w.end()) - w.begin();
```

```
      for (int i = 0; i < n; i++) w[i] += mat[t][i];
    }
    best = min(best, {w[t] - mat[t][t], co[t]});
    co[s].insert(co[s].end(), co[t].begin(), co[t].end());
    for (int i = 0; i < n; i++) mat[s][i] += mat[t][i];
    for (int i = 0; i < n; i++) mat[i][s] = mat[s][i];
    mat[0][t] = INT_MIN;
  }
  return best;
}
```

GomoryHu.h

Description: Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.
Usage: `auto t = gomoryHu(n, eds); // eds {a, b, w}; mincut(u, v) = // min weight on the u-v path of tree t`
Time: $\mathcal{O}(V)$ Flow Computations

"PushRelabel.h" 13 lines

```
typedef array<ll, 3> Edge;
vector<Edge> gomoryHu(int N, vector<Edge> ed) {
  vector<Edge> tree;
  vector<int> par(N);
  for (int i = 1; i < N; i++) {
    PushRelabel D(N); // Dinic also works
    for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
    tree.push_back({i, par[i], D.calc(i, par[i])});
    for (int j = i+1; j < N; j++)
      if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
  }
  return tree;
}
```

7.3 Matching

HopcroftKarp.h

Description: Fast bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and r should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $r[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Usage: `vector<int> r(m, -1); int k = hopcroftKarp(g, r); // g[left] = right`
Time: $\mathcal{O}\left(E\sqrt{V}\right)$

22 lines

```
int hopcroftKarp(vector<vector<int>>& g, vector<int>& r) {
  int n = g.size(), res = 0;
  vector<int> l(n, -1), q(n), d(n);
  auto dfs = [&](auto f, int u) -> bool {
    int t = exchange(d[u], 0) + 1;
    for (int v : g[u])
      if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
        return l[u] = v, r[v] = u, 1;
    return 0;
  };
  for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0)) {
    for (int i = 0; i < n; i++)
      if (l[i] == -1) q[t++] = i, d[i] = 1;
    for (int i = 0; i < t; i++) for (int v : g[q[i]]) {
      if (r[v] == -1) f = 1;
      else if (!d[r[v]]) d[r[v]] = d[q[i]] + 1, q[t++] = r[v];
    }
    if (!f) return res;
    for (int i = 0; i < n; i++)
      if (l[i] == -1) res += dfs(dfs, i);
  }
}
```

DFSMatching.h

Description: Simple bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and *btoa* should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. *btoa*[i] will be the match for vertex i on the right side, or -1 if it's not matched.
Usage: `vector<int> btoa(m, -1); dfsMatching(g, btoa);`
Time: $\mathcal{O}(VE)$

24 lines

```
bool find(int j, vector<vector<int>>& g, vector<int>& btoa,
  vector<int>& vis) {
  if (btoa[j] == -1) return 1;
  vis[j] = 1; int di = btoa[j];
  for (int e : g[di])
    if (!vis[e] && find(e, g, btoa, vis)) {
      btoa[e] = di;
      return 1;
    }
  return 0;
}
int dfsMatching(vector<vector<int>>& g, vector<int>& btoa) {
  vector<int> vis;
  for (int i = 0; i < g.size(); i++) {
    vis.assign((int)btoa.size(), 0);
    for (int j : g[i])
      if (find(j, g, btoa, vis)) {
        btoa[j] = i;
        break;
      }
  }
  return (int)btoa.size() -
    (int)count(btoa.begin(), btoa.end(), -1);
}
```

MinimumVertexCover.h

Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.
Usage: `vector<int> c = cover(g, n, m); // left: 0..n-1, right: n+j`

"DFSMatching.h" 22 lines

```
vector<int> cover(vector<vector<int>>& g, int n, int m) {
  vector<int> match(m, -1);
  int res = dfsMatching(g, match);
  vector<bool> lfound(n, true), seen(m);
  for (int it : match) if (it != -1) lfound[it] = false;
  vector<int> q, cover;
  for (int i = 0; i < n; i++) if (lfound[i]) q.push_back(i);
  while (!q.empty()) {
    int i = q.back(); q.pop_back();
    lfound[i] = 1;
    for (int e : g[i]) if (!seen[e] && match[e] != -1) {
      seen[e] = true;
      q.push_back(match[e]);
    }
  }
  for (int i = 0; i < n; i++)
    if (!lfound[i]) cover.push_back(i);
  for (int i = 0; i < m; i++)
    if (seen[i]) cover.push_back(n + i);
  assert((int)cover.size() == res);
  return cover;
}
```

WeightedMatching.h

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes cost[N][M], where cost[i][j] = cost for L[i] to be matched with R[j], and returns (min cost, match), where L[i] is matched with R[match[i]]. Negate costs for max cost. Requires $N \leq M$ (add zero-cost dummy columns, or transpose, if needed).
Usage: `auto [cost, match] = weightedMatching(C);`
Time: $\mathcal{O}\left(N^2M\right)$

25 lines

```
pair<ll, vector<int>> weightedMatching(vector<vector<ll>> &C) {
  int i = C.size(), m = i ? (int)C[0].size() : 0, c, s, r;
  vector<ll> dist(m), pot(m); // ll -> double if needed
  vector<int> match(i), rev(m, -1), cols(m), prev(m);
  ll d = 0, nd, cost = 0;
  while (i--) {
    for (int c = 0; c < m; c++)
      dist[c] = C[i][c], cols[c] = c, prev[c] = i;
    for (s = 0;;) {
      for (int j = s; j < m; j++) {
        c = cols[j], nd = dist[c] - pot[c];
        if (j == s || d > nd) d = nd, swap(cols[s], cols[j]);
      }
      if ((r = rev[c = cols[s++]]) == -1) break;
      for (int j = 0; j < m; j++)
        if (dist[j] > (nd = C[r][j] - C[r][c] + dist[c]))
          dist[j] = nd, prev[j] = r;
    }
    cost += dist[c];
    while (s--) pot[cols[s]] = dist[cols[s]] - d;
    for (; r != i; swap(c, match[r]))
      r = rev[c] = prev[c];
  }
  return {cost, match};
}
```

GeneralMatching.h

Description: Maximum matching in a general (non-bipartite) undirected graph. Returns the size; mt[v] = partner of v or -1. For bipartite graphs prefer HopcroftKarp.
Usage: `vector<int> mt; int k = generalMatching(adj, mt);`
Time: $\mathcal{O}\left(V^3\right)$

62 lines

```
int generalMatching(vector<vector<int>>& g, vector<int>& mt) {
  int n = g.size(), res = 0;
  vector<int> p(n), base(n), q(n);
  vector<bool> used(n), bl(n);
  mt.assign(n, -1);
  auto lca = [&](int a, int b) -> int {
    vector<bool> seen(n);
    for (;;) {
      a = base[a]; seen[a] = 1;
```

```
    if (mt[a] < 0) break;
    a = p[mt[a]];
  }
  for (;; b = p[mt[b]]) if (seen[b = base[b]]) return b;
};
auto mark = [&](int v, int b, int x) {
  while (base[v] != b) {
    bl[base[v]] = bl[base[mt[v]]] = 1;
    p[v] = x; x = mt[v]; v = p[x];
  }
};
auto path = [&](int root) -> int {
  fill(used.begin(), used.end(), 0);
  fill(p.begin(), p.end(), -1);
  iota(base.begin(), base.end(), 0);
  int qh = 0, qt = 0;
  used[root] = 1;
  q[qt++] = root;
  while (qh < qt) {
    int v = q[qh++];
    for (int to : g[v]) {
      if (base[v] == base[to] || mt[v] == to) continue;
      if (to == root || (mt[to] >= 0 && p[mt[to]] >= 0)) {
        int b = lca(v, to);
        fill(bl.begin(), bl.end(), 0);
        mark(v, b, to);
        mark(to, b, v);
        for (int i = 0; i < n; i++)
          if (bl[base[i]]) {
            base[i] = b;
            if (!used[i]) used[i] = 1, q[qt++] = i;
          }
      } else if (p[to] < 0) {
        p[to] = v;
        if (mt[to] < 0) return to;
        used[mt[to]] = 1;
        q[qt++] = mt[to];
      }
    }
  }
  return -1;
};
for (int i = 0; i < n; i++) if (mt[i] < 0) {
  int v = path(i);
  if (v < 0) continue;
  res++;
  while (v >= 0) {
    int pv = p[v], nv = mt[pv];
    mt[v] = pv, mt[pv] = v, v = nv;
  }
}
return res;
}
```

7.4 DFS algorithms

SCC.h

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa. Components are numbered in reverse topological order: an edge $u \to v$ implies comp[u] $\geq$ comp[v] (a component only has edges to components with lower index), so the condensation DAG can be processed by iterating c = 0..ncomps-1.
Usage: scc(g); // g: vector<vector<int>> adjacency list
comp[v] = component of v, comps[c] = vertices of c, ncomps
Time: $\mathcal{O}(E+V)$

25 lines

```
vector<int> val, comp, z;
vector<vector<int>> comps;
int Time, ncomps;
int dfs(int j, vector<vector<int>>& g) {
  int low = val[j] = ++Time, x; z.push_back(j);
  for (int e : g[j]) if (comp[e] < 0)
    low = min(low, val[e] ?: dfs(e, g));

  if (low == val[j]) {
    comps.emplace_back();
    do {
      x = z.back(); z.pop_back();
      comp[x] = ncomps;
      comps.back().push_back(x);
    } while (x != j);
    ncomps++;
  }
  return val[j] = low;
}
void scc(vector<vector<int>>& g) {
  int n = g.size();
  val.assign(n, 0); comp.assign(n, -1); comps.clear();
  Time = ncomps = 0;
  for (int i = 0; i < n; i++) if (comp[i] < 0) dfs(i, g);
}
```

BiconnectedComponents.h

Description: Finds all biconnected components in an undirected multigraph, plus bridges and articulation points. In a biconnected component there are at least two internally disjoint paths between any two nodes (a cycle exists through them). A node can be in several components. comps[i] = edge ids of component i. An edge in no component is a bridge (not on any cycle). A node is an articulation point (isCut) iff removing it disconnects its component.
Usage: int eid = 0; ed.resize(N);
for each edge (a,b) {
ed[a].emplace_back(b, eid);
ed[b].emplace_back(a, eid++); }
bicomps(eid); // then read comps, isBridge[e], isCut[v]
Time: $\mathcal{O}(E+V)$

34 lines

```
vector<int> num, st, isBridge, isCut;
vector<vector<pair<int, int>>> ed;
vector<vector<int>> comps;
int Time;
int dfs(int at, int par) {
  int me = num[at] = ++Time, top = me, kids = 0;
  for (auto [y, e] : ed[at]) if (e != par) {
    if (num[y]) {
      top = min(top, num[y]);
      if (num[y] < me)
        st.push_back(e);
    } else {
      int si = st.size();
      int up = dfs(y, e);
      top = min(top, up); kids++;
      if (up >= me && (par != -1 || kids > 1)) isCut[at] = 1;
      if (up == me) {
        st.push_back(e);
        comps.emplace_back(st.begin() + si, st.end());
        st.resize(si);
      }
      else if (up < me) st.push_back(e);
      else isBridge[e] = 1;
    }
  }
  return top;
}

void bicomps(int nedges) {
  num.assign((int)ed.size(), 0);
  isCut.assign((int)ed.size(), 0);
  isBridge.assign(nedges, 0); comps.clear(); Time = 0;
  for (int i = 0; i < ed.size(); i++) if (!num[i]) dfs(i, -1);
}
```

2sat.h

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $(a||b)\&\&(!a||c)\&\&(d||!b)\&\&...$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).
Usage: TwoSat ts(number of boolean variables);
ts.either(0, ~3); // Var 0 is true or var 3 is false
ts.setValue(2); // Var 2 is true
ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true
ts.solve(); // Returns true iff it is solvable
ts.values[0..N-1] holds the assigned values to the vars
Time: $\mathcal{O}(N+E)$, where N is the number of boolean variables, and E is the number of clauses.

57 lines

```
struct TwoSat {
  int N;
  vector<vector<int>> gr;
  vector<int> values; // 0 = false, 1 = true

  TwoSat(int n = 0) : N(n), gr(2*n) {}

  int addVar() { // (optional)
    gr.emplace_back();
    gr.emplace_back();
    return N++;
  }

  void either(int f, int j) {
    f = max(2*f, -1-2*f);
    j = max(2*j, -1-2*j);
    gr[f].push_back(j^1);
    gr[j].push_back(f^1);
  }
  void setValue(int x) { either(x, x); }

  void atMostOne(const vector<int>& li) { // (optional)
    if ((int)li.size() <= 1) return;
    int cur = ~li[0];
```

```
    for (int i = 2; i < li.size(); i++) {
      int next = addVar();
      either(cur, ~li[i]);
      either(cur, next);
      either(~li[i], next);
      cur = ~next;
    }
    either(cur, ~li[1]);
  }

  vector<int> val, comp, z; int time = 0;
  int dfs(int i) {
    int low = val[i] = ++time, x; z.push_back(i);
    for(int e : gr[i]) if (!comp[e])
      low = min(low, val[e] ?: dfs(e));
    if (low == val[i]) do {
      x = z.back(); z.pop_back();
      comp[x] = low;
      if (values[x>>1] == -1)
        values[x>>1] = x&1;
    } while (x != i);
    return val[i] = low;
  }

  bool solve() {
    values.assign(N, -1);
    val.assign(2*N, 0); comp = val;
    for (int i = 0; i < 2*N; i++) if (!comp[i]) dfs(i);
    for (int i = 0; i < N; i++)
      if (comp[2 * i] == comp[2 * i + 1]) return 0;
    return 1;
  }
};
```

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.
Usage: `vector<int> path = eulerWalk(gr, nedges, src); // gr[v] = {(u, eid)}`
Time: $\mathcal{O}(V+E)$

17 lines

```
vector<int> eulerWalk(vector<vector<pair<int, int>>>& gr,
  int nedges, int src = 0) {
  int n = gr.size();
  vector<int> D(n), its(n), eu(nedges), ret, s = {src};
  D[src]++; // to allow Euler paths, not just cycles
  while (!s.empty()) {
    int x = s.back(), y, e, &it = its[x], end = gr[x].size();
    if (it == end){ ret.push_back(x); s.pop_back(); continue; }
    tie(y, e) = gr[x][it++];
    if (!eu[e]) {
      D[x]--, D[y]++;
      eu[e] = 1; s.push_back(y);
    }}
  for (int x : D)
    if (x < 0 || (int)ret.size() != nedges + 1) return {};
  return {ret.rbegin(), ret.rend()};
}
```

DominatorTree.h

Description: Dominator tree of a directed graph from root r: u dominates v if every path $r \to v$ passes through u. Returns idom[v] = immediate dominator (parent in the tree), idom[r] = r, -1 for vertices unreachable from r. Typical use: "vertices whose removal disconnects v from r" = ancestors of v in the dominator tree.
Usage: `vector<int> idom = dominators(adj, root);`
Time: $\mathcal{O}(E \log V)$

38 lines

```
vector<int> dominators(vector<vector<int>>& g, int root) {
  int n = g.size(), T = 0;
  vector<int> id(n, -1), rv(n), par(n), sd(n), dm(n), uf(n),
    lb(n);
  vector<vector<int>> rg(n), bk(n);
  auto dfs = [&](auto&& self, int v) -> void {
    rv[T] = v, id[v] = sd[T] = uf[T] = lb[T] = T, T++;
    for (int u : g[v]) {
      if (id[u] < 0) self(self, u), par[id[u]] = id[v];
      rg[id[u]].push_back(id[v]);
    }
  };
  dfs(dfs, root);
  auto find = [&](auto&& self, int u, int x) -> int {
    if (u == uf[u]) return x ? -1 : u;
    int v = self(self, uf[u], x + 1);
    if (v < 0) return u;
    if (sd[lb[uf[u]]] < sd[lb[u]]) lb[u] = lb[uf[u]];
    uf[u] = v;
    return x ? v : lb[u];
  };
  for (int i = T - 1; i >= 0; i--) {
    for (int j : rg[i]) sd[i] = min(sd[i], sd[find(find,j,0)]);
    if (i) bk[sd[i]].push_back(i);
    for (int j : bk[i]) {
      int v = find(find, j, 0);
      dm[j] = sd[v] == sd[j] ? sd[j] : v;
    }
    if (i) uf[i] = par[i];
  }
  vector<int> idom(n, -1);
  idom[root] = root;
  for (int i = 1; i < T; i++) {
    if (dm[i] != sd[i]) dm[i] = dm[dm[i]];
    idom[rv[i]] = rv[dm[i]];
  }
  return idom;
}
```

DynamicConnectivity.h

Description: Offline dynamic connectivity. Time is $0..Q-1$; each edge is alive during a time interval $[l, r)$ (from its insertion to its deletion). Computes for every time the number of components; put other queries (e.g. uf.find(a) == uf.find(b)) at the leaf. Build intervals with map<pair<int, int>,int> start time; end = Q if never removed.
Usage: `DynCon d(n, Q); d.addEdge(a, b, l, r); d.solve(); d.ans[t]`
Time: $\mathcal{O}(Q \log Q \log n)$

"../data-structures/UnionFindRollback.h" 22 lines

```
struct DynCon {
  int Q, comps;
  vector<vector<pair<int, int>>> seg;
  RollbackUF uf;
  vector<int> ans;
  DynCon(int n, int Q) : Q(Q), comps(n), seg(2 * Q), uf(n),
    ans(Q) {}
  void addEdge(int a, int b, int l, int r) {
    for (auto [a, b] : seg[v]) comps -= uf.join(a, b);
    if (v >= Q) ans[v - Q] = comps; // answer queries at v=Q
    else dfs(2 * v), dfs(2 * v + 1);
    uf.rollback(t); comps = c;
  }
  void solve() { if (Q) dfs(1); }
};
```

7.5 Coloring

EdgeColoring.h

Description: Given a simple, undirected graph with max degree D, computes a $(D+1)$-coloring of the edges such that no neighboring edges share a color. (D-coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)
Usage: `vector<int> col = edgeColoring(n, eds); // col[i] in [0, maxdeg]`
Time: $\mathcal{O}(NM)$

31 lines

```
vector<int> edgeColoring(int N, vector<pair<int, int>> eds) {
  vector<int> cc(N + 1), ret(eds.size()), fan(N), free(N), loc;
  for (pair<int, int> e : eds) ++cc[e.first], ++cc[e.second];
  int u, v, ncols = *max_element(cc.begin(), cc.end()) + 1;
  vector<vector<int>> adj(N, vector<int>(ncols, -1));
  for (pair<int, int> e : eds) {
    tie(u, v) = e;
    fan[0] = v;
    loc.assign(ncols, 0);
    int at = u, end = u, d, c = free[u], ind = 0, i = 0;
    while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
      loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
    cc[loc[d]] = c;
    for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd])
      swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
    while (adj[fan[i]][d] != -1) {
      int left = fan[i], right = fan[++i], e = cc[i];
      adj[u][e] = left;
      adj[left][e] = u;
      adj[right][e] = -1;
      free[right] = e;
    }
    adj[u][d] = fan[i];
    adj[fan[i]][d] = u;
    for (int y : {fan[0], u, end})
      for (int& z = free[y] = 0; adj[y][z] != -1; z++);
  }
  for (int i = 0; i < eds.size(); i++)
    for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];
  return ret;
}
```

7.6 Trees

BinaryLifting.h

Description: Calculate power of two jumps in a tree, to support fast upward jumps and LCAs. Assumes the root node points to itself.
Usage: `auto tbl = treeJump(par); // par[root] = root`
`int u = jmp(tbl, v, k), c = lca(tbl, depth, a, b);`
Time: construction $\mathcal{O}(N \log N)$, queries $\mathcal{O}(\log N)$

27 lines

```
vector<vector<int>> treeJump(vector<int>& P){
  int on = 1, d = 1;
  while(on < (int)P.size()) on *= 2, d++;
  vector<vector<int>> jmp(d, P);
  for (int i = 1; i < d; i++)
    for (int j = 0; j < P.size(); j++)
      jmp[i][j] = jmp[i - 1][jmp[i - 1][j]];
  return jmp;
}

int jmp(vector<vector<int>>& tbl, int nod, int steps){
  for (int i = 0; i < tbl.size(); i++)
    if(steps&(1<<i)) nod = tbl[i][nod];
  return nod;
}

int lca(
  vector<vector<int>>& tbl, vector<int>& depth, int a, int b) {
  if (depth[a] < depth[b]) swap(a, b);
  a = jmp(tbl, a, depth[a] - depth[b]);
  if (a == b) return a;
  for (int i = tbl.size(); i--;) {
    int c = tbl[i][a], d = tbl[i][b];
    if (c != d) a = c, b = d;
  }
  return tbl[0][a];
}
```

LCA.h

Description: Data structure for computing lowest common ancestors in a tree (with 0 as root). C should be an adjacency list of the tree, either directed or undirected.
Usage: `LCA L(adj); int c = L.lca(a, b);`
Time: $\mathcal{O}(N \log N + Q)$

"../data-structures/RMQ.h" 22 lines

```
struct LCA {
  int T = 0;
  vector<int> time, path, ret;
  RMQ rmq;

  LCA(vector<vector<int>>& C) :
      time(C.size()), rmq((dfs(C, 0, -1), ret)) {}
  void dfs(vector<vector<int>>& C, int v, int par) {
    time[v] = T++;
    for (int y : C[v]) if (y != par) {
      path.push_back(v), ret.push_back(time[v]);
      dfs(C, y, v);
    }
  }

  int lca(int a, int b) {
    if (a == b) return a;
    tie(a, b) = minmax(time[a], time[b]);
    return path[rmq.query(a, b)];
  }
  //dist(a,b){return depth[a] + depth[b] - 2*depth[lca(a,b)];}
};
```

CompressTree.h

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S| - 1$) pairwise LCA's and compressing edges. Returns a list of (par, orig_index) representing a tree rooted at 0. The root points to itself.
Usage: `LCA L(adj); auto t = compressTree(L, subset);`
`// t[i] = {parent index in t, original vertex}`
Time: $\mathcal{O}(|S| \log |S|)$

"LCA.h" 21 lines

```
typedef vector<pair<int, int>> vpi;
vpi compressTree(LCA& lca, const vector<int>& subset) {
  static vector<int> rev; rev.resize(lca.time.size());
  vector<int> li = subset, &T = lca.time;
  auto cmp = [&](int a, int b) { return T[a] < T[b]; };
  sort(li.begin(), li.end(), cmp);
  int m = (int)li.size()-1;
  for (int i = 0; i < m; i++) {
    int a = li[i], b = li[i+1];
    li.push_back(lca.lca(a, b));
  }
  sort(li.begin(), li.end(), cmp);
  li.erase(unique(li.begin(), li.end()), li.end());
  for (int i = 0; i < li.size(); i++) rev[li[i]] = i;
  vpi ret = {pair<int, int>(0, li[0])};
  for (int i = 0; i < (int)li.size()-1; i++) {
    int a = li[i], b = li[i+1];
    ret.emplace_back(rev[lca.lca(a, b)], b);
  }
  return ret;
}
```

HLD.h

Description: Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most log(n) light edges. Code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. Takes as input the full adjacency list. valsEdges = true means that values are stored in the edges, as opposed to the nodes (edge (v, par[v]) lives at pos[v]). All values initialized to the segtree default. Root must be 0.
Usage: `HLD h(adj); h.modifyPath(u, v, x); h.queryPath(u, v);`
Time: $\mathcal{O}\left((\log N)^2\right)$

"../data-structures/LazySegmentTree.h" 47 lines

```
struct HLD {
  int N, tim = 0; bool VALS_EDGES;
  vector<vector<int>> adj;
  vector<int> par, siz, rt, pos;
  Node *tree;
  HLD(vector<vector<int>> adj_, bool valsEdges = false)
    : N(adj_.size()), VALS_EDGES(valsEdges), adj(adj_),
      par(N, -1), siz(N, 1),
      rt(N),pos(N),tree(new Node(0, N)){ dfsSz(0); dfsHld(0); }
  void dfsSz(int v) {
    for (int& u : adj[v]) {
      adj[u].erase(find(adj[u].begin(), adj[u].end(), v));
      par[u] = v;
      dfsSz(u);
      siz[v] += siz[u];
      if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
    }
  }
  void dfsHld(int v) {
    pos[v] = tim++;
    for (int u : adj[v]) {
      rt[u] = (u == adj[v][0] ? rt[v] : u);
      dfsHld(u);
    }
  }
  void process(int u, int v, function<void(int,int)> op) {
    for (;; v = par[rt[v]]) {
      if (pos[u] > pos[v]) swap(u, v);
      if (rt[u] == rt[v]) break;
      op(pos[rt[v]], pos[v] + 1);
    }
    op(pos[u] + VALS_EDGES, pos[v] + 1);
  }
  void modifyPath(int u, int v, int val) {
    process(u, v, [&](int l, int r) { tree->add(l, r, val); });
  }
  int queryPath(int u, int v) { // Modify depending on problem
    int res = -1e9;
    process(u, v, [&](int l, int r) {
        res = max(res, tree->query(l, r));
    });
    return res;
  }
  int querySubtree(int v) { // modifySubtree is similar
    return tree->query(pos[v] + VALS_EDGES, pos[v] + siz[v]);
  }
};
```

CentroidDecomposition.h

Description: Centroid decomposition of a tree. Repeatedly removes a centroid (all remaining parts have $\leq$ half the size). par[c] = parent of c in the centroid tree (-1 for the root); it has depth $O(\log n)$ and the path u–v passes through their centroid-tree LCA. Typical use: at the marked line, DFS from c over vertices with !rem[] to handle all paths through c (e.g. count paths of length K).
Usage: `Centroid cd(adj); // cd.par[v]`
Time: $\mathcal{O}(n \log n)$ plus the work at each centroid times $\mathcal{O}(\log n)$

27 lines

```
struct Centroid {
  vector<vector<int>> adj; vector<int> par, siz, rem;
  Centroid(vector<vector<int>> g) :
      adj(g), par((int)g.size(), -1), siz(g.size()),
      rem(g.size()) {
    if (g.size()) build(0, -1);
  }
  int dfsSz(int v, int p) {
    siz[v] = 1;
    for (int u : adj[v]) if (u != p && !rem[u])
      siz[v] += dfsSz(u, v);
    return siz[v];
  }
  void build(int v, int p) {
    int n = dfsSz(v, -1), c = v, prev = -1;
    for (bool moved = 1; moved;) {
      moved = 0;
      for (int u : adj[c])
```

```
      if (u != prev && !rem[u] && 2 * siz[u] > n) {
        prev = c, c = u, moved = 1; break;
      }
    }
    par[c] = p; rem[c] = 1;
    // process all paths through c here
    for (int u : adj[c]) if (!rem[u]) build(u, c);
  }
};
```

LinkCutTree.h

Description: Represents a forest of unrooted trees. You can add and remove edges (as long as the result is still a forest), and check whether two nodes are in the same tree.
Usage: `LinkCut lc(n); lc.link(u, v); lc.cut(u, v);`
`bool c = lc.connected(u, v);`
Time: All operations take amortized $\mathcal{O}(\log N)$.

90 lines

```
struct Node { // Splay tree. Root's pp contains tree's parent.
  Node *p = 0, *pp = 0, *c[2];
  bool flip = 0;
  Node() { c[0] = c[1] = 0; fix(); }
  void fix() {
    if (c[0]) c[0]->p = this;
    if (c[1]) c[1]->p = this;
    // (+ update sum of subtree elements etc. if wanted)
  }
  void pushFlip() {
    if (!flip) return;
    flip = 0; swap(c[0], c[1]);
    if (c[0]) c[0]->flip ^= 1;
    if (c[1]) c[1]->flip ^= 1;
  }
  int up() { return p ? p->c[1] == this : -1; }
  void rot(int i, int b) {
    int h = i ^ b;
    Node *x = c[i], *y = b == 2 ? x : x->c[h], *z = b ? y : x;
    if ((y->p = p)) p->c[up()] = y;
    c[i] = z->c[i ^ 1];
    if (b < 2) {
      x->c[h] = y->c[h ^ 1];
      y->c[h ^ 1] = x;
    }
    z->c[i ^ 1] = this;
    fix(); x->fix(); y->fix();
    if (p) p->fix();
    swap(pp, y->pp);
  }
  void splay() {
    for (pushFlip(); p; ) {
      if (p->p) p->p->pushFlip();
      p->pushFlip(); pushFlip();
      int c1 = up(), c2 = p->up();
      if (c2 == -1) p->rot(c1, 2);
      else p->p->rot(c2, c1 != c2);
    }
  }
  Node* first() {
    pushFlip();
    return c[0] ? c[0]->first() : (splay(), this);
  }
};
```

```
struct LinkCut {
  vector<Node> node;
  LinkCut(int N) : node(N) {}

  void link(int u, int v) { // add an edge (u, v)
    assert(!connected(u, v));
    makeRoot(&node[u]);
    node[u].pp = &node[v];
  }
  void cut(int u, int v) { // remove an edge (u, v)
    Node *x = &node[u], *top = &node[v];
    makeRoot(top); x->splay();
    assert(top == (x->pp ?: x->c[0]));
    if (x->pp) x->pp = 0;
    else {
      x->c[0] = top->p = 0;
      x->fix();
    }
  }
  bool connected(int u, int v) { // are u, v in the same tree?
    Node* nu = access(&node[u])->first();
    return nu == access(&node[v])->first();
  }
  void makeRoot(Node* u) {
    access(u);
    u->splay();
    if(u->c[0]) {
      u->c[0]->p = 0;
      u->c[0]->flip ^= 1;
      u->c[0]->pp = u;
      u->c[0] = 0;
      u->fix();
    }
  }
  Node* access(Node* u) {
    u->splay();
    while (Node* pp = u->pp) {
      pp->splay(); u->pp = 0;
      if (pp->c[1]) {
        pp->c[1]->p = 0; pp->c[1]->pp = pp; }
      pp->c[1] = u; pp->fix(); u = pp;
    }
    return u;
  }
};
```

DirectedMST.h

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.
Usage: `vector<Edge> g = {{a, b, w}, ...};`
`auto [cost, par] = dmst(n, root, g); // cost -1: none`
Time: $\mathcal{O}(E \log V)$

"../data-structures/UnionFindRollback.h" 60 lines

```
struct Edge { int a, b; ll w; };
struct Node {
  Edge key;
  Node *l, *r;
  ll delta;
  void prop() {
    key.w += delta;
    if (l) l->delta += delta;
    if (r) r->delta += delta;
    delta = 0;
  }
  Edge top() { prop(); return key; }
};
Node *merge(Node *a, Node *b) {
  if (!a || !b) return a ?: b;
  a->prop(), b->prop();
  if (a->key.w > b->key.w) swap(a, b);
  swap(a->l, (a->r = merge(b, a->r)));
  return a;
}
void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

pair<ll, vector<int>> dmst(int n, int r, vector<Edge>& g) {
  RollbackUF uf(n);
  vector<Node*> heap(n);
  for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
  ll res = 0;
  vector<int> seen(n, -1), path(n), par(n);
  seen[r] = r;
  vector<Edge> Q(n), in(n, {-1,-1}), comp;
  deque<tuple<int, int, vector<Edge>>> cycs;
  for (int s = 0; s < n; s++) {
    int u = s, qi = 0, w;
    while (seen[u] < 0) {
      if (!heap[u]) return {-1,{}};
      Edge e = heap[u]->top();
      heap[u]->delta -= e.w, pop(heap[u]);
      Q[qi] = e, path[qi++] = u, seen[u] = s;
      res += e.w, u = uf.find(e.a);
      if (seen[u] == s) {
        Node* cyc = 0;
        int end = qi, time = uf.time();
        do cyc = merge(cyc, heap[w = path[--qi]]);
        while (uf.join(u, w));
        u = uf.find(u), heap[u] = cyc, seen[u] = -1;
        cycs.push_front({u, time, {&Q[qi], &Q[end]}});
      }
    }
    for (int i = 0; i < qi; i++) in[uf.find(Q[i].b)] = Q[i];
  }

  for (auto& [u,t,comp] : cycs) { // restore sol (optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for (auto& e : comp) in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
  }
  for (int i = 0; i < n; i++) par[i] = in[i].a;
  return {res, par};
}
```

7.7 Math

7.7.1 Number of Spanning Trees

Create an $N \times N$ matrix `mat`, and for each edge $a \to b \in G$, do `mat[a][b]--, mat[b][b]++` (and `mat[b][a]--, mat[a][a]++` if G is undirected). Remove the ith row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column). Determinant mod p: IntDeterminant.

7.7.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \cdots \geq d_n$ exists iff $d_1 + \cdots + d_n$ is even and for every $k = 1 \ldots n$,

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k).$$

7.7.3 Matchings, covers, flows

Kőnig (bipartite): max matching = min vertex cover; max independent set = V − max matching; min edge cover = V − max matching (no isolated vertices). Min vertex cover from a max flow: left vertices *not* reachable from s in the residual graph + right vertices reachable (MinimumVertexCover).

Hall: a left-perfect matching exists iff $|N(S)| \geq |S|$ for all left S.

DAG path cover: min vertex-disjoint path cover = V − max matching on (out-copy, in-copy); with paths allowed to share vertices: first take the transitive closure. **Dilworth:** min chains = max antichain.

Max-flow = min-cut. Lower bounds $l \leq f \leq c$ on $u \to v$: give it capacity $c - l$ and add $S' \to v$ and $u \to T'$ with capacity l (new source/sink), plus $t \to s$ with ∞; feasible iff the max flow $S' \to T'$ saturates all S' edges. Then max flow $s \to t$ in what is left.

Project selection / closure: profit $p_i > 0$: $s \to i$ (p_i); cost: $i \to t$ $(-p_i)$; "i requires j": $i \to j$ (∞). Best = $\sum_{p_i>0} p_i -$ mincut.

Geometry (8)

8.1 Geometric primitives

Point.h

Description: Class to handle points in the plane. All geometry code uses this `P`. Set `T = ll` for exact integer algorithms (hulls, inside tests); `double` is exact for integer coordinates up to $\sim 3 \cdot 10^7$ (products below 2^{53}). Functions returning new points (intersections, rotations) need double.

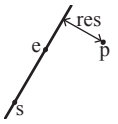
Usage: `P a(1, 2), b(3, 4); T cr = a.cross(b); double d = (a-b).dist();`

27 lines

```
typedef double T; // or ll / long double
int sgn(T x) { return (x > 0) - (x < 0); }
struct P {
  T x, y;
  explicit P(T x=0, T y=0) : x(x), y(y) {}
  bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
  bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
  P operator+(P p) const { return P(x+p.x, y+p.y); }
  P operator-(P p) const { return P(x-p.x, y-p.y); }
  P operator*(T d) const { return P(x*d, y*d); }
  P operator/(T d) const { return P(x/d, y/d); }
  T dot(P p) const { return x*p.x + y*p.y; }
  T cross(P p) const { return x*p.y - y*p.x; }
  T cross(P a, P b) const { return (a-*this).cross(b-*this); }
  T dist2() const { return x*x + y*y; }
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  P unit() const { return *this/dist(); } // makes dist()=1
  P perp() const { return P(-y, x); } // rotates +90 degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw around the origin
  P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
  friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << "," << p.y << ")"; }
};
```

lineDistance.h

Description: Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. It uses products in intermediate steps so watch out for overflow if using int or long long. For 3D: (b-a).cross(p-a).dist()/(b-a).dist().



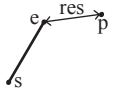
Usage: `double d = lineDist(a, b, p); // > 0 if p left of a->b`

"Point.h" 3 lines

```
double lineDist(P a, P b, P p) {
  return (double)(b-a).cross(p-a)/(b-a).dist();
}
```

SegmentDistance.h

Description: Returns the shortest distance between point p and the line segment from point s to e.



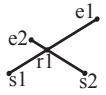
Usage: `P a, b(2,2), p(1,1);`
`bool onSegment = segDist(a,b,p) < 1e-10;`

"Point.h" 7 lines

```
double segDist(P s, P e, P p) { // works for T = ll too
  if (s==e) return (p-s).dist();
  P a = p-s, b = e-s;
  double d = (double)b.dist2();
  double t = min(d, max(.0, (double)a.dot(b)));
  return hypot(a.x*d-b.x*t, a.y*d-b.y*t)/d;
}
```

SegmentIntersection.h

Description: If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if T is ll and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.



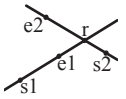
Usage: `vector<P> inter = segInter(s1,e1,s2,e2);`
`if ((int)inter.size()==1)`
`cout << "segments intersect at " << inter[0] << endl;`

"Point.h", "OnSegment.h" 13 lines

```
vector<P> segInter(P a, P b, P c, P d) {
  auto oa = c.cross(d, a), ob = c.cross(d, b),
       oc = a.cross(b, c), od = a.cross(b, d);
  // Checks if intersection is single non-endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
    return {(a * ob - b * oa) / (ob - oa)};
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {s.begin(), s.end()};
}
```

lineIntersection.h

Description: If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if T is ll and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.



Usage: `auto res = lineInter(s1,e1,s2,e2);`
`if (res.first == 1)`
`cout << "intersection point at " << res.second << endl;`

"Point.h" 7 lines

```
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
  auto d = (e1 - s1).cross(e2 - s2);
  if (d == 0) // if parallel
    return {-(s1.cross(e1, s2) == 0), P(0, 0)};
  auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
  return {1, (s1 * p + e1 * q) / d};
}
```

sideOf.h

Description: Returns where p is as seen from s towards e. 1/0/-1 $\Leftrightarrow$ left/on line/right. If the optional argument *eps* is given 0 is returned if p is within distance *eps* from the line. It uses products in intermediate steps so watch out for overflow if using int or long long.

Usage: `bool left = sideOf(p1,p2,q)==1;`

"Point.h" 7 lines

```
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }

int sideOf(P s, P e, P p, double eps) {
  auto a = (e-s).cross(p-s);
  double l = (e-s).dist()*eps;
  return (a > l) - (a < -l);
}
```

OnSegment.h

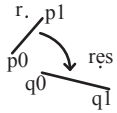
Description: Returns true iff p lies on the line segment from s to e. Use `(segDist(s,e,p)<=epsilon)` instead when using T = double.

Usage: `bool on = onSegment(s, e, p); // exact with T = ll`

"Point.h" 3 lines

```
bool onSegment(P s, P e, P p) {
  return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

linearTransformation.h



Description:

Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.

Usage: `P r2 = linearTransformation(p0, p1, q0, q1, r);`

"Point.h" 4 lines

```
P linearTransformation(P p0, P p1, P q0, P q1, P r) {
  P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
  return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
}
```

LineProjectionReflection.h

Description: Projects point p onto line ab. Set refl=true to get reflection of point p across line ab instead. The wrong point will be returned if T is integer and the desired point doesn't have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow.

Usage: `P q = lineProj(a, b, p), r = lineProj(a, b, p, true);`

"Point.h" 4 lines

```
P lineProj(P a, P b, P p, bool refl=false) {
  P v = b - a;
  return p - v.perp()*(1+refl)*v.cross(p-a)/v.dist2();
}
```

Angle.h

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.

Usage: `vector<Angle> v = {w[0], w[0].t360() ...}; // sorted`
`int j = 0; for (int i = 0; i < n; i++) { while (v[j] < v[i].t180()) ++j; }`
`// sweeps j such that (j-i) represents the number of positively oriented triangles with vertices at 0 and i`

35 lines

```
struct Angle {
  int x, y;
  int t;
  Angle(int x, int y, int t=0) : x(x), y(y), t(t) {}
  Angle operator-(Angle b) const { return {x-b.x, y-b.y, t}; }
  int half() const {
    assert(x || y);
    return y < 0 || (y == 0 && x < 0);
  }
  Angle t90() const { return {-y, x, t + (half() && x >= 0)}; }
  Angle t180() const { return {-x, -y, t + half()}; }
  Angle t360() const { return {x, y, t + 1}; }
};
bool operator<(Angle a, Angle b) {
  // add a.dist2() and b.dist2() to also compare distances
  return make_tuple(a.t, a.half(), a.y * (ll)b.x) <
         make_tuple(b.t, b.half(), a.x * (ll)b.y);
}

// Given two points, this calculates the smallest angle between
// them, i.e., the angle that covers the defined line segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
  if (b < a) swap(a, b);
  return (b < a.t180() ?
          make_pair(a, b) : make_pair(b, a.t360()));
}
Angle operator+(Angle a, Angle b) { // point a + vector b
  Angle r(a.x + b.x, a.y + b.y, a.t);
  if (a.t180() < r) r.t--;
  return r.t180() < a ? r.t360() : r;
}
Angle angleDiff(Angle a, Angle b) { // angle b - angle a
  int tu = b.t - a.t; a.t = b.t;
  return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b < a)};
}
```

8.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

Usage: `pair<P, P> o; if (circleInter(a, b, r1, r2, &o)) ...`

"Point.h" 10 lines

```
bool circleInter(P a,P b,double r1,double r2,pair<P, P>* out) {
  if (a == b) { assert(r1 != r2); return false; }
  P vec = b - a;
  double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
         p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2;
  if (sum*sum < d2 || dif*dif > d2) return false;
  P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2);
  *out = {mid + per, mid - per};
  return true;
}
```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

Usage: `auto t = tangents(c1, r1, c2, r2); // t[i] = {on c1, on c2}`

"Point.h" 12 lines

```
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {
  P d = c2 - c1;
  double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
  if (d2 == 0 || h2 < 0)  return {};
  vector<pair<P, P>> out;
  for (double sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
    out.push_back({c1 + v * r1, c2 + v * r2});
  }
  if (h2 == 0) out.pop_back();
  return out;
}
```

CircleLine.h

Description: Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points. Needs T = double.

Usage: `auto v = circleLine(c, r, a, b); // points on line ab`

"Point.h" 8 lines

```
vector<P> circleLine(P c, double r, P a, P b) {
  P ab = b - a, p = a + ab * (c-a).dot(ab) / ab.dist2();
  double s = a.cross(b, c), h2 = r*r - s*s / ab.dist2();
  if (h2 < 0) return {};
  if (h2 == 0) return {p};
  P h = ab.unit() * sqrt(h2);
  return {p - h, p + h};
}
```

CirclePolygonIntersection.h

Description: Returns the area of the intersection of a circle with a ccw polygon.

Usage: `double A = circlePoly(c, r, poly); // poly ccw`

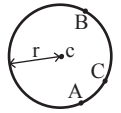
Time: $\mathcal{O}(n)$

"Point.h" 18 lines

```
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
  auto tri = [&](P p, P q) {
    auto r2 = r * r / 2;
    P d = q - p;
    auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2();
    auto det = a * a - b;
    if (det <= 0) return arg(p, q) * r2;
    auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det));
    if (t < 0 || 1 <= s) return arg(p, q) * r2;
    P u = p + d * s, v = q + d * (t-1);
    return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2;
  };
  auto sum = 0.0;
  for (int i = 0; i < ps.size(); i++)
    sum += tri(ps[i] - c, ps[(i + 1) % (int)ps.size()] - c);
  return sum;
}
```

circumcircle.h

Description:



The circumcirle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.

Usage: `P o = ccCenter(a, b, c); double R = ccRadius(a, b, c);`

"Point.h" 8 lines

```
double ccRadius(P A, P B, P C) {
  return (B-A).dist()*(C-B).dist()*(A-C).dist()/
      abs((B-A).cross(C-A))/2;
}
P ccCenter(P A, P B, P C) {
  P b = C-A, c = B-A;
  return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}
```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.

Usage: `auto [o, r] = mec(pts);`

Time: expected $\mathcal{O}(n)$

"circumcircle.h" 20 lines

```
pair<P, double> mec(vector<P> ps) {
  shuffle(ps.begin(), ps.end(), mt19937(time(0)));
  P o = ps[0];
  double r = 0, EPS = 1 + 1e-8;
  for (int i = 0; i < ps.size(); i++)
    if ((o - ps[i]).dist() > r * EPS) {
      o = ps[i], r = 0;
      for (int j = 0; j < i; j++)
        if ((o - ps[j]).dist() > r * EPS) {
          o = (ps[i] + ps[j]) / 2;
          r = (o - ps[i]).dist();
          for (int k = 0; k < j; k++)
            if ((o - ps[k]).dist() > r * EPS) {
              o = ccCenter(ps[i], ps[j], ps[k]);
```

```
            r = (o - ps[i]).dist();
          }
        }
      }
    }
  return {o, r};
}
```

CircleUnion.h

Description: Area of the union of circles. For each circle, the arcs not covered by other circles form the boundary of the union; their area is summed with Green's theorem ($\frac{1}{2}\oint x\,dy - y\,dx$ over each arc). Duplicate circles are handled. Needs T = double.
Usage: `double A = circleUnion(centers, radii);`
Time: $\mathcal{O}\left(n^2 \log n\right)$

"Point.h" 33 lines

```
double circleUnion(vector<P>& c, vector<double>& r) {
  int n = c.size(); double res = 0, eps = 1e-9;
  auto arc = [&](int i, double a, double b) {
    return (r[i] * r[i] * (b - a) + r[i] * (c[i].x * (sin(b) -
      sin(a)) - c[i].y * (cos(b) - cos(a)))) / 2; };
  for (int i = 0; i < n; i++) {
    vector<pair<double, double>> cov; bool inside = 0;
    for (int j = 0; j < n; j++) if (j != i) {
      P d = c[j] - c[i]; double D = d.dist();
      if (D + r[i] < r[j] + eps && (r[i] < r[j] - eps || j<i))
        { inside = 1; break; } // circle i is covered by j
      if (D + r[j] < r[i] + eps || D > r[i] + r[j] - eps)
        continue; // j inside i, or disjoint
      double a = d.angle(), w = acos(max(-1.0, min(1.0,
        (r[i]*r[i] + D*D - r[j]*r[j]) / (2*r[i]*D))));
      double lo = a - w, hi = a + w;
      if (lo < -M_PI) cov.push_back({lo + 2*M_PI, M_PI}),
        lo = -M_PI;
      if (hi > M_PI) cov.push_back({-M_PI, hi - 2*M_PI}),
        hi = M_PI;
      cov.push_back({lo, hi});
    }
    if (inside) continue;
    sort(cov.begin(), cov.end());
    double cur = -M_PI;
    for (auto [lo, hi] : cov) {
      if (lo > cur) res += arc(i, cur, lo);
      cur = max(cur, hi);
    }
    if (cur < M_PI) res += arc(i, cur, M_PI);
  }
  return res;
}
```

8.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.
Usage: `vector<P> v = {P{4,4}, P{1,2}, P{2,1}};`
`bool in = inPolygon(v, P{3, 3}, false);`
Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h" 10 lines

```
bool inPolygon(vector<P> &p, P a, bool strict = true) {
  int cnt = 0, n = p.size();
  for (int i = 0; i < n; i++) {
    P q = p[(i + 1) % n];
    if (onSegment(p[i], q, a)) return !strict;
    //or: if (segDist(p[i], q, a) <= eps) return !strict;
    cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
  }
  return cnt;
}
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Exact with T = ll.
Usage: `T a2 = polygonArea2(poly); // twice the area, ccw > 0`

"Point.h" 6 lines

```
T polygonArea2(vector<P>& v) {
  T a = v.back().cross(v[0]);
  for (int i = 0; i < (int)v.size() - 1; i++)
    a += v[i].cross(v[i + 1]);
  return a;
}
```

PolygonCenter.h

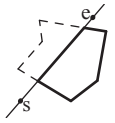
Description: Returns the center of mass for a polygon.
Usage: `P g = polygonCenter(poly); // centroid of the area`
Time: $\mathcal{O}(n)$

"Point.h" 9 lines

```
P polygonCenter(const vector<P>& v) {
  P res(0, 0); double A = 0;
  for (int i = 0, j = (int)v.size() - 1; i < (int)v.size();
    j = i++) {
    res = res + (v[i] + v[j]) * v[j].cross(v[i]);
    A += v[j].cross(v[i]);
  }
  return res / A / 3;
}
```

PolygonCut.h

Description:
Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.
Usage: `vector<P> p = ...;`
`p = polygonCut(p, P(0,0), P(1,0));`



"Point.h" 12 lines

```
vector<P> polygonCut(const vector<P>& poly, P s, P e) {
  vector<P> res;
  for (int i = 0; i < poly.size(); i++) {
    P cur = poly[i], prev = i ? poly[i-1] : poly.back();
    auto a = s.cross(e, cur), b = s.cross(e, prev);
    if ((a < 0) != (b < 0))
      res.push_back(cur + (prev - cur) * (a / (a - b)));
    if (a < 0)
      res.push_back(cur);
  }
  return res;
}
```

PolygonUnion.h

Description: Calculates the area of the union of n polygons (not necessarily convex). The points within each polygon must be given in CCW order. (Epsilon checks may optionally be added to sideOf/sgn, but shouldn't be needed.)
Usage: `double A = polyUnion(polys); // each polygon ccw`
Time: $\mathcal{O}\left(N^2\right)$, where N is the total number of points

"Point.h", "sideOf.h" 39 lines

```
double rat(P a, P b) { return sgn(b.x) ? a.x/b.x : a.y/b.y; }
double polyUnion(vector<vector<P>>& poly) {
  double ret = 0;
  for (int i = 0; i < poly.size(); i++)
    for (int v = 0; v < poly[i].size(); v++) {
      P A = poly[i][v],
        B = poly[i][(v + 1) % (int)poly[i].size()];
      vector<pair<double, int>> segs = {{0, 0}, {1, 0}};
      for (int j = 0; j < poly.size(); j++)
        if (i != j) {
          for (int u = 0; u < poly[j].size(); u++) {
            P C = poly[j][u],
              D = poly[j][(u + 1) % (int)poly[j].size()];
            int sc = sideOf(A, B, C), sd = sideOf(A, B, D);
            if (sc != sd) {
              double sa = C.cross(D, A), sb = C.cross(D, B);
              if (min(sc, sd) < 0)
                segs.emplace_back(
                  sa / (sa - sb), sgn(sc - sd));
            } else if (!sc && !sd && j < i &&
              sgn((B - A).dot(D - C)) > 0) {
              segs.emplace_back(rat(C - A, B - A), 1);
              segs.emplace_back(rat(D - A, B - A), -1);
            }
          }
        }
      sort(segs.begin(), segs.end());
      for (auto& s : segs)
        s.first = min(max(s.first, 0.0), 1.0);
      double sum = 0;
      int cnt = segs[0].second;
      for (int j = 1; j < segs.size(); j++) {
        if (!cnt) sum += segs[j].first - segs[j - 1].first;
        cnt += segs[j].second;
      }
      ret += A.cross(B) * sum;
    }
  return ret / 2;
}
```

ConvexHull.h

Description:
Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.



Usage: `vector<P> h = convexHull(pts); // ccw, starts at lowest x`
Time: $\mathcal{O}\left(n \log n\right)$

"Point.h" 13 lines

```
vector<P> convexHull(vector<P> pts) {
  if ((int)pts.size() <= 1) return pts;
  sort(pts.begin(), pts.end());
  vector<P> h((int)pts.size()+1);
  int s = 0, t = 0;
  for (int it = 2; it--;
    s = --t, reverse(pts.begin(), pts.end()))
    for (P p : pts) {
      while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).
Usage: `auto [a, b] = hullDiameter(convexHull(pts));`
Time: $\mathcal{O}(n)$

"Point.h" 11 lines

```
array<P, 2> hullDiameter(vector<P> S) {
  int n = S.size(), j = n < 2 ? 0 : 1;
  pair<T, array<P, 2>> res({0, {S[0], S[0]}});
  for (int i = 0; i < j; i++)
    for (;; j = (j + 1) % n) {
      res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}});
      if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
        break;
    }
  return res.second;
}
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.
Time: $\mathcal{O}(\log N)$

"Point.h", "sideOf.h", "OnSegment.h" 13 lines

```
bool inHull(const vector<P>& l, P p, bool strict = true) {
  int a = 1, b = (int)l.size() - 1, r = !strict;
  if ((int)l.size() < 3)
    return r && onSegment(l[0], l.back(), p);
  if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
  if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p)<= -r)
    return false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
  }
  return sgn(l[a].cross(l[b], p)) < r;
}
```

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: • $(-1,-1)$ if no collision, • $(i,-1)$ if touching the corner i, • (i,i) if along side $(i,i+1)$, • (i,j) if crossing sides $(i,i+1)$ and $(j,j+1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i,i+1)$. The points are returned in the same order as the line hits the polygon. `extrVertex` returns the point of a hull with the max projection onto a line.
Usage: `auto [i, j] = lineHull(a, b, hull); // line a-b`
`int k = extrVertex(hull, dir); // max dot with dir`
Time: $\mathcal{O}(\log n)$

"Point.h" 39 lines

```
#define cmp(i,j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
int extrVertex(vector<P>& poly, P dir) {
  int n = poly.size(), lo = 0, hi = n;
  if (extr(0)) return 0;
  while (lo + 1 < hi) {
    int m = (lo + hi) / 2;
    if (extr(m)) return m;
    int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
    (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
  }
  return lo;
}

#define cmpL(i) sgn(a.cross(poly[i], b))
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
  int endA = extrVertex(poly, (a - b).perp());
  int endB = extrVertex(poly, (b - a).perp());
  if (cmpL(endA) < 0 || cmpL(endB) > 0)
    return {-1, -1};
  array<int, 2> res;
  for (int i = 0; i < 2; i++) {
    int lo = endB, hi = endA, n = poly.size();
    while ((lo + 1) % n != hi) {
      int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
      (cmpL(m) == cmpL(endB) ? lo : hi) = m;
    }
    res[i] = (lo + !cmpL(hi)) % n;
    swap(endA, endB);
  }
  if (res[0] == res[1]) return {res[0], -1};
  if (!cmpL(res[0]) && !cmpL(res[1]))
    switch ((res[0] - res[1] + (int)poly.size() + 1) %
      (int)poly.size()) {
    case 0: return {res[0], res[0]};
    case 2: return {res[1], res[1]};
    }
  return res;
}
```

HalfPlane.h

Description: Intersection of half-planes; each is a directed line $s \to e$ and keeps its left side. Returns the vertices of the convex result in ccw order, or empty if the intersection is empty or degenerate (zero area). If it may be unbounded, add the four sides of a big bounding box. Needs T = double.
Usage: `vector<array<P,2>> h = {{P(0,0), P(1,0)}, ...};`
`vector<P> poly = halfPlanes(h);`
Time: $\mathcal{O}(n \log n)$

"Point.h" 40 lines

```
typedef array<P, 2> HP;
vector<P> halfPlanes(vector<HP> h) {
  const double eps = 1e-9;
  auto dir = [](const HP& a) { return a[1] - a[0]; };
  auto out = [&](const HP& a, P p) { // p strictly right of a
    return dir(a).cross(p - a[0]) < -eps; };
  auto inter = [&](const HP& a, const HP& b) {
    P d = dir(a), e = dir(b);
    return a[0] + d * ((b[0] - a[0]).cross(e) / d.cross(e)); };
  sort(h.begin(), h.end(), [&](const HP& a, const HP& b) {
    return dir(a).angle() < dir(b).angle(); });
  deque<HP> q;
  for (HP& l : h) {
    while ((int)q.size() > 1 &&
      out(l, inter(q.back(), q[(int)q.size() - 2])))
      q.pop_back();
    while ((int)q.size() > 1 && out(l, inter(q[0], q[1])))
      q.pop_front();
    if ((int)q.size() &&
      abs(dir(l).cross(dir(q.back()))) < eps) {
      if (dir(l).dot(dir(q.back())) < 0) return {};
      if (out(l, q.back()[0])) q.pop_back();
      else continue;
    }
    q.push_back(l);
  }
  while ((int)q.size() > 2 &&
    out(q[0], inter(q.back(), q[(int)q.size() - 2])))
    q.pop_back();
  while ((int)q.size() > 2 && out(q.back(), inter(q[0], q[1])))
    q.pop_front();
  if ((int)q.size() < 3) return {};
  vector<P> res; double A = 0;
  for (int i = 0; i < q.size(); i++)
    res.push_back(inter(q[i], q[(i + 1) % (int)q.size()]));
  for (int i = 0; i < res.size(); i++)
    A += res[i].cross(res[(i + 1) % (int)res.size()]);
  if (A < eps) return {}; // empty sets can collapse to a point
  return res;
}
```

MinkowskiSum.h

Description: Minkowski sum $\{a+b : a \in A, b \in B\}$ of two convex polygons given in ccw order (e.g. from convexHull). The result is ccw and may contain collinear points. Uses: distance between convex polygons = distance from origin to $A + (-B)$; collision of a moving convex shape with an obstacle.
Usage: `vector<P> s = minkowski(convexHull(a), convexHull(b));`
Time: $\mathcal{O}(n+m)$

"Point.h" 19 lines

```
vector<P> minkowski(vector<P> a, vector<P> b) {
  auto fix = [](vector<P>& v) { // start at lowest point, pad
    auto low = [](P p, P q) {
      return tie(p.y, p.x) < tie(q.y, q.x); };
    rotate(v.begin(), min_element(v.begin(), v.end(), low),
      v.end());
    v.push_back(v[0]); v.push_back(v[1]);
  };
  fix(a); fix(b);
  vector<P> res;
  for (int i = 0, j = 0;
    i < (int)a.size() - 2 || j < (int)b.size() - 2;) {
    res.push_back(a[i] + b[j]);
    auto c = (a[i+1] - a[i]).cross(b[j+1] - b[j]);
    if (c >= 0 && i < (int)a.size() - 2) i++;
    if (c <= 0 && j < (int)b.size() - 2) j++;
  }
  return res;
}
```

8.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.
Usage: `auto [a, b] = closest(pts); // needs >= 2 points`
Time: $\mathcal{O}(n \log n)$

"Point.h" 17 lines

```
pair<P, P> closest(vector<P> v) {
  assert((int)v.size() > 1);
  set<P> S;
  sort(v.begin(), v.end(), [](P a, P b) { return a.y < b.y; });
  pair<T, pair<P, P>> ret{numeric_limits<T>::max(),
    {P(), P()}};
  int j = 0;
  for (P p : v) {
```

```
    P d{1 + (T)sqrt((double)ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
    auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
    for (; lo != hi; ++lo)
      ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
  }
  return ret.second;
}
```

ManhattanMST.h

Description: Given N points, returns up to 4*N edges, which are guaranteed to contain a minimum spanning tree for the graph with edge weights w(p, q) = |p.x - q.x| + |p.y - q.y|. Edges are in the form (distance, src, dst). Use a standard MST algorithm on the result to find the final MST. Integer coordinates (T = ll).

Usage: `auto e = manhattanMST(pts); // {w, i, j}, then Kruskal`

Time: $\mathcal{O}(N \log N)$

"Point.h" 22 lines

```
vector<array<ll, 3>> manhattanMST(vector<P> ps) {
  vector<int> id(ps.size());
  iota(id.begin(), id.end(), 0);
  vector<array<ll, 3>> edges;
  for (int k = 0; k < 4; k++) {
    sort(id.begin(), id.end(), [&](int i, int j) {
         return (ps[i]-ps[j]).x < (ps[j]-ps[i]).y;});
    map<T, int> sweep;
    for (int i : id) {
      for (auto it = sweep.lower_bound(-ps[i].y);
                it != sweep.end(); sweep.erase(it++)) {
        int j = it->second;
        P d = ps[i] - ps[j];
        if (d.y > d.x) break;
        edges.push_back({(ll)(d.y + d.x), i, j});
      }
      sweep[-ps[i].y] = i;
    }
    for (P& p : ps) if (k & 1) p.x = -p.x; else swap(p.x, p.y);
  }
  return edges;
}
```

kdTree.h

Description: KD-tree (2d, can be extended to 3d). Use T = ll for integer points.

Usage: `KDTree kd(pts); auto [d2, q] = kd.nearest(p);`

"Point.h" 63 lines

```
const T INF = numeric_limits<T>::max();

bool on_x(P a, P b) { return a.x < b.x; }
bool on_y(P a, P b) { return a.y < b.y; }

struct Node {
  P pt; // if this is a leaf, the single point in it
  T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds
  Node *first = 0, *second = 0;

  T distance(P p) { // min squared distance to a point
    T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x);
    T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y);
    return (P(x,y) - p).dist2();
  }

  Node(vector<P>&& vp) : pt(vp[0]) {
    for (P p : vp) {
      x0 = min(x0, p.x); x1 = max(x1, p.x);
      y0 = min(y0, p.y); y1 = max(y1, p.y);
    }
    if (vp.size() > 1) {
      // split on x if width >= height (not ideal...)
      sort(vp.begin(), vp.end(),
        x1 - x0 >= y1 - y0 ? on_x : on_y);
      // divide by taking half the array for each child (not
      // best performance with many duplicates in the middle)
      int half = (int)vp.size()/2;
      first = new Node({vp.begin(), vp.begin() + half});
      second = new Node({vp.begin() + half, vp.end()});
    }
  }
};

struct KDTree {
  Node* root;
  KDTree(const vector<P>& vp) :
      root(new Node({vp.begin(), vp.end()})) {}

  pair<T, P> search(Node *node, P p) {
    if (!node->first) {
      // uncomment if we should not find the point itself:
      // if (p == node->pt) return {INF, P()};
      return make_pair((p - node->pt).dist2(), node->pt);
    }

    Node *f = node->first, *s = node->second;
    T bfirst = f->distance(p), bsec = s->distance(p);
    if (bfirst > bsec) swap(bsec, bfirst), swap(f, s);

    // search closest side first, other side if needed
    auto best = search(f, p);
    if (bsec < best.first)
      best = min(best, search(s, p));
    return best;
  }

  // find nearest point to a point, and its squared distance
  // (requires an arbitrary operator< for Point)
  pair<T, P> nearest(P p) {
    return search(root, p);
  }
};
```

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Use T = ll (doubles work too, with precision issues in 'circ').

Usage: `auto t = triangulate(pts); // triangle i = t[3i..3i+2]`
Returns triangles in order
`t[0][0], t[0][1], t[0][2], t[1][0], ...`
, all counter-clockwise.

Time: $\mathcal{O}(n \log n)$

"Point.h" 90 lines

```
typedef struct Quad* Q;
typedef __int128_t lll; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX,LLONG_MAX); // not equal to any other point

struct Quad {
  Q rot, o; P p = arb; bool mark;
  P& F() { return r()->p; }
  Q& r() { return rot->rot; }
  Q prev() { return rot->o->rot; }
  Q next() { return r()->prev(); }
} *H;

bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
  lll p2 = p.dist2(), A = a.dist2()-p2,
      B = b.dist2()-p2, C = c.dist2()-p2;
  return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B > 0;
}
Q makeEdge(P orig, P dest) {
  Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};
  H = r->o; r->r()->r() = r;
  for (int i = 0; i < 4; i++)
    r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r();
  r->p = orig; r->F() = dest;
  return r;
}
void splice(Q a, Q b) {
  swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
}
Q connect(Q a, Q b) {
  Q q = makeEdge(a->F(), b->p);
  splice(q, a->next());
  splice(q->r(), b);
  return q;
}

pair<Q,Q> rec(const vector<P>& s) {
  if ((int)s.size() <= 3) {
    Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
    if ((int)s.size() == 2) return { a, a->r() };
    splice(a->r(), b);
    auto side = s[0].cross(s[1], s[2]);
    Q c = side ? connect(b, a) : 0;
    return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
  }

#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
  Q A, B, ra, rb;
  int half = (int)s.size() / 2;
  tie(ra, A) = rec({s.begin(), s.end() - half});
  tie(B, rb) = rec(
    {(int)s.size() - half + s.begin(), s.end()});
  while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
         (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
  Q base = connect(B->r(), A);
  if (A->p == ra->p) ra = base->r();
  if (B->p == rb->p) rb = base;

#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
    while (circ(e->dir->F(), H(base), e->F())) { \
      Q t = e->dir; \
      splice(e, e->prev()); \
      splice(e->r(), e->r()->prev()); \
```

```
      e->o = H; H = e; e = t; \
    }
  for (;;) {
    DEL(LC, base->r(), o);  DEL(RC, base, prev());
    if (!valid(LC) && !valid(RC)) break;
    if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
      base = connect(RC, base->r());
    else
      base = connect(base->r(), LC->r());
  }
  return { ra, rb };
}

vector<P> triangulate(vector<P> pts) {
  sort(pts.begin(), pts.end());
  assert(unique(pts.begin(), pts.end()) == pts.end());
  if ((int)pts.size() < 2) return {};
  Q e = rec(pts).first;
  vector<Q> q = {e};
  int qi = 0;
  while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
  q.push_back(c->r()); c = c->next(); } while (c != e); }
  ADD; pts.clear();
  while (qi < (int)q.size()) if (!(e = q[qi++])->mark) ADD;
  return pts;
}
```

PlanarFaces.h

Description: Faces of a planar straight-line graph (edges only meet at endpoints, no duplicate points, no multi-edges). adj[v] lists the neighbours of v (each edge in both lists). Returns the boundary walk of every face as a vertex list. Bounded faces are ccw (positive polygonArea2); each connected component also yields one walk with area ≤ 0: its outer boundary. A connected graph has $E - V + 2$ walks (Euler).

Usage: `vector<vector<int>> f = planarFaces(pts, adj);`

Time: $\mathcal{O}(E \log E)$

"Point.h" 31 lines

```
vector<vector<int>> planarFaces(
  const vector<P>& pts, vector<vector<int>> adj) {
  int n = pts.size();
  auto half = [](P p) { return p.y < 0 || (!p.y && p.x < 0); };
  map<pair<int, int>, int> id;
  for (int v = 0; v < n; v++) {
    sort(adj[v].begin(), adj[v].end(), [&](int a, int b) {
      P p = pts[a] - pts[v], q = pts[b] - pts[v];
      if (half(p) != half(q)) return half(p) < half(q);
      return p.cross(q) > 0; });
    for (int i = 0; i < adj[v].size(); i++)
      id[{v, adj[v][i]}] = i;
  }
  vector<vector<int>> used(n), faces;
  for (int v = 0; v < n; v++)
    used[v].assign((int)adj[v].size(), 0);
  for (int s = 0; s < n; s++)
    for (int i = 0; i < adj[s].size(); i++)
      if (!used[s][i]) {
        vector<int> f;
        for (int v = s, e = i; !used[v][e];) {
          used[v][e] = 1;
          f.push_back(v);
          int u = adj[v][e], j = id[{u, v}];
          v = u, e = (j + 1) % (int)adj[u].size();
        }
        reverse(f.begin(), f.end());
        faces.push_back(f);
      }
  return faces;
}
```

8.5 3D

Point3D.h

Description: Class to handle points in 3D space. T can be e.g. double or long long (same T as in Point.h).

Usage: `P3 a(1,0,0), b(0,1,0); P3 n = a.cross(b); // (0,0,1)`

31 lines

```
typedef double T;
struct P3 {
  T x, y, z;
  explicit P3(T x=0, T y=0, T z=0) : x(x), y(y), z(z) {}
  bool operator<(P3 p) const {
    return tie(x, y, z) < tie(p.x, p.y, p.z); }
  bool operator==(P3 p) const {
    return tie(x, y, z) == tie(p.x, p.y, p.z); }
  P3 operator+(P3 p) const { return P3(x+p.x, y+p.y, z+p.z); }
  P3 operator-(P3 p) const { return P3(x-p.x, y-p.y, z-p.z); }
  P3 operator*(T d) const { return P3(x*d, y*d, z*d); }
  P3 operator/(T d) const { return P3(x/d, y/d, z/d); }
  T dot(P3 p) const { return x*p.x + y*p.y + z*p.z; }
  P3 cross(P3 p) const {
    return P3(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x);
  }
  T dist2() const { return x*x + y*y + z*z; }
  double dist() const { return sqrt((double)dist2()); }
  //Azimuthal angle (longitude) to x-axis in interval [-pi, pi]
  double phi() const { return atan2(y, x); }
  //Zenith angle (latitude) to the z-axis in interval [0, pi]
  double theta() const { return atan2(sqrt(x*x+y*y),z); }
  P3 unit() const { return *this/(T)dist(); } //makes dist()=1
  //returns unit vector normal to *this and p
  P3 normal(P3 p) const { return cross(p).unit(); }
  //returns point rotated 'angle' radians ccw around axis
  P3 rotate(double angle, P3 axis) const {
    double s = sin(angle), c = cos(angle); P3 u = axis.unit();
    return u*dot(u)*(1-c) + (*this)*c - cross(u)*s;
  }
};
```

3dHull.h

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards. Face f is triangle A[f.a], A[f.b], A[f.c] with outward normal f.q.

Usage: `vector<F> fs = hull3d(pts);`

Time: $\mathcal{O}(n^2)$

"Point3D.h" 50 lines

```
struct PR {
  void ins(int x) { (a == -1 ? a : b) = x; }
  void rem(int x) { (a == x ? a : b) = -1; }
  int cnt() { return (a != -1) + (b != -1); }
  int a, b;
};

struct F { P3 q; int a, b, c; };

vector<F> hull3d(const vector<P3>& A) {
  assert((int)A.size() >= 4);
  vector<vector<PR>> E(
    (int)A.size(), vector<PR>((int)A.size(), {-1, -1}));
#define E(x,y) E[f.x][f.y]
  vector<F> FS;
  auto mf = [&](int i, int j, int k, int l) {
    P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
    if (q.dot(A[l]) > q.dot(A[i]))
      q = q * -1;
    F f{q, i, j, k};
    E(a,b).ins(k); E(a,c).ins(j); E(b,c).ins(i);
    FS.push_back(f);
  };
  for (int i = 0; i < 4; i++)
    for (int j = i + 1; j < 4; j++)
      for (int k = j + 1; k < 4; k++)
        mf(i, j, k, 6 - i - j - k);

  for (int i = 4; i < A.size(); i++) {
    for (int j = 0; j < FS.size(); j++) {
      F f = FS[j];
      if(f.q.dot(A[i]) > f.q.dot(A[f.a])) {
        E(a,b).rem(f.c);
        E(a,c).rem(f.b);
        E(b,c).rem(f.a);
        swap(FS[j--], FS.back());
        FS.pop_back();
      }
    }
    int nw = FS.size();
    for (int j = 0; j < nw; j++) {
      F f = FS[j];
#define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f.c);
      C(a, b, c); C(a, c, b); C(b, c, a);
    }
  }
  for (F& it : FS) if ((A[it.b] - A[it.a]).cross(
    A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b);
  return FS;
};
```

PolyhedronVolume.h

Description: Magic formula for the volume of a polyhedron. Faces should point outwards. Works directly on the output of hull3d.

Usage: `double V = signedPolyVolume(pts, hull3d(pts));`

"3dHull.h" 6 lines

```
double signedPolyVolume(const vector<P3>& p,
    const vector<F>& tri) {
  double v = 0;
  for (auto i : tri) v += p[i.a].cross(p[i.b]).dot(p[i.c]);
  return v / 6;
}
```

sphericalDistance.h

Description: Returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) f1 (ϕ_1) and f2 (ϕ_2) from x axis and zenith angles (latitude) t1 (θ_1) and t2 (θ_2) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. dx*radius is then the difference between the two points in the x direction and d*radius is the total distance between the points.
Usage: `double d = sphericalDistance(phi1, th1, phi2, th2, R);`

8 lines

```
double sphericalDistance(double f1, double t1,
    double f2, double t2, double radius) {
  double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
  double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
  double dz = cos(t2) - cos(t1);
  double d = sqrt(dx*dx + dy*dy + dz*dz);
  return radius*2*asin(d/2);
}
```

Strings (9)

KMP.h

Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -> 0010123). Can be used to find all occurrences of a string.
Usage: `vector<int> p = pi(s); vector<int> at = match(text, pat); // start positions`
Time: $\mathcal{O}(n)$

17 lines

```
vector<int> pi(const string& s) {
  vector<int> p(s.size());
  for (int i = 1; i < s.size(); i++) {
    int g = p[i-1];
    while (g && s[i] != s[g]) g = p[g-1];
    p[i] = g + (s[i] == s[g]);
  }
  return p;
}

vector<int> match(const string& s, const string& pat) {
  vector<int> p = pi(pat + '\0' + s), res;
  for (int i = (int)p.size()-(int)s.size(); i < p.size(); i++)
    if (p[i] == pat.size())
      res.push_back(i - 2 * (int)pat.size());
  return res;
}
```

Zfunc.h

Description: z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)
Usage: `vector<int> z = Z(s); // occurrences of p in t: Z(p + char(0) + t)`
Time: $\mathcal{O}(n)$

12 lines

```
vector<int> Z(const string& S) {
  vector<int> z(S.size());
  int l = -1, r = -1;
  for (int i = 1; i < S.size(); i++) {
    z[i] = i >= r ? 0 : min(r - i, z[i - l]);
    while (i + z[i] < (int)S.size() && S[i + z[i]] == S[z[i]])
      z[i]++;
    if (i + z[i] > r)
      l = i, r = i + z[i];
  }
  return z;
}
```

Manacher.h

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down).
Usage: `auto p = manacher(s); int k = p[1][i], e = p[0][i]; // odd: s[i-k .. i+k], even: s[i-e .. i+e) (center i-0.5)`
Time: $\mathcal{O}(N)$

14 lines

```
array<vector<int>, 2> manacher(const string& s) {
  int n = s.size();
  array<vector<int>,2> p = {vector<int>(n+1), vector<int>(n)};
  for (int z = 0; z < 2; z++)
    for (int i = 0, l = 0, r = 0; i < n; i++) {
      int t = r - i + !z;
      if (i < r) p[z][i] = min(t, p[z][l + t]);
      int L = i - p[z][i], R = i + p[z][i] - !z;
      while (L >= 1 && R + 1 < n && s[L - 1] == s[R + 1])
        p[z][i]++, L--, R++;
      if (R > r) l = L, r = R;
    }
  return p;
}
```

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.
Usage: `rotate(v.begin(), v.begin()+minRotation(v), v.end());`
Time: $\mathcal{O}(N)$

11 lines

```
int minRotation(string s) {
  int a=0, N=(int)s.size(); s += s;
  for (int b = 0; b < N; b++) for (int k = 0; k < N; k++) {
      if (a + k == b || s[a + k] < s[b + k]) {
        b += max<int>(0, k - 1);
        break;
      }
    if (s[a+k] > s[b+k]) { a = b; break; }
  }
  return a;
}
```

Lyndon.h

Description: Duval's algorithm: factors $s = w_1 w_2 \dots w_k$ into Lyndon words with $w_1 \geq w_2 \geq \dots \geq w_k$ (a Lyndon word is strictly smaller than each of its proper suffixes). Returns the start index of every factor. The minimal rotation of s starts at the last factor start $< n$ of duval(s+s) (or use MinRotation).
Usage: `vector<int> st = duval(s); // factor i = s[st[i] .. st[i+1])`
Time: $\mathcal{O}(n)$

10 lines

```
vector<int> duval(const string& s) {
  int n = s.size(), i = 0; vector<int> res;
  while (i < n) {
    int j = i + 1, k = i;
    for (; j < n && s[k] <= s[j]; j++)
      k = s[k] < s[j] ? i : k + 1;
    for (; i <= k; i += j - k) res.push_back(i);
  }
  return res;
}
```

SuffixArray.h

Description: Builds suffix array for a string. `sa[i]` is the starting index of the suffix which is i'th in the sorted suffix array. The returned vector is of size $n+1$, and `sa[0] = n`. The `lcp` array contains longest common prefixes for neighbouring strings in the suffix array: `lcp[i] = lcp(sa[i], sa[i-1])`, `lcp[0] = 0`. The input string must not contain any nul chars.
Usage: `SuffixArray S(s); S.sa[i], S.lcp[i]; // sa[0] = n (empty)`
Time: $\mathcal{O}(n \log n)$

26 lines

```
struct SuffixArray {
  vector<int> sa, lcp;
  SuffixArray(string s, int lim=256) { // or vector<int>
    s.push_back(0); int n = s.size(), k = 0, a, b;
    vector<int> x(s.begin(), s.end()), y(n), ws(max(n, lim));
    sa = lcp = y, iota(sa.begin(), sa.end(), 0);
    for (int j = 0, p = 0; p < n;
      j = max<int>(1, j * 2), lim = p) {
      p = j, iota(y.begin(), y.end(), n - j);
      for (int i = 0; i < n; i++)
        if (sa[i] >= j) y[p++] = sa[i] - j;
      fill(ws.begin(), ws.end(), 0);
      for (int i = 0; i < n; i++) ws[x[i]]++;
      for (int i = 1; i < lim; i++) ws[i] += ws[i - 1];
      for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
      swap(x, y), p = 1, x[sa[0]] = 0;
      for (int i = 1; i < n; i++)
        a = sa[i - 1], b = sa[i],
        x[b] = (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1
                                                     : p++;
    }
    for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
      for (k && k--, j = sa[x[i] - 1];
          s[i + k] == s[j + k]; k++);
  }
};
```

SuffixAutomaton.h

Description: Suffix automaton: smallest DFA accepting all substrings of s ($\leq 2n$ states). Each state is a class of substrings with the same set of end positions; len = longest of them, link = suffix link (a tree). Substring t occurs iff walking nx from 0 succeeds. #distinct substrings = $\sum_{v>0}$ len[v] − len[link[v]]. After count(), occ[v] = #occurrences of each string of state v. LCS of s, t: walk t, on a missing edge follow links (shrink the current length to len), track the max length.
Usage: `SuffixAutomaton sa(s); sa.count(); int v = 0; for (char c : t) v = sa.st[v].nx[c]; // check sa.st[v].nx.count(c)!`
Time: $\mathcal{O}(n \log \Sigma)$

39 lines

```
struct SuffixAutomaton {
  struct St {
    int len = 0, link = -1, occ = 0; map<char, int> nx;
  };
  vector<St> st{St()}; int last = 0;
  SuffixAutomaton(const string& s) { for (char c : s) add(c); }
  void add(char c) {
    int cur = st.size(), p = last;
    st.push_back({st[last].len + 1, 0, 1, {}});
    for (; p >= 0 && !st[p].nx.count(c); p = st[p].link)
      st[p].nx[c] = cur;
    if (p >= 0) {
```

```
      int q = st[p].nx[c];
      if (st[p].len + 1 == st[q].len) st[cur].link = q;
      else {
        int cl = st.size();
        st.push_back(st[q]);
        st[cl].len = st[p].len + 1, st[cl].occ = 0;
        for (; p >= 0 && st[p].nx[c] == q; p = st[p].link)
          st[p].nx[c] = cl;
        st[q].link = st[cur].link = cl;
      }
    }
    last = cur;
  }
  void count() { // occ = occurrences (call once)
    vector<int> ord(st.size());
    iota(ord.begin(), ord.end(), 0);
    sort(ord.begin(), ord.end(), [&](int a, int b) {
      return st[a].len > st[b].len; });
    for (int v : ord) if (v) st[st[v].link].occ += st[v].occ;
  }
  ll distinct() {
    ll r = 0;
    for (int v = 1; v < st.size(); v++)
      r += st[v].len - st[st[v].link].len;
    return r;
  }
};
```

Hashing.h

Description: Self-explanatory methods for string hashing.
Usage: `HashInterval h(s); H x = h.hashInterval(a, b); // [a, b)`

44 lines

```
// Arithmetic mod 2^64-1. 2x slower than mod 2^64 and more
// code, but works on evil test data (e.g. Thue-Morse, where
// ABBA... and BAAB... of length 2^10 hash the same mod 2^64).
// "typedef ull H;" instead if you think test data is random,
// or work mod 10^9+7 if the Birthday paradox is not a problem.
typedef uint64_t ull;
struct H {
  ull x; H(ull x=0) : x(x) {}
  H operator+(H o) { return x + o.x + (x + o.x < x); }
  H operator-(H o) { return *this + ~o.x; }
  H operator*(H o) { auto m = (__uint128_t)x * o.x;
    return H((ull)m) + (ull)(m >> 64); }
  ull get() const { return x + !~x; }
  bool operator==(H o) const { return get() == o.get(); }
  bool operator<(H o) const { return get() < o.get(); }
};
static const H C = (ll)1e11+3; // (order ~ 3e9; random also ok)

struct HashInterval {
  vector<H> ha, pw;
  HashInterval(string& str) : ha((int)str.size()+1), pw(ha) {
    pw[0] = 1;
    for (int i = 0; i < str.size(); i++)
      ha[i+1] = ha[i] * C + str[i],
      pw[i+1] = pw[i] * C;
  }
  H hashInterval(int a, int b) { // hash [a, b)
    return ha[b] - ha[a] * pw[b - a];
  }
};

vector<H> getHashes(string& str, int length) {
  if ((int)str.size() < length) return {};
  H h = 0, pw = 1;
  for (int i = 0; i < length; i++)
    h = h * C + str[i], pw = pw * C;
  vector<H> ret = {h};
  for (int i = length; i < str.size(); i++) {
    ret.push_back(h = h * C + str[i] - pw * str[i-length]);
  }
  return ret;
}

H hashString(string& s){H h{}; for(char c:s) h=h*C+c;return h;}
```

AhoCorasick.h

Description: Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(−, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries.
Usage: `AhoCorasick ac(pats); vector<int> r = ac.find(text);`
`auto all = ac.findAll(pats, text);`
Time: construction takes $\mathcal{O}(26N)$, where N = sum of length of patterns. find(x) is $\mathcal{O}(N)$, where N = length of x. findAll is $\mathcal{O}(NM)$.

67 lines

```
struct AhoCorasick {
  enum {alpha = 26, first = 'A'}; // change this!
  struct Node {
    // (nmatches is optional)
    int back, next[alpha], start = -1, end = -1, nmatches = 0;
    Node(int v) { memset(next, v, sizeof(next)); }
  };
  vector<Node> N;
  vector<int> backp;
  void insert(string& s, int j) {
    assert(!s.empty());
    int n = 0;
    for (char c : s) {
      int& m = N[n].next[c - first];
      if (m == -1) { n = m = N.size(); N.emplace_back(-1); }
      else n = m;
    }
    if (N[n].end == -1) N[n].start = j;
    backp.push_back(N[n].end);
    N[n].end = j;
    N[n].nmatches++;
  }
  AhoCorasick(vector<string>& pat) : N(1, -1) {
    for (int i = 0; i < pat.size(); i++) insert(pat[i], i);
    N[0].back = N.size();
    N.emplace_back(0);

    queue<int> q;
    for (q.push(0); !q.empty(); q.pop()) {
      int n = q.front(), prev = N[n].back;
      for (int i = 0; i < alpha; i++) {
        int &ed = N[n].next[i], y = N[prev].next[i];
        if (ed == -1) ed = y;
        else {
          N[ed].back = y;
          (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
            = N[y].end;
          N[ed].nmatches += N[y].nmatches;
          q.push(ed);
        }
      }
    }
  }
  vector<int> find(string word) {
    int n = 0;
    vector<int> res; // ll count = 0;
    for (char c : word) {
      n = N[n].next[c - first];
      res.push_back(N[n].end);
      // count += N[n].nmatches;
    }
    return res;
  }
  vector<vector<int>> findAll(
    vector<string>& pat, string word) {
    vector<int> r = find(word);
    vector<vector<int>> res(word.size());
    for (int i = 0; i < word.size(); i++) {
      int ind = r[i];
      while (ind != -1) {
        res[i - (int)pat[ind].size() + 1].push_back(ind);
        ind = backp[ind];
      }
    }
    return res;
  }
};
```

PalindromicTree.h

Description: Palindromic tree (eertree) of a lowercase string: one node per distinct palindromic substring (nodes 2.., so #distinct = (int)t.size()−2). Node 0 = imaginary root (len −1), 1 = empty string. link = longest proper palindromic suffix. After add(c), last = node of the longest palindrome ending at the new character. After count(), occ = #occurrences of the node's palindrome in the whole string.
Usage: `Eertree e; for (char c : s) e.add(c); e.count();`
Time: $\mathcal{O}(n)$

24 lines

```
struct Eertree {
  struct N { int len, link, occ = 0; array<int, 26> nx{}; };
  vector<N> t{{-1, 0}, {0, 0}}; string s; int last = 1;
  int get(int v) { // longest pal. suffix of v, extendable
    int i = (int)s.size() - 1;
    while (i - t[v].len < 1 || s[i - t[v].len - 1] != s[i])
      v = t[v].link;
    return v;
  }
  void add(char ch) {
    s += ch;
    int c = ch - 'a', v = get(last);
    if (!t[v].nx[c]) {
      int w = t[get(t[v].link)].nx[c];
      t.push_back({t[v].len + 2, w ? w : 1});
      t[v].nx[c] = (int)t.size() - 1;
```

```
    }
    last = t[v].nx[c]; t[last].occ++;
  }
  void count() { // call once, after all add()
    for (int v = (int)t.size() - 1; v > 1; v--)
      t[t[v].link].occ += t[v].occ;
  }
};
```

BitsetLCS.h

Description: Length of the longest common subsequence of a and b, using bit-parallel rows (D = row of the DP in difference form): $X = M_c \mid D$, $D = X \,\&\, \neg(X - ((D \ll 1) \mid 1))$ with multi-word subtraction.
Usage: `int L = lcs(a, b);`
Time: $\mathcal{O}(|a||b|/64)$

19 lines

```
typedef unsigned long long ull;
int lcs(const string& a, const string& b) {
  int W = (int)b.size() / 64 + 1, res = 0;
  vector<vector<ull>> M(256, vector<ull>(W));
  for (int i = 0; i < b.size(); i++)
    M[(unsigned char)b[i]][i / 64] |= 1ULL << i % 64;
  vector<ull> D(W);
  for (unsigned char c : a) {
    ull sh = 1, br = 0; // carry of the shift, borrow
    for (int i = 0; i < W; i++) {
      ull x = M[c][i] | D[i], y = D[i] << 1 | sh, z = x-y-br;
      sh = D[i] >> 63;
      br = x < y || (x == y && br);
      D[i] = x & ~z;
    }
  }
  for (ull d : D) res += __builtin_popcountll(d);
  return res;
}
```

CyclicLCS.h

Description: Longest common subsequence of b and any rotation of a. Builds the LCS DP of $a + a$ vs b once (pa = which neighbour each cell came from) and, for each rotation, re-roots the path tree instead of recomputing.
Usage: `int L = cyclicLCS(a, b);`
Time: $\mathcal{O}(|a||b|)$

45 lines

```
int cyclicLCS(string a, const string& b) {
  int m = a.size(), n = b.size(), ans = 0;
  a += a;
  vector<vector<int>> dp(2 * m + 1, vector<int>(n + 1)),
    pa = dp; // 1 left 2 diag 3 up
  for (int i = 0; i < 2 * m + 1; i++)
    for (int j = 0; j < n + 1; j++)
      if (i || j) {
        int& d = dp[i][j] = -1;
        if (j && dp[i][j - 1] > d)
          d = dp[i][j - 1], pa[i][j] = 1;
        if (i && j && a[i - 1] == b[j - 1] &&
          dp[i - 1][j - 1] + 1 > d)
          d = dp[i - 1][j - 1] + 1, pa[i][j] = 2;
        if (i && dp[i - 1][j] > d)
          d = dp[i - 1][j], pa[i][j] = 3;
      }
  auto trace = [&](int sx, int ex) { // #diagonals on path
    int l = 0, x = ex, y = n;
    while (x != sx || y) {
      int p = pa[x][y];
      if (p != 3) y--;
      if (p != 1) x--;
      l += p == 2;
    }
    return l;
  };
  ans = trace(0, m);
  for (int r = 1; r < m;
    r++) { // re-root: remove row r from the path tree
    int i = r, j = 1;
    while (j <= n && pa[i][j] != 2) j++;
    if (j <= n) {
      pa[i][j] = 1;
      while (i < 2*m && j < n) {
        if (pa[i+1][j] == 3) pa[++i][j] = 1;
        else if (pa[i+1][j+1] == 2) pa[++i][++j] = 1;
        else j++;
      }
      while (i < 2*m && pa[i+1][j] == 3) pa[++i][j] = 1;
    }
    ans = max(ans, trace(r, m + r));
  }
  return ans;
}
```

Various (10)

10.1 Intervals

IntervalContainer.h

Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive).
Usage: `set<pair<int, int>> is; addInterval(is, L, R); removeInterval(is, L, R);`
Time: $\mathcal{O}(\log N)$

24 lines

```
set<pair<int, int>>::iterator addInterval(
  set<pair<int, int>>& is, int L, int R) {
  if (L == R) return is.end();
  auto it = is.lower_bound({L, R}), before = it;
  while (it != is.end() && it->first <= R) {
    R = max(R, it->second);
    before = it = is.erase(it);
  }
  if (it != is.begin() && (--it)->second >= L) {
    L = min(L, it->first);
    R = max(R, it->second);
    is.erase(it);
  }
  return is.insert(before, {L,R});
}

void removeInterval(set<pair<int, int>>& is, int L, int R) {
  if (L == R) return;
  auto it = addInterval(is, L, R);
  auto r2 = it->second;
  if (it->first == L) is.erase(it);
  else (int&)it->second = L;
  if (R != r2) is.emplace(R, r2);
}
```

IntervalCover.h

Description: Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change (A) to add `|| R.empty()`. Returns empty set on failure (or if G is empty).
Usage: `vector<int> idx = cover({0, 10}, {{0, 4}, {3, 10}, {2, 5}}); // {0, 1}`
Time: $\mathcal{O}(N \log N)$

20 lines

```
typedef pair<double, double> pdd; // or pair<ll, ll>
vector<int> cover(pdd G, vector<pdd> I) {
  vector<int> S(I.size()), R;
  iota(S.begin(), S.end(), 0);
  sort(S.begin(), S.end(),
    [&](int a, int b) { return I[a] < I[b]; });
  double cur = G.first;
  int at = 0;
  while (cur < G.second) { // (A)
    pair<double, int> mx = make_pair(cur, -1);
    while (at < (int)I.size() && I[S[at]].first <= cur) {
      mx = max(mx, make_pair(I[S[at]].second, S[at]));
      at++;
    }
    if (mx.second == -1) return {};
    cur = mx.first;
    R.push_back(mx.second);
  }
  return R;
}
```

10.2 Misc. algorithms

TernarySearch.h

Description: Find the smallest i in $[a, b]$ that maximizes $f(i)$, assuming that $f(a) < \ldots < f(i) \geq \cdots \geq f(b)$. To reverse which of the sides allows non-strict inequalities, change the < marked with (A) to <=, and reverse the loop at (B). To minimize f, change it to >, also at (B).
Usage: `int ind = ternSearch(0,n-1,[&](int i){return a[i];});`
Time: $\mathcal{O}(\log(b - a))$

11 lines

```
int ternSearch(int a, int b, function<ll(int)> f) {
  assert(a <= b);
  while (b - a >= 5) {
    int mid = (a + b) / 2;
    if (f(mid) < f(mid+1)) a = mid; // (A)
    else b = mid+1;
  }
  for (int i = a + 1; i < b + 1; i++)
    if (f(a) < f(i)) a = i; // (B)
  return a;
}
```

LIS.h

Description: Compute indices for the longest increasing subsequence.
Usage: `vector<int> idx = lis(a); // a[idx[0]] < a[idx[1]] < ...`
Time: $\mathcal{O}(N \log N)$

17 lines

```
vector<int> lis(const vector<int>& S) { // ll: change int here
  if (S.empty()) return {};
```

```
  vector<int> prev(S.size());
  typedef pair<int, int> p; // and the first int here
  vector<p> res;
  for (int i = 0; i < S.size(); i++) {
    // change 0 -> i for longest non-decreasing subsequence
    auto it = lower_bound(res.begin(), res.end(), p{S[i], 0});
    if (it == res.end()) res.emplace_back(), it = res.end()-1;
    *it = {S[i], i};
    prev[i] = it == res.begin() ? 0 : (it-1)->second;
  }
  int L = res.size(), cur = res.back().second;
  vector<int> ans(L);
  while (L--) ans[L] = cur, cur = prev[cur];
  return ans;
}
```

FastKnapsack.h

Description: Given N non-negative integer weights w and a non-negative target t, computes the maximum S <= t such that S is the sum of some subset of the weights.

Usage: `int best = knapsack(w, t); // max subset sum <= t`

Time: $\mathcal{O}(N \max(w_i))$

18 lines

```
int knapsack(vector<int> w, int t) {
  int a = 0, b = 0, x;
  while (b < (int)w.size() && a + w[b] <= t) a += w[b++];
  if (b == w.size()) return a;
  int m = *max_element(w.begin(), w.end());
  vector<int> u, v(2*m, -1);
  v[a+m-t] = b;
  for (int i = b; i < w.size(); i++) {
    u = v;
    for (int x = 0; x < m; x++)
      v[x + w[i]] = max(v[x + w[i]], u[x]);
    for (x = 2 * m; --x > m;)
      for (int j = max<int>(0, u[x]); j < v[x]; j++)
        v[x - w[j]] = max(v[x - w[j]], j);
  }
  for (a = t; v[a+m-t] < 0; a--) ;
  return a;
}
```

10.3 Dynamic programming

KnuthDP.h

Description: When doing DP on intervals: $a[i][j] = \min_{i<k<j}(a[i][k] + a[k][j]) + f(i,j)$, where the (minimal) optimal k increases with both i and j, one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j-1]$ and $p[i+1][j]$. This is known as Knuth DP. Sufficient criteria for this are if $f(b,c) \le f(a,d)$ and $f(a,c) + f(b,d) \le f(a,d) + f(b,c)$ for all $a \le b \le c \le d$. Consider also: LineContainer (ch. Data structures), monotone queues, ternary search.

Time: $\mathcal{O}(N^2)$

DivideAndConquerDP.h

Description: Given $a[i] = \min_{lo(i)\le k<hi(i)}(f(i,k))$ where the (minimal) optimal k increases with i, computes $a[i]$ for $i = L..R-1$.

Usage: `DP d; d.solve(0, n); // after filling in lo, hi, f, store`

Time: $\mathcal{O}((N + (hi - lo))\log N)$

20 lines

```
struct DP { // Modify at will:
  int lo(int ind) { return 0; }
  int hi(int ind) { return ind; }
  ll f(int ind, int k) { return dp[ind][k]; }
  void store(int ind, int k, ll v) {
    res[ind] = pair<int, int>(k, v);
  }

  void rec(int L, int R, int LO, int HI) {
    if (L >= R) return;
    int mid = (L + R) >> 1;
    pair<ll, int> best(LLONG_MAX, LO);
    for (int k = max(LO,lo(mid)); k < (min(HI,hi(mid))); k++)
      best = min(best, make_pair(f(mid, k), k));
    store(mid, best.second, best.first);
    rec(L, mid, LO, best.second+1);
    rec(mid+1, R, best.second, HI);
  }
  void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
};
```

SMAWK.h

Description: Row minima of an implicit $n \times m$ totally monotone matrix (e.g. Monge: $f(i,j) + f(i',j') \le f(i,j') + f(i',j)$ for $i < i', j < j'$). Returns for every row the column of its leftmost minimum; these are non-decreasing. Use for DP transitions $dp[i] = \min_j(\dots)$ with Monge cost when D&C opt is too slow.

Usage: `vector<int> c = smawk(n, m, [&](int i, int j) { return A[i][j]; });`

Time: $\mathcal{O}(n+m)$ evaluations of f

33 lines

```
typedef function<ll(int, int)> F;
void rec(const vector<int>& rows, const vector<int>& cols,
  F& f, vector<int>& ans) {
  if (rows.empty()) return;
  vector<int> st, odd;
  for (int c : cols) { // REDUCE to at most |rows| columns
    while ((int)st.size() &&
      f(rows[(int)st.size() - 1], st.back()) >
        f(rows[(int)st.size() - 1], c))
      st.pop_back();
    if ((int)st.size() < (int)rows.size()) st.push_back(c);
  }
  for (int i = 1; i < (int)rows.size(); i += 2)
    odd.push_back(rows[i]);
  rec(odd, st, f, ans);
  for (int i = 0, k = 0; i < (int)rows.size();
    i += 2) { // INTERPOLATE
    int hi = i + 1 < (int)rows.size() ? ans[rows[i + 1]]
                                      : st.back();
    ll best = f(rows[i], st[k]); ans[rows[i]] = st[k];
    while (st[k] != hi) {
      ll v = f(rows[i], st[++k]);
      if (v < best) best = v, ans[rows[i]] = st[k];
    }
  }
}
vector<int> smawk(int n, int m, F f) {
  vector<int> rows(n), cols(m), ans(n);
  iota(rows.begin(), rows.end(), 0);
  iota(cols.begin(), cols.end(), 0);
  rec(rows, cols, f, ans);
  return ans;
}
```

SlopeTrick.h

Description: Maintains a convex piecewise linear function f (initially 0) as two heaps of breakpoints. mn $= \min f$, minimum attained on [topL(), topR()]. Operations: f += $\max(0, x-a)$, $\max(0, a-x)$, $|x-a|$; prefMin: $f(x) := \min_{y\le x} f(y)$; sufMin: $\min_{y\ge x}$; shift(a,b): $f(x) := \min_{x-b\le y\le x-a} f(y)$ ($a \le b$; shift(a,a) translates f right by a). Classic use: $\min \sum |a_i - b_i|$ with b non-decreasing: for each a_i: addAbs(a_i), prefMin(); answer mn.

Usage: `SlopeTrick s; for (ll a : v) s.addAbs(a), s.prefMin(); ll best = s.mn; // min of f; argmin in [topL(), topR()]`

Time: $\mathcal{O}(\log n)$ per operation

21 lines

```
struct SlopeTrick {
  priority_queue<ll> L;
  priority_queue<ll, vector<ll>, greater<ll>> R;
  ll addL = 0, addR = 0, mn = 0, inf = LLONG_MAX / 4;
  ll topL() { return L.empty() ? -inf : L.top() + addL; }
  ll topR() { return R.empty() ? inf : R.top() + addR; }
  void pushL(ll x) { L.push(x - addL); }
  void pushR(ll x) { R.push(x - addR); }
  void addXA(ll a) { // f += max(0, x - a)
    mn += max(0LL, topL() - a);
    pushL(a); pushR(topL()); L.pop();
  }
  void addAX(ll a) { // f += max(0, a - x)
    mn += max(0LL, a - topR());
    pushR(a); pushL(topR()); R.pop();
  }
  void addAbs(ll a) { addXA(a), addAX(a); }
  void prefMin() { R = {}; }
  void sufMin() { L = {}; }
  void shift(ll a, ll b) { addL += a, addR += b; }
};
```

10.3.1 Aliens trick (WQS binary search)

Need the best cost $g(k)$ using *exactly* k items/groups, g convex in k, but the DP without the count is easy. Add a penalty λ per item and solve `solve(lam) = {min(cost + lam*cnt), min cnt among optimal}`; cnt decreases as λ grows. Integer costs: integer λ is enough.

```
ll lo = -C, hi = C; // C > max |g(k+1) - g(k)|
while (lo < hi) {
  ll m = lo + (hi - lo) / 2;
  if (solve(m).second <= k) hi = m; else lo = m + 1;
}
ll ans = solve(lo).first - lo * k;
```

For a concave g (maximum), use $-\lambda$ or negate costs.

10.3.2 Sum over subsets, other tricks

SOS DP, $D[S] = \sum_{T\subseteq S} A[T]$ for all S in $O(K2^K)$ (start with $D = A$; supersets: test `!(i >> b & 1)` and add `D[i | 1 << b]`; inverse: subtract):

```
for (int b = 0; b < K; b++)
  for (int i = 0; i < 1 << K; i++)
    if (i >> b & 1) D[i] += D[i ^ 1 << b];
```

Bitmask DP over submasks of all masks: $O(3^n)$. Knapsack with bitset: `b |= b << w`; size known at runtime via

```
template<int L = 1> void go(int n) { // L >= n, L <= MAXN
  if (L < n) return go<min(2 * L, MAXN)>(n);
  bitset<L> b; /* ... */ }
```

Digit DP: state (position, tight, started, ...) from the most significant digit. Broken profile: process cells row-major, state = mask of the last m cells.

10.4 Debugging tricks

- `signal(SIGSEGV, [](int) { _Exit(0); });` converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). `_GLIBCXX_DEBUG` failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).
- `feenableexcept(29);` kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16).
- Compile with `-D_GLIBCXX_DEBUG` (checked `vector[]`, iterators) and `-fsanitize=address,undefined` (out of bounds, overflow, use after free).
- `#define dbg(x) cerr << #x << " = " << (x) << endl`

10.5 Optimization tricks

`__builtin_ia32_ldmxcsr(40896);` disables denormals (which make floats 20x slower near their minimum value).

10.5.1 Bit hacks

- `x & -x` is the least bit in `x`.
- `for (int x = m; x; ) { --x &= m; ... }` loops over all subset masks of `m` (except `m` itself).
- `c = x&-x, r = x+c; (((r^x) >> 2)/c) | r` is the next number after `x` with the same number of bits set.
- Sums over all subsets of every mask: SOS DP (Dynamic programming above).
- `__builtin_popcountll`, `__builtin_ctzll`, `__builtin_clzll`, `__lg(x)` (= floor log2, $x > 0$).
- Gray code: `i ^ (i >> 1)`; inverse: `int i = 0; for (; g; g >>= 1) i ^= g;`

10.5.2 Pragmas

- `#pragma GCC optimize ("Ofast")` will make GCC auto-vectorize loops and optimizes floating points better.
- `#pragma GCC target ("avx2")` can double performance of vectorized code, but causes crashes on old machines.
- `#pragma GCC optimize ("trapv")` kills the program on integer overflows (but is really slow).

FastMod.h

Description: Compute $a\%b$ about 5 times faster than usual, where b is constant but not known at compile time. Returns a value congruent to a (mod b) in the range $[0, 2b)$.
Usage: `FastMod F(b); ull r = F.reduce(a); // r = a mod b (+ b)`

8 lines

```
typedef unsigned long long ull;
struct FastMod {
  ull b, m;
  FastMod(ull b) : b(b), m(-1ULL / b) {}
  ull reduce(ull a) { // a % b + (0 or b)
    return a - (ull)((__uint128_t(m) * a) >> 64) * b;
  }
};
```

FastInput.h

Description: Read an integer from stdin. Usage requires your program to pipe in input from file.
Usage: `./a.out < input.txt`
Time: About 5x as fast as cin/scanf.

17 lines

```
inline char gc() { // like getchar()
  static char buf[1 << 16];
  static size_t bc, be;
  if (bc >= be) {
    buf[0] = 0, bc = 0;
    be = fread(buf, 1, sizeof(buf), stdin);
  }
  return buf[bc++]; // returns 0 on EOF
}

int readInt() {
  int a, c;
  while ((a = gc()) < 40);
  if (a == '-') return -readInt();
  while ((c = gc()) >= 48) a = a * 10 + c - 480;
  return a - 48;
}
```

Tips & tricks (11)

11.1 Constraints → complexity

About 10^8–10^9 simple operations per second.

$n \le 10$	$O(n!\,n)$, brute force / permutations
$n \le 20$	$O(2^n n)$, bitmask DP, $O(3^n)$ submasks up to ~ 15
$n \le 40$	meet in the middle, $O(2^{n/2})$
$n \le 500$	$O(n^3)$: Floyd, interval DP, Hungarian
$n \le 5000$	$O(n^2)$, $O(n^2 \log n)$
$n \le 10^6$	$O(n \log n)$, $O(n \log^2 n)$, $O(n\sqrt{n})$ up to $2 \cdot 10^5$
$n \le 10^8$	$O(n)$, sieve, two pointers
$n \le 10^{18}$	$O(\sqrt{n})$, $O(\log n)$: math, matrix power, digits

11.2 If you see ... think of ...

- **"minimize the maximum" / monotone answer:** binary search on the answer; for reals iterate 60–100 times instead of `while (r-l > eps)`.
- **Many binary searches over the same updates:** parallel binary search.
- **Sum over all pairs/subarrays:** contribution technique (count how often each element contributes; monotonic stack for "is the minimum of" counts).
- **Expected value:** linearity of expectation with indicator variables (no independence needed); DP backwards from the end; cyclic dependencies → linear system.
- **Counting with conditions:** complement, inclusion–exclusion, Burnside for symmetry, Möbius over gcd ($\sum_{d|g} \mu(d) = [g = 1]$).
- **Greedy ordering:** exchange argument; compare $a_i b_j < a_j b_i$ by cross-multiplication.
- **Queries offline:** sort them; process deletions in reverse as insertions (DSU); Mo's algorithm; divide & conquer over time (DynamicConnectivity, CDQ for 3D dominance).
- **Merging sets on a tree:** small-to-large (move the smaller set into the larger: $O(n \log n)$ moves); subtree = contiguous range in Euler tour order (tin/tout).
- **Paths in a tree:** LCA, $d(u, v) = d_u + d_v - 2d_{lca}$; all paths through a point: centroid decomposition; path updates: HLD; dynamic tree: LinkCutTree.
- **DP over all roots:** rerooting: down[] by DFS, then up[] using prefix/suffix combinations of the children of each vertex.
- **Shortest paths:** 0/1 weights: deque BFS; small weights: bucket queue; state graph (vertex, extra info); negative edges: Bellman–Ford or potentials (Johnson); exactly k edges: min-plus matrix power; "maximize the minimum edge": max spanning tree.
- **Matching/assignment/"choose with conflicts":** bipartite matching, König, flows, min cut (see Graph/Math), 2-SAT for binary choices with pairwise constraints.
- **Manhattan distance:** $(x, y) \to (x + y, x - y)$ turns L_1 into L_∞; $\max |x_i - x_j| + |y_i - y_j|$: max minus min of $\pm x \pm y$.
- **Linear recurrence / huge n:** matrix power, Berlekamp–Massey from brute-forced small values, then LinearRecurrence.
- **XOR of subsets:** XorBasis (Gaussian elimination over $\mathbb{F}_2$); max xor of a pair/range: binary trie.
- **Strings equality/periods:** hashing; period $p = n - \pi[n-1]$ (KMP) is a full period iff $p \mid n$; distinct substrings: suffix automaton or $\sum(n - sa_i) - lcp_i$.
- **Games:** Sprague–Grundy: g = mex of successors, sum of games = xor; Nim: first player wins iff xor $\neq 0$; misère Nim: same unless all piles ≤ 1 (then parity flips); staircase Nim: xor of odd steps; look for patterns by brute force.
- **Geometry:** keep integers and cross products as long as possible; angular sort with half-plane + cross (Angle); Pick's theorem for lattice points; rotating calipers; sweep line with events.
- **Randomization:** shuffle input against adversarial order; random 64-bit value per element and xor/sum to hash (multi)sets; random pivots; Zobrist hashing of states.
- **Bitset** speedups: transitive closure/reachability in $O(n^3/64)$, subset sums, LCS (BitsetLCS).
- **Few distinct values:** $\lfloor n/i \rfloor$ takes $O(\sqrt{n})$ values; positive integers with total sum n take $O(\sqrt{n})$ distinct values; heavy/light split at $\sqrt{m}$ (e.g. triangles in $O(m\sqrt{m})$: orient edges from low to high degree).
- **Interactive:** flush after every query (`cout << endl` or `fflush(stdout)`); count queries.

- **Stuck:** brute force small cases and look at the numbers; solve the special case (tree, path, $k = 1$); reverse time; think about the answer's structure/invariant.

11.3 C++ pitfalls

- **Overflow:** `1 << 40` → `1LL << 40`; `ll x = a * b` with int a, b overflows first: `(ll)a * b`; `accumulate(v.begin(), v.end(), 0)` sums in int: use `0LL`; `__builtin_popcount/clz/ctz` are 32-bit: use the `ll` versions; `__builtin_clz(0)` is undefined; products of two values $\leq 10^{18}$: `__int128`.
- **INF:** use ∼`LLONG_MAX/4` or `1e18` so that `INF + INF` and `INF + w` don't overflow; `memset(a, 0x3f, ...)` gives $1.06 \cdot 10^9$ per int (sum of two fits), but $4.5 \cdot 10^{18}$ per `ll` (sum overflows).
- **Division:** rounds toward zero: `-7/2 == -3`, `-7%2 == -1`; floor: `a/b - ((a%b != 0) && ((a<0) != (b<0)))`; mod: `((a%m)+m)%m`; `x%2 == 1` is false for negative odd `x`: use `x & 1`.
- **Binary search:** `(l + r) / 2` rounds toward zero for negatives → infinite loops: use `l + (r - l) / 2` or `(l + r) >> 1` (floor); also avoids `l + r` overflow.
- **Unsigned sizes:** `v.size() - 1` is huge when empty: use `(int)v.size() - 1`.
- **Precedence:** `a & b == c` is `a & (b == c)`; `1 << n - 1` is `1 << (n-1)`; parenthesize bit operations. Shifting by $\geq$ the width is undefined.
- **Floating point:** never compare with `==`; clamp before `acos/asin/sqrt` (`acos(min(1.0, max(-1.0, x)))`); `cout` prints 6 significant digits by default: `cout << fixed << setprecision(10)`; avoid printing `-0.000`; `long double` needs `%Lf`; `pow(10, k)` and `sqrt(n)` on integers are inexact: `ll r = sqrtl(n); while (r*r > n) r--; while ((r+1)*(r+1) <= n) r++;`; rounding: `llround(x)`; eps comparisons break sort comparators.
- **STL:** sort comparators must be strict (`<=` is undefined behavior and can crash); `lower_bound(s.begin(), s.end(), x)` on a set is $O(n)$, use `s.lower_bound(x)`; `m[key]` inserts a default value, use `count/find` to test; erase while iterating: `it = s.erase(it)`; `multiset::erase(x)` removes *all* copies, use `erase(find(x))`; `priority_queue` is a max-heap (`greater<>` for min); `vector<bool>` elements are proxies; `next_permutation` needs a sorted start; `unordered_map` can be hacked/slow: HashMap.
- **Strings:** `s = s + c` copies (quadratic): use `s += c`; `substr` copies; `getline` after `cin >> x` reads an empty rest of line: `cin >> ws` first; index arrays with `(unsigned char)c`.
- **Multiple test cases:** reset *all* global state, but only the used part (resetting a 10^6 array per test case is TLE); don't stop reading input early when the answer is known.
- **Stack:** deep recursion (DFS on a path of 10^6 vertices) may overflow; local big arrays belong in global scope; `function<>` lambdas have bigger frames; locally run `ulimit -s unlimited`. Bigger stack inside the program (Linux):

```
#include <ucontext.h>
static char stk[1 << 28]; // 256 MB, only used pages count
int main() { // solve() is the real main
  ucontext_t c, back; getcontext(&c);
  c.uc_stack.ss_sp = stk; c.uc_stack.ss_size = sizeof stk;
  c.uc_link = &back; makecontext(&c, solve, 0);
  swapcontext(&back, &c);
}
```

- **#define int long long** (our template): every `int` is 64-bit, so `int` arrays take twice the memory (10^7 ints = 80 MB) and are slower; literals are still 32-bit: `max(x, 0)` with `int x` does not compile (`max(x, 0LL)` or `max<int>(x, 0)`), `1 << 40` still overflows; functional casts `int(x)` do not compile (use `(int)x`); `INT_MAX` is no longer the max of `int` (`LLONG_MAX`); `__builtin_popcount/clz/ctz` silently truncate to 32 bits (use the `ll` versions); `scanf/printf` need `%lld`; *every* `#include` (`<bits/extc++.h>`, `<cassert>`, ...) must come *before* the define, otherwise system headers break ("long long long", `__assert_fail`).
- **Recursive lambdas:** `auto f = [&](auto&& self, int v) -> void { ... self(self, u); }; f(f, 0);` (faster than `function<>`).
- **I/O:** `endl` flushes (slow, but required in interactive problems), use `'\n'`; after `sync_with_stdio(0)` don't mix `cin` with `scanf`; `%lld` for `ll`; print `__int128` by converting to a string by hand.
- **Randomness:** `rand()` has `RAND_MAX` = 32767 on Windows:

```
mt19937 rng(chrono::steady_clock::now()
  .time_since_epoch().count());
int x = uniform_int_distribution<int>(1, r)(rng);
```

`random_shuffle` is deprecated (removed in C++17): `shuffle(v.begin(), v.end(), rng)`.

- **Headers:** newer GCC (seen with 16.x) no longer includes `<cassert>` in `<bits/stdc++.h>`: if `assert` is "not declared", include it. `min(a, b)` with `int` and `ll` does not compile: `min<ll>(a, b)`.
- **Misc:** `lcm` overflow: `a / gcd(a, b) * b`; `__gcd`/`gcd` with negative numbers; `abs(INT_MIN)` is undefined; global arrays are zero-initialized, local ones are not; check the output format (spaces, newlines, case, "YES"/"Yes").

11.4 Contest strategy

- Read all problems early; sort by difficulty (scoreboard helps); one person on the keyboard, others solve on paper; print code instead of debugging at the keyboard.
- Before coding: write the algorithm and its complexity, check it against the limits, check the extremes ($n = 1$, all equal, maximum values, empty input).
- Wrong answer: see troubleshoot.txt; write a brute force + generator and use stress.sh.